

TRAMA

Documentación técnica

Portal de noticias y sistema de gestión editorial

Tecnologías

Laravel 13 · Vue 3 · Inertia.js 2 · MySQL

Documento técnico del proyecto

Autor: Gastón Rizzo

Índice

1. Descripción general: qué es TRAMA	3
2. Tecnologías, entorno de desarrollo y librerías	4
3. Arquitectura general	12
4. Estructura del proyecto	15
5. Archivos de configuración y variables de entorno	19
6. Base de datos MySQL: tablas y relaciones	25
7. Cómo se conectan Laravel, Inertia y Vue	31
8. Fecha que usa TRAMA y TramaClock	37
9. Server-Side Rendering (SSR) e hidratación	40
10. Portada pública: cómo se seleccionan y organizan las noticias	44
11. Registro, autenticación, recuperación y sesiones	46
12. Roles, paneles y permisos	54
13. Dashboard del CMS y métricas según el rol	56
14. Creación de una noticia: campos y validaciones	58
15. Autoguardado y protección de cambios	63
16. Flujo entre periodista y editor	65
17. Programación, publicación, archivo y portada	70
18. Historial de la noticia, revisiones y restauración	72
19. Categorías y etiquetas	75
20. Imágenes: validación, procesamiento y recuperación	78
21. Comentarios y participación de usuarios registrados	83
22. Moderación automática de comentarios	85
23. Revisión manual de comentarios y cuentas	89
24. Bloqueo y eliminación de usuarios y empleados	95
25. Contacto y páginas informativas	101
26. Publicidad: banners, impresiones, clics y reporte	103
27. Vistas, perfiles y búsquedas	110
28. Colas, tareas programadas, transacciones y concurrencia	117
29. Manejo de errores, logs y páginas de contingencia	123
30. Instalación local de TRAMA	131
31. Cómo ejecutar TRAMA en desarrollo	139
32. Cómo publicar TRAMA en un servidor	143
33. Datos de prueba y seeders	147
34. Pruebas automatizadas y comprobaciones técnicas	154

1. Descripción general: qué es TRAMA

TRAMA es un portal de noticias en español organizado en dos grandes áreas dentro de la misma aplicación. La primera es el sitio público, donde cualquier visitante puede leer noticias y los usuarios registrados pueden participar. La segunda es el área interna, donde periodistas, editores y administradores trabajan con el contenido y con la gestión del sitio.

El proyecto reproduce un flujo editorial completo. Una noticia puede comenzar como borrador, enviarse al editor, volver al periodista con una observación, quedar programada, publicarse o archivar. Además, el sistema conserva revisiones anteriores, procesa imágenes, modera comentarios, registra métricas de publicidad y ejecuta tareas en segundo plano.

En el código existe el rol interno reader. Ese valor identifica a las cuentas creadas desde el portal. En la interfaz y en este documento se utiliza el nombre usuario registrado, porque describe mejor qué tipo de cuenta es.

Cómo está escrito este documento

Cada sección explica primero qué ocurre en TRAMA con un ejemplo práctico. Después aparecen las clases, tablas, métodos y nombres técnicos que hacen posible ese comportamiento. Los nombres del código se conservan exactamente como existen en el proyecto.

Cómo se usan los ejemplos paso a paso en esta documentación

Para que las explicaciones técnicas no queden aisladas, los capítulos siguen a personas concretas. La acción visible se presenta primero y después se explica qué ocurre dentro de Vue, Inertia, Laravel, MySQL, Node.js o los procesos auxiliares que correspondan.

- Carlos representa al desarrollador que instala, ejecuta, configura, despliega y prueba TRAMA.
- Candela representa a una usuaria registrada que navega por el portal, busca noticias y participa.
- Valentina representa a una periodista que crea, corrige y envía noticias a revisión.
- Mateo representa a un editor que revisa, programa, publica y modera.
- Marina representa a una administradora que mantiene categorías, etiquetas, publicidad y cuentas.

Paso a paso: una noticia recorre TRAMA de punta a punta

Este recorrido sirve como mapa mental del proyecto antes de entrar en clases y archivos concretos.

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina entra al CMS y crea una noticia como borrador. Vue mantiene los valores del formulario mientras escribe.
2	Al guardar, Inertia envía la petición a Laravel. Un Form Request valida los datos y el controlador delega las reglas editoriales en los servicios correspondientes.
3	Eloquent guarda la noticia y sus relaciones en MySQL. El borrador puede quedar asociado a categoría, etiquetas, portada y revisiones.
4	Valentina envía la noticia a revisión. Mateo la abre desde su panel y puede devolverla, programarla o publicarla según el estado y sus permisos.
5	Si queda programada, Laravel Scheduler revisa periódicamente si llegó la fecha de publicación y cambia el estado cuando corresponde.
6	Candela abre la noticia desde el portal. Laravel consulta los datos públicos, registra la vista y entrega a Vue la noticia, comentarios visibles, relacionadas y publicidad.

Paso	Qué hace la persona y qué ocurre internamente
7	Si Candela comenta, la petición termina rápido y la moderación automática continúa mediante una cola y un worker. Así el portal, el CMS y los procesos en segundo plano forman un único sistema.

2. Tecnologías, entorno de desarrollo y librerías

TRAMA utiliza varias tecnologías, pero cada una resuelve una tarea concreta. En esta sección se explica qué hace cada una antes de usar su nombre técnico en el resto del documento.

Tecnología	Qué hace en TRAMA
Laravel 13 / PHP	Es la parte que se ejecuta en el servidor. Recibe las peticiones, valida datos, aplica permisos, consulta MySQL, maneja sesiones, ejecuta tareas y prepara las respuestas.
Vue 3	Construye la interfaz que el usuario ve en el navegador y actualiza partes de la pantalla cuando cambian los datos.
Inertia.js 2	Permite que Laravel entregue datos directamente a las pantallas Vue sin crear una capa separada de rutas de datos para cada pantalla.
MySQL	Guarda de forma permanente usuarios, noticias, comentarios, revisiones, publicidad, sesiones y demás información del sistema.
Laravel Fortify	Aporta la base para iniciar sesión y recuperar contraseñas. TRAMA agrega sobre esa base sus propias reglas de cuentas y verificación.
Laravel Queue	Permite ejecutar trabajos en segundo plano. TRAMA lo utiliza para procesar la moderación automática de comentarios sin bloquear la petición del navegador.
Laravel Scheduler	Permite definir tareas que Laravel debe revisar de forma periódica, como la publicación de noticias programadas.
Node.js	Permite ejecutar JavaScript fuera del navegador. TRAMA lo usa para procesar imágenes, ejecutar modelos locales y generar el HTML inicial de la portada.

Qué es Node.js y para qué se usa en TRAMA

Node.js es un entorno que permite ejecutar JavaScript fuera del navegador. En TRAMA no reemplaza a Laravel: PHP y Laravel siguen siendo el servidor principal, reciben las peticiones, consultan MySQL, aplican permisos y preparan las respuestas. Node.js se utiliza cuando el proyecto necesita ejecutar herramientas o procesos escritos en JavaScript del lado del servidor.

Node.js y NPM cumplen funciones distintas. Node.js es el programa que ejecuta JavaScript; NPM es el gestor que descarga e instala las librerías de JavaScript definidas en package.json. Por eso npm install instala las dependencias y después Node.js puede ejecutar Vite, Sharp, ONNX Runtime, franc y los scripts propios de TRAMA.

En este proyecto Node.js interviene en cuatro tareas concretas:

Uso	Qué hace Node.js en TRAMA
Vite	npm run dev y npm run build ejecutan Vite con Node.js para procesar el JavaScript, Vue y CSS del frontend y generar los archivos de desarrollo o producción.

Uso	Qué hace Node.js en TRAMA
SSR	php artisan inertia:start-ssr utiliza el bundle SSR generado desde resources/js/ssr.js. Node.js ejecuta Vue en el servidor para producir el HTML inicial de la portada.
Comentarios	El Job ModerateComment inicia un proceso Node.js con scripts/comments/moderate-comment.mjs. Ese proceso analiza un comentario, devuelve un resultado JSON a Laravel y termina.
Imágenes	Laravel inicia scripts/images/process-image.mjs y scripts/images/classify-image-safety.mjs. Sharp procesa las imágenes y ONNX Runtime ejecuta la clasificación local de seguridad.

Los archivos con extensión .mjs son módulos JavaScript. Los que están dentro de scripts/ no son componentes Vue ni código que se ejecute en el navegador: son programas auxiliares que Node.js ejecuta cuando Laravel necesita realizar una tarea concreta. Para los procesos iniciados desde PHP, la ruta del ejecutable se obtiene de TRAMA_NODE_BINARY y queda disponible como config('trama.node_binary').

Composer, NPM y librerías

Composer es la herramienta que instala y mantiene las librerías PHP que necesita Laravel. Una librería es código ya preparado para resolver una tarea concreta y que el proyecto puede reutilizar en lugar de implementarla desde cero.

NPM es el gestor de paquetes de JavaScript utilizado por TRAMA. Lee package.json y se encarga de descargar, instalar y mantener las librerías que necesitan el frontend y los procesos ejecutados con Node.js. Entre esas dependencias están Chart.js, Axios, Sharp, franc, los tokenizadores de Hugging Face y ONNX Runtime.

Librería	Uso real en TRAMA
tightenco/ziggy + ziggy-js	Permiten que Vue y el SSR utilicen las rutas con nombre definidas por Laravel, sin escribir las URL manualmente.
@lucide/vue	Proporciona iconos reutilizables en layouts, botones, acciones editoriales, búsquedas y navegación.
@tanstack/vue-table	Gestiona la estructura y el ordenamiento de las grillas de Noticias, Comentarios, Usuarios, Categorías y Etiquetas.
@vuepic/vue-datepicker	Aporta los selectores de fecha y hora usados en filtros, programación de noticias y vigencia de publicidades.
Chart.js + vue-chartjs	Representan el gráfico diario de impresiones y clics del módulo de Publicidad.
Axios	Envía peticiones desde JavaScript al servidor, por ejemplo durante el autoguardado.
Sharp	Gira, recorta, redimensiona y convierte imágenes.
ONNX Runtime	Ejecuta localmente modelos de clasificación de texto e imágenes.
Hugging Face Tokenizers	Prepara el texto para algunos modelos de moderación.
Franc	Detecta automáticamente el idioma de los comentarios para apoyar el proceso de moderación.

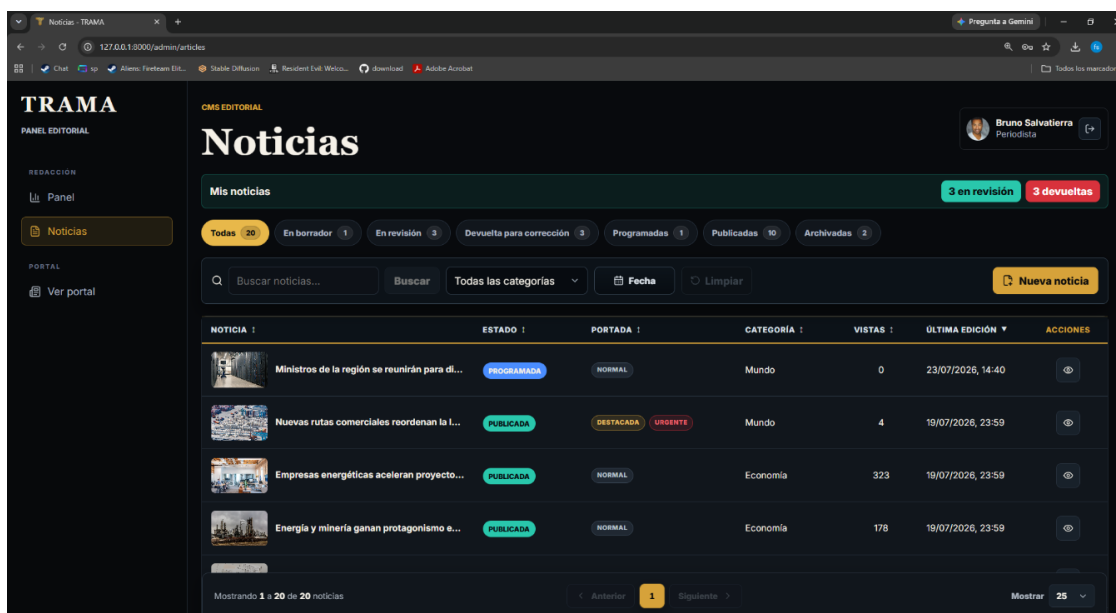
Librerías de interfaz e integraciones de Vue

La tabla anterior resume las dependencias principales, pero algunas intervienen de forma directa en la construcción de las pantallas del CMS. TRAMA utiliza TanStack Vue Table para varias grillas administrativas, Vue DatePicker para fechas y horarios, Chart.js junto con vue-chartjs para las métricas gráficas de publicidad y Ziggy/Inertia para conectar Vue con las rutas y la navegación de Laravel.

Estas herramientas no reemplazan la lógica del servidor. Laravel continúa validando los datos, aplicando permisos, consultando MySQL, paginando y decidiendo qué información se entrega. Las librerías de Vue se ocupan principalmente de la interacción y de la representación de esos datos en el navegador. Lucide Vue, por ejemplo, aporta iconos reutilizables, pero el HTML, el CSS y las reglas de negocio siguen perteneciendo a TRAMA.

TanStack Vue Table: grillas del CMS

TRAMA utiliza @tanstack/vue-table en los listados de Noticias, Comentarios, Usuarios, Categorías y Etiquetas. Es una librería *headless*, es decir, se encarga de la lógica necesaria para organizar columnas, filas, celdas y ordenamiento, pero no incluye un diseño visual predeterminado. Por eso TRAMA sigue construyendo la tabla con su propio HTML y aplicando los estilos CSS del panel editorial.



Grilla de Noticias del panel editorial.

Los ejemplos de TanStack de esta subsección se toman de `resources/js/Pages/Admin/Articles/Index.vue`. Para mostrar cómo se conecta el ordenamiento con Laravel también se utiliza `app/Http/Requests/Editorial/ArticleIndexRequest.php`. Los fragmentos conservan los nombres y estructuras reales del proyecto, pero omiten líneas que no son necesarias para explicar esta integración.

En `resources/js/Pages/Admin/Articles/Index.vue` se importan las piezas de TanStack necesarias para construir la grilla y se habilita la funcionalidad de ordenamiento utilizada por esta pantalla:

```
import {
  FlexRender,
  rowSortingFeature,
  tableFeatures,
  useTable
} from '@tanstack/vue-table';

const features = tableFeatures({
  rowSortingFeature,
});
```

En el mismo archivo se define cada columna mediante un objeto. `id` identifica la columna, `accessorFn` obtiene el valor desde la noticia recibida de Laravel, `header` define el encabezado y `enableSorting` indica si esa columna puede ordenar el listado. En las columnas ordenables, esos identificadores coinciden con los valores que `app/Http/Requests/Editorial/ArticleIndexRequest.php` admite en el parámetro `sort`.

```
{
  id: 'category',
  accessorFn: (article) =>
    article.category?.name || 'Sin categoría',
  header: 'Categoría',
  enableSorting: true,
  cell: (info) => info.getValue(),
},
```

La instancia de la tabla también se crea en `resources/js/Pages/Admin/Articles/Index.vue` y recibe las noticias de la página actual mediante `props.articles.data`. La opción `manualSorting: true` es importante: TanStack no debe ordenar sólo las filas visibles, porque la paginación y el ordenamiento completo se realizan en Laravel.

```
const tableData = computed(() => props.articles.data || []);

const table = useTable({
  key: 'editorial-articles-table',
  features,
  columns,
  data: tableData,
  manualSorting: true,
  enableMultiSort: false,
  enableSortingRemoval: false,
  state: {
    get sorting() {
      return sorting.value;
    },
  },
  onSortingChange: changeSorting,
});
```

Cuando el usuario hace clic en un encabezado, TanStack informa qué columna y qué dirección eligió. `changeSorting()` convierte esa selección en los parámetros `sort` y `direction` y TRAMA vuelve a pedir el listado mediante Inertia. Laravel aplica el orden sobre la consulta completa y devuelve la página correspondiente.

```
function changeSorting(updater) {
  const nextSorting = typeof updater === 'function'
    ? updater(sorting.value)
    : updater;

  const selected = nextSorting[0] || {
    id: 'updated_at',
    desc: true,
  };

  sorting.value = [selected];

  visitList({
    sort: selected.id,
    direction: selected.desc ? 'desc' : 'asc',
  });
}
```

Al representar la tabla, `table.getRowModel().rows` aporta las filas de la página actual. Dentro de cada fila, `row.getAllCells()` recorre sus celdas y FlexRender resuelve el contenido. TanStack organiza el modelo y el estado de la grilla; Vue sigue creando el HTML y TRAMA conserva sus propios estilos CSS.

```

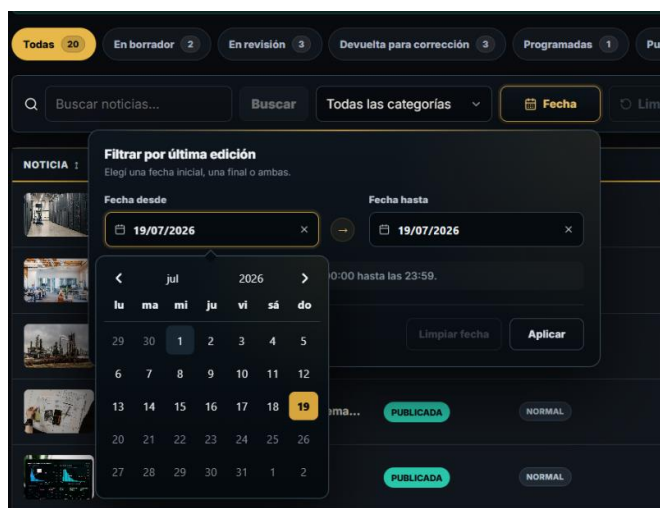
<tr
  v-for="row in table.getRowModel().rows"
  :key="row.id"
>

<td
  v-for="cell in row.getAllCells()"
  :key="cell.id"
>
  <FlexRender :cell="cell" />
</td>

```

Vue DatePicker: filtros, programación y vigencia

TRAMA utiliza @vuepic/vue-datepicker cuando una pantalla necesita seleccionar una fecha o una combinación de fecha y hora. La captura y el código siguientes se toman de `resources/js/Pages/Admin/Articles/Index.vue`, donde el componente se usa para filtrar noticias por fecha. La misma librería se reutiliza en `resources/js/Pages/Admin/Articles/Form.vue` para programar noticias y en `resources/js/Pages/Admin/Advertisements/Index.vue` para definir la vigencia de campañas. `resources/css/app.css` importa los estilos base del componente.



Selector de fechas utilizado para filtrar el listado de Noticias.

En ese filtro, el usuario ve la fecha en formato DD/MM/AAAA, mientras `model-type` mantiene el valor que recibe Vue como YYYY-MM-DD. La localización española se importa desde `date-fns/locale`.

```

const datePickerFormats = {
  input: 'dd/MM/yyyy',
};

<VueDatePicker
  v-model="form.date_from"
  model-type="yyyy-MM-dd"
  :formats="datePickerFormats"
  :locale="es"
  :max-date="dateFromMaximum"
  :time-config="{ enableTimePicker: false }"
/>

```

`v-model` vincula el calendario con el estado de Vue; `formats` controla la representación visible; `model-type` define el valor estable que puede enviarse al servidor; y `min-date` o `max-date` limitan las opciones disponibles desde la interfaz. Estas restricciones mejoran la experiencia de uso, pero Laravel vuelve a validar los valores recibidos.

En `resources/js/Pages/Admin/Articles/Form.vue`, el mismo componente utiliza `model-type="yyyy-MM-dd'T'HH:mm"`, muestra DD/MM/AAAA HH:mm y habilita la selección de hora en formato de 24 horas. En

`resources/js/Pages/Admin/Advertisements/Index.vue` se reutiliza para `starts_at` y `ends_at`. La librería resuelve la interacción del calendario, pero Laravel vuelve a validar las fechas que recibe.

Chart.js y vue-chartjs: métricas de publicidad

Cuando el administrador abre Publicidades, Laravel entrega las métricas del período y Vue recibe las fechas, impresiones y clics que alimentan el gráfico. La captura y los fragmentos siguientes se toman de `resources/js/Pages/Admin/Advertisements/Index.vue`. En ese archivo, Chart.js es el motor que construye el gráfico y vue-chartjs aporta el componente Vue Line que lo integra en la pantalla. TRAMA registra sólo los módulos que necesita para ese gráfico de líneas. La captura corresponde a una base local ya utilizada, por lo que sus acumulados pueden diferir de los valores de una carga limpia de datos de prueba documentados en la sección 33.



Gráfico de impresiones y clics del módulo Publicidades.

```
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  LineElement,
  PointElement,
  Tooltip,
  Legend
} from 'chart.js';
import { Line } from 'vue-chartjs';

ChartJS.register(
  CategoryScale,
  LinearScale,
  LineElement,
  PointElement,
  Tooltip,
  Legend,
);
```

Laravel entrega en `props.chart` las etiquetas del período, las impresiones y los clics. `buildChartData()` transforma esas series al formato que espera Chart.js y asigna impresiones y clics a ejes verticales independientes para que ambas magnitudes puedan leerse con claridad.

```
function buildChartData(chart) {
  return {
    labels: chart?.labels || [],
    datasets: [
      {
        label: 'Impresiones',
        data: chart?.impressions || [],
        yAxisID: 'yImpressions',
      },
      {
        label: 'Clicks',
        data: chart?.clicks || [],
        yAxisID: 'yClicks',
      },
    ],
  };
}
```

El template finalmente representa el gráfico con el componente `Line`. Las opciones de TRAMA utilizan dos escalas independientes porque impresiones y clics tienen magnitudes diferentes. La pantalla también conserva una firma de `props.chart` para evitar redibujar el gráfico cuando una nueva respuesta Inertia sólo cambia el orden de la grilla y las métricas siguen siendo iguales.

```
<Line
  :data="chartData"
  :options="chartOptions"
/>
```

Integraciones de Vue con Laravel: Inertia y Ziggy

`createInertiaApp()` entrega a setup el plugin de Inertia. TRAMA lo registra en la instancia de Vue mediante `.use(plugin)`. A continuación registra `ZiggyVue`, que permite que el frontend utilice las rutas con nombre definidas por Laravel.

```
return createVueApp({
  render: () => h(App, props),
})
.use(plugin)
.use(ZiggyVue)
.mount(el);
```

Gracias a Ziggy, una pantalla puede generar una dirección a partir del nombre de la ruta y sus parámetros. Por ejemplo, el autoguardado del formulario de noticias utiliza `route('admin.articles.autosave', props.article.id)` en lugar de escribir manualmente una URL con el identificador de la noticia.

La misma integración existe en `resources/js/ssr.js`. Allí Vue se crea con `createSSRApp()`, se registra el plugin de Inertia y `ZiggyVue` recibe además las rutas compartidas por Laravel. Así `route()` también puede resolverse mientras Node.js genera el HTML inicial de la portada.

Plugins utilizados durante el build con Vite

`vite.config.js` registra los plugins que intervienen durante el desarrollo y la compilación. `laravel-vite-plugin` conecta Vite con las entradas `resources/css/app.css` y `resources/js/app.js` y también declara `resources/js/ssr.js` como entrada SSR. `@vitejs/plugin-vue` permite que Vite procese los componentes `.vue`.

```

plugins: [
  laravel({
    input: ['resources/css/app.css', 'resources/js/app.js'],
    ssr: 'resources/js/ssr.js',
    refresh: true,
  }),
  vue(),
  tailwindcss(),
],

```

El archivo conserva también `@tailwindcss/vite` y `tailwind.config.js` con `@tailwindcss/forms`. Sin embargo, la interfaz actual está escrita con CSS propio: `resources/css/app.css` no contiene directivas `@tailwind` ni importa `tailwindcss`. Por esa razón Tailwind queda documentado como configuración presente en el proyecto, pero no como una tecnología de estilos de la que dependa la interfaz actual.

Entorno de desarrollo utilizado

El desarrollo de TRAMA se realizó principalmente en Visual Studio Code, utilizado para editar, organizar y depurar el código del proyecto. `composer.json` declara PHP ^8.3, pero el `composer.lock` actual fija dependencias de Symfony 8.1 que requieren PHP 8.4.1 o superior. El entorno local documentado utiliza PHP 8.5.8, que es la versión recomendada para reproducir esta instalación. Composer instala las dependencias PHP; Node.js y NPM se utilizan para Vue, Vite y los procesos auxiliares; y MySQL guarda los datos de la aplicación.

Elemento	Uso en el entorno de TRAMA
Visual Studio Code	Editor utilizado para escribir, organizar y depurar el código durante el desarrollo.
PHP 8.5.8	Versión utilizada y recomendada para reproducir el entorno documentado. Aunque <code>composer.json</code> declara PHP ^8.3, el <code>composer.lock</code> actual contiene paquetes que requieren PHP 8.4.1 o superior.
Composer	Instala y mantiene las dependencias PHP del backend Laravel.
Node.js + NPM	Node.js ejecuta Vite, SSR y los scripts auxiliares de comentarios e imágenes. NPM instala las dependencias JavaScript definidas en <code>package.json</code> , como Vue, Sharp, franc y ONNX Runtime.
MySQL	Almacena los datos permanentes de la aplicación.

Modelos de inteligencia artificial en TRAMA

TRAMA está preparado para utilizar modelos de inteligencia artificial en dos funciones concretas del sistema: la moderación automática de comentarios y la validación de imágenes. En los comentarios aportan señales para detectar spam, amenazas y toxicidad. En las imágenes permiten realizar una clasificación de seguridad antes de aceptar una portada de noticia o una fotografía de empleado.

Un modelo de inteligencia artificial es un conjunto de operaciones y parámetros obtenidos durante su entrenamiento. Cuando recibe una entrada nueva, como un comentario o una imagen, genera una predicción o varias puntuaciones. El proceso de usar un modelo ya entrenado para analizar una entrada se denomina inferencia.

Los modelos que esperan los scripts de TRAMA utilizan el formato ONNX (Open Neural Network Exchange). ONNX permite representar modelos de aprendizaje automático para ejecutarlos desde distintas herramientas. Cuando esos archivos están instalados, TRAMA utiliza ONNX Runtime desde Node.js para cargar los `.onnx` y obtener sus resultados localmente.

Comentario -> tokenización -> modelo ONNX -> puntuaciones -> reglas de TRAMA -> aprobado / pendiente / rechazado

Los modelos que analizan texto necesitan además un tokenizador, que convierte el comentario en una representación numérica que el modelo puede procesar. Después de la inferencia, los scripts de TRAMA interpretan las puntuaciones y las combinan con reglas propias. Los modelos aportan señales; la decisión final la toma la lógica del proyecto. Los scripts esperan encontrar los modelos y sus archivos auxiliares dentro de `resources/models`. Para ejecutar la inferencia, esas carpetas deben contener los archivos de los modelos y sus auxiliares correspondientes.

Estilos visuales

CSS es el lenguaje que define la apariencia visual de una página. La interfaz utiliza CSS escrito para el proyecto. La hoja principal es `resources/css/app.css` y el panel tiene archivos específicos dentro de `resources/css/admin/`, por ejemplo `article-form.css`, `comments.css`, `advertisements.css`, `categories.css`, `tags.css`, `users.css` y `editorial-listing.css`.

Paso a paso: Carlos cambia una pantalla y cada tecnología hace su trabajo

La forma más simple de entender el stack es seguir un cambio pequeño desde el código hasta el navegador.

Paso	Qué hace la persona y qué ocurre internamente
1	Carlos abre el repositorio. Composer describe las dependencias PHP de Laravel y NPM las dependencias JavaScript usadas por Vue, Inertia, Vite y los scripts de Node.js.
2	Inicia Laravel y Vite. Laravel atiende las rutas y Vite sirve los recursos del frontend durante el desarrollo.
3	Carlos modifica un componente <code>.vue</code> . Vite recompila el módulo afectado y el navegador recibe el cambio sin reconstruir manualmente todo el proyecto.
4	Cuando la pantalla necesita datos, Inertia permite que Laravel responda con props en lugar de separar la aplicación en una API independiente y otra aplicación distinta.
5	Eloquent consulta o modifica MySQL cuando la operación necesita datos persistentes.
6	Node.js entra en tareas específicas: SSR, procesamiento de imágenes y ejecución local de modelos de moderación o seguridad.
7	El resultado visible es una sola aplicación, pero cada tecnología resuelve una responsabilidad diferente.

3. Arquitectura general

TRAMA está organizada para que cada parte del sistema tenga una responsabilidad clara. Laravel controla las direcciones, el acceso, las validaciones y la lógica del servidor; MySQL conserva los datos; Inertia comunica el backend con la interfaz; y Vue representa las pantallas que utiliza el usuario. Esta separación evita concentrar rutas, permisos, consultas y presentación dentro de un único archivo.

Por ejemplo, cuando un editor abre Noticias, la petición entra por una ruta de Laravel. El middleware comprueba que la cuenta pueda acceder al panel, el controlador prepara el listado, MySQL devuelve las noticias e Inertia entrega esos datos a Vue para construir la grilla. Si después el editor ejecuta una operación que necesita varios pasos, el controlador puede delegarla a un Service en lugar de concentrar toda la lógica en una sola clase.

Ese recorrido se repite con variaciones en muchas pantallas. `routes/web.php` identifica la dirección solicitada; los middleware aplican reglas comunes; los Requests validan los datos de entrada; Controllers y Services coordinan la operación; Models/Eloquent trabajan con MySQL; y Resources preparan la información que debe llegar a la interfaz. La tabla resume el papel de cada pieza.

No todo el trabajo termina dentro de la petición que espera el navegador. La moderación de comentarios se envía a una cola para que un worker la procese en segundo plano; Laravel Scheduler revisa tareas periódicas, como las

publicaciones programadas; y Node.js interviene en procesos concretos como SSR, procesamiento de imágenes y ejecución de modelos locales. La tabla siguiente resume las piezas principales de esta arquitectura.

Antes de seguir: qué es una petición HTTP, una ruta y una respuesta

Cuando Candela pulsa una noticia, el navegador necesita pedirle algo al servidor. Esa comunicación se realiza mediante HTTP. Una petición HTTP es el mensaje que el navegador envía a Laravel para solicitar una página o ejecutar una acción. La petición incluye una dirección —la URL— y un método que indica qué tipo de operación se intenta realizar.

Laravel compara esa combinación de método y URL con las rutas definidas por el proyecto. Una ruta es la regla que relaciona una dirección con el código que debe atenderla. Si encuentra una coincidencia, la petición continúa por los middleware y llega al controlador correspondiente. El resultado que Laravel devuelve al navegador se denomina respuesta HTTP.

Ejemplo: GET /noticias/congreso-acelera-el-debate-por-la-reforma-digital significa “obtener esta noticia”. En cambio, POST /noticias/{article}/comentarios representa el envío de datos para crear una participación y PATCH se utiliza cuando TRAMA modifica parcialmente un recurso existente, como ocurre con el autoguardado.

Método	Idea principal en TRAMA
GET	Obtener una página o datos sin crear un registro nuevo. Ejemplo: abrir una noticia.
POST	Enviar datos para crear o ejecutar una acción. Ejemplo: registrar una cuenta o crear un comentario.
PATCH	Actualizar una parte de un registro existente. Ejemplo: autoguardado de una noticia.
DELETE	Solicitar la eliminación de un registro cuando la operación está permitida.

En esta documentación también aparece la palabra *endpoint*. En este contexto significa simplemente un punto de acceso HTTP del sistema: una ruta preparada para recibir una petición concreta. Cuando se habla de *payload*, se hace referencia al conjunto de datos que viaja dentro de esa petición.

Form Request y Resource: controlar lo que entra y lo que sale

Una vez que la petición llega a Laravel, TRAMA no entrega automáticamente todos los datos al controlador. En muchas operaciones utiliza clases que extienden *FormRequest*. Un *Form Request* es una clase de Laravel pensada para autorizar, preparar y validar los datos de entrada antes de que el *Controller* trabaje con ellos.

Por ejemplo, *ArticleRequest* puede comprobar si la cuenta tiene permiso para editar la noticia, normalizar textos y aplicar reglas distintas según el estado editorial solicitado. Si la validación falla, el controlador no continúa con la operación y Laravel devuelve los errores al formulario de Vue.

En la salida aparece otra pieza: los *Resources* de Laravel. Un *Resource* recibe datos que ya fueron obtenidos por Eloquent y prepara la forma exacta que debe llegar a la interfaz. No representa otra tabla ni reemplaza al *Model*: transforma y limita la información que se envía, por ejemplo para que una tarjeta pública reciba sólo los campos que necesita.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela solicita una noticia mediante una URL. <code>routes/web.php</code> reconoce la ruta.
2	Los <i>middleware</i> aplican reglas comunes antes de llegar al controlador.
3	Si la operación recibe datos, un <i>Form Request</i> puede autorizarlos, normalizarlos y validarlos.
4	El <i>Controller</i> coordina la operación y puede delegar consultas o reglas en <i>Services</i> .

Paso	Qué hace la persona y qué ocurre internamente
5	Eloquent consulta o modifica MySQL.
6	Un Resource puede transformar los datos antes de entregarlos a Inertia y Vue.
7	Laravel devuelve la respuesta HTTP y el navegador muestra el resultado.
Parte del proyecto	Responsabilidad
routes/web.php	Relaciona cada dirección del sitio con el controlador que debe atenderla y con los controles de acceso que se aplican antes.
Middleware	Son clases que Laravel ejecuta antes o después de llegar al controlador para aplicar reglas comunes, por ejemplo comprobar que una cuenta siga activa.
Controllers	Reciben la petición y deciden qué operación debe realizarse. Si la tarea necesita varios pasos, llaman a otras clases encargadas de esa operación.
Requests de validación	Comprueban los datos recibidos: longitudes, formatos, campos obligatorios, permisos y condiciones del formulario.
Services	Agrupar operaciones que necesitan varios pasos o varias tablas, por ejemplo el flujo editorial de una noticia.
Models / Eloquent	Representan los datos de MySQL y sus relaciones desde PHP.
Resources	Preparan la información que se envía a la interfaz para no exponer campos que una pantalla no necesita.
Jobs / Commands	Representan trabajos que pueden ejecutarse más tarde o desde la consola.

Paso a paso: Candela abre una noticia y atraviesa la arquitectura

El recorrido muestra cómo las capas colaboran sin convertir el controlador en una clase que hace todo.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela selecciona una noticia y el navegador solicita una URL pública de Laravel.
2	routes/web.php identifica la ruta y los middleware aplicables antes de ejecutar el controlador.
3	ArticleController recibe la petición y utiliza servicios de consulta para obtener una noticia públicamente visible.
4	Eloquent consulta MySQL y carga relaciones como autor, categoría y etiquetas.
5	Los Resources preparan solamente los campos que necesita la interfaz pública.
6	TramaBridge convierte ese resultado en una respuesta Inertia y agrega los datos compartidos que correspondan.
7	Trama.vue resuelve la pantalla concreta y ArticleShow.vue pinta la noticia. La misma arquitectura se repite con variaciones en otras funciones del proyecto.

4. Estructura del proyecto

TRAMA parte de la estructura habitual de Laravel y agrega carpetas propias para el flujo editorial, la interfaz Vue, los procesos de Node.js y la ubicación prevista para los modelos de inteligencia artificial de moderación y seguridad de imágenes. El árbol siguiente muestra las carpetas importantes para entender el proyecto y deja afuera dependencias instaladas y archivos temporales que no ayudan a explicar su funcionamiento.

Organización general

```

TRAMA/
├── app/
│   ├── Actions/Fortify/
│   ├── Console/Commands/
│   ├── Http/
│   │   ├── Controllers/{Admin, Auth, Public}/
│   │   ├── Middleware/
│   │   ├── Requests/{Admin, Editorial}/
│   │   ├── Resources/
│   │   └── Responses/
│   ├── Jobs/
│   ├── Logging/
│   ├── Mail/
│   ├── Models/
│   ├── Notifications/
│   ├── Observers/
│   ├── Providers/
│   ├── Rules/
│   ├── Services/{Categories, Editorial, Images}/
│   └── Support/
├── config/
├── database/
│   ├── migrations/
│   ├── seeders/
│   │   ├── data/
│   │   └── Support/
├── public/images/{articles, banners, brand, categories, team}/
├── resources/
│   ├── css/admin/
│   ├── js/
│   │   ├── Components/
│   │   ├── Layouts/
│   │   ├── Pages/{Admin, Auth, Public}/
│   │   ├── Support/
│   │   ├── app.js
│   │   └── ssr.js
│   ├── models/
│   │   ├── comment-spam/
│   │   ├── comment-threat-detoxify/
│   │   ├── comment-toxicity/
│   │   └── image-safety/
│   └── views/{emails, errors}/
├── routes/
├── scripts/{comments, images}/
├── storage/
├── tests/{Feature, Unit}/
├── composer.json
├── package.json
└── vite.config.js

```

Cómo orientarse dentro del proyecto

Zona	Qué reúne y cuándo conviene buscar ahí
Backend Laravel	app/Http, app/Services y app/Models contienen rutas de entrada, validaciones, coordinación de operaciones y acceso a datos.
Interfaz Vue	resources/js reúne layouts, componentes y pantallas que Inertia muestra en el navegador.
Estilos	resources/css contiene la hoja general y los estilos específicos del panel editorial.
Procesos Node.js	scripts/comments y scripts/images ejecutan moderación y procesamiento de imágenes fuera del navegador.
Modelos locales	resources/models reúne las carpetas donde los scripts esperan encontrar los modelos ONNX y sus archivos auxiliares.
Base y datos de prueba	database/migrations define la estructura; database/seeder carga y verifica el escenario de datos de prueba.
Pruebas	tests agrupa las comprobaciones automatizadas del comportamiento de TRAMA.

Piezas que reaparecen a lo largo del proyecto

No hace falta memorizar todas las clases del repositorio. Estas son las piezas que vuelven a aparecer en varias secciones porque conectan partes importantes del sistema.

Pieza	Ubicación	Por qué importa
ArticleWorkflowService	app/Services/Editorial	Coordina cambios de estado y operaciones del flujo editorial.
ModerateComment	app/Jobs	Toma comentarios de la cola y coordina su moderación automática.
TramaBridge	app/Support	Conecta Laravel con la página raíz de Inertia.
TramaClock	app/Support	Mantiene la fecha editorial del escenario de pruebas.
OperationFailureHandler	app/Support	Registra fallos inesperados y devuelve mensajes seguros a la interfaz.
Trama.vue	resources/js/Pages	Selecciona la pantalla Vue que Laravel indicó mediante Inertia.
ssr.js	resources/js	Inicia el renderizado SSR de Vue para la portada.

Ubicación prevista de los modelos locales

Dentro de `resources/models` TRAMA reserva cuatro carpetas para los modelos que intervienen en la revisión automática de comentarios y en la seguridad de imágenes. Cada carpeta corresponde a una función distinta y los scripts de Node.js buscan allí el modelo necesario cuando llega el momento de realizar la comprobación. Para ejecutar estas comprobaciones, las carpetas deben contener los archivos ONNX y auxiliares requeridos por cada modelo.

Carpeta	Función prevista en TRAMA
comment-spam	Analiza comentarios para detectar señales de spam, promociones, ofertas y llamadas a la acción.
comment-threat-detoxify	Analiza posibles amenazas y aporta una señal adicional a las reglas de contexto de la moderación.
comment-toxicity	Evalúa toxicidad, ataques e insultos dentro de los comentarios.
image-safety	Analiza imágenes para determinar si su contenido cumple las reglas de seguridad antes de aceptarlas.

Nombre	Fecha de modificación	Tipo	Tamaño
comment-spam	20/8/2026 15:54	Carpeta de archivos	
comment-threat-detoxify	20/8/2026 15:54	Carpeta de archivos	
comment-toxicity	20/8/2026 15:54	Carpeta de archivos	
image-safety	20/8/2026 15:54	Carpeta de archivos	

Carpetas reservadas para los modelos locales dentro de `resources/models`.

Para funcionar, un modelo de texto necesita el archivo ONNX y archivos auxiliares que describen su configuración y la forma de convertir texto en tokens. Por ejemplo, `comment-threat-detoxify` espera una estructura como la siguiente cuando sus archivos están instalados:

Archivo requerido	Función
model.quant.onnx	Modelo ONNX cuantizado que ONNX Runtime carga para realizar la inferencia.
tokenizer.json	Define cómo se divide y transforma el texto en tokens que el modelo puede procesar.
tokenizer_config.json	Guarda parámetros adicionales utilizados al inicializar el tokenizador.
config.json	Describe parámetros generales y metadatos necesarios para interpretar el modelo.

Nombre	Fecha de modificación	Tipo	Tamaño
config.json	18/8/2026 17:44	Archivo JSON	2 KB
model.quant.onnx	18/8/2026 17:44	Archivo ONNX	272.272 KB
tokenizer.json	18/8/2026 17:44	Archivo JSON	16.683 KB
tokenizer_config.json	18/8/2026 17:44	Archivo JSON	2 KB

Ejemplo de archivos requeridos por comment-threat-detoxify cuando el modelo está instalado.

Archivos de entrada y navegación del frontend

Además de las carpetas principales, hay cuatro piezas que ayudan a entender cómo entra una respuesta de Laravel en la aplicación Vue y cómo se mantiene la interfaz pública durante la navegación Inertia.

Archivo	Función dentro de TRAMA
resources/views/app.blade.php	Documento HTML base donde Inertia inserta la aplicación. En la portada también recibe el HTML generado por SSR.
resources/js/inertia-pages.js	Resolver de Inertia. Acepta únicamente la página raíz Trama y le asigna el layout persistente.
resources/js/Layouts/PersistentInertiaLayout.vue	Mantiene el layout público montado mientras se navega entre pantallas Public/ y deja pasar sin ese layout las pantallas Admin/, Auth/ y las páginas ficticias de anunciantes.
lang/es	Reúne textos de idioma utilizados por Laravel para mensajes y validaciones cuando corresponde.

Paso a paso: Carlos quiere cambiar una regla y sabe dónde buscar

La estructura de carpetas deja de ser una lista abstracta cuando se usa para resolver una tarea concreta.

Paso	Qué hace la persona y qué ocurre internamente
1	Carlos quiere cambiar una regla del formulario de noticias. Primero mira resources/js/Pages/Admin/Articles/Form.vue para entender qué ve y qué envía la interfaz.
2	Si la regla debe impedir que datos inválidos lleguen al sistema, revisa app/Http/Requests/Admin/ArticleRequest.php.
3	Si cambia el flujo de estados o la forma de guardar una noticia, busca ArticleWorkflowService dentro de app/Services/Editorial.
4	Si necesita comprender columnas o relaciones, revisa el modelo Article y las migraciones de database/migrations.
5	Si la operación tiene una ruta propia, busca su definición en routes/web.php y el controlador correspondiente en app/Http/Controllers.
6	Si el cambio debe comprobarse automáticamente, localiza o agrega una prueba en tests/Feature o tests/Unit.
7	Así la carpeta correcta se elige por responsabilidad: interfaz, entrada HTTP, regla de negocio, persistencia o prueba.

5. Archivos de configuración y variables de entorno

TRAMA separa la configuración del código para que una misma base del proyecto pueda ejecutarse en distintos equipos. Datos que cambian según el entorno —como la conexión a MySQL, el correo SMTP, las rutas de archivos, el ejecutable de Node.js o el servidor SSR— se definen fuera de las clases y después Laravel los organiza mediante sus archivos de config/.

.env y .env.example

El archivo `.env` contiene los valores que TRAMA usa en el equipo donde se ejecuta. Ahí se define, por ejemplo, cómo conectarse a MySQL, qué URL usa la aplicación, cómo se envía correo, qué rutas de archivos utiliza el proyecto, dónde está el ejecutable de Node.js y cómo se conecta Laravel con el servidor SSR. También contiene valores sensibles como `APP_KEY`, `DB_PASSWORD` o `MAIL_PASSWORD`.

El archivo `.env.example` no es la configuración activa. Es la referencia del proyecto: muestra qué variables espera TRAMA y qué formato tienen, sin incluir contraseñas ni claves reales. El `.env` de cada equipo puede usar valores distintos en campos como `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, las credenciales SMTP o `TRAMA_NODE_BINARY`.

```
APP_NAME=TRAMA
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000
APP_TIMEZONE=America/Argentina/Buenos_Aires

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=<puerto-mysql>
DB_DATABASE=<base-de-datos>
DB_USERNAME=<usuario>
DB_PASSWORD=<contraseña>

SESSION_DRIVER=database
SESSION_LIFETIME=120
QUEUE_CONNECTION=database
CACHE_STORE=database

MAIL_MAILER=smtp
MAIL_HOST=<servidor-smtp>
MAIL_PORT=<puerto-smtp>
MAIL_USERNAME=<usuario-smtp>
MAIL_PASSWORD=<contraseña-smtp>
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS="noticias@trama.local"
MAIL_FROM_NAME="TRAMA"
TRAMA_CONTACT_EMAIL=<correo-contacto>

TRAMA_REFERENCE_DATE=2026-07-19
TRAMA_REFERENCE_TIME=15:30:00
TRAMA_LOG_PATH=storage/logs
TRAMA_ARTICLE_COVERS_PATH=public/images/articles
TRAMA_BANNERS_PATH=public/images/banners
TRAMA_NODE_BINARY=<ruta-al-ejecutable-node>

INERTIA_SSR_ENABLED=true
INERTIA_SSR_URL=http://127.0.0.1:13714
INERTIA_SSR_ENSURE_BUNDLE_EXISTS=true
INERTIA_SSR_BUNDLE=bootstrap/ssr/ssr.js
VITE_APP_NAME="${APP_NAME}"
```

Los valores escritos entre `< >` en el ejemplo representan datos que cambian según el equipo. En el entorno documentado, las sesiones, las colas y la caché utilizan la base de datos; las portadas y los banners se guardan dentro del proyecto; y `TRAMA_NODE_BINARY` debe apuntar al ejecutable de Node.js disponible en ese equipo.

Qué hace cada grupo de variables principales

El bloque anterior deja reunida la configuración principal, pero cada variable cumple una responsabilidad distinta. Las siguientes son las familias que intervienen realmente en el funcionamiento documentado de TRAMA. Las opciones de Redis, Memcached o AWS que también aparecen en `.env.example` pertenecen a alternativas generales de Laravel y no forman parte del entorno local principal de este proyecto.

Grupo	Variables principales	Qué controlan en TRAMA
Aplicación	APP_NAME, APP_ENV, APP_DEBUG, APP_URL, APP_TIMEZONE, APP_KEY	Identifican la aplicación y el entorno, definen URL y zona horaria y controlan diagnóstico y funciones criptográficas. APP_KEY es secreta y se genera para cada instalación.
MySQL	DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD	Indican motor, servidor, puerto, base, usuario y contraseña. config/database.php usa estos valores para construir la conexión utilizada por Eloquent y Query Builder.
Sesiones	SESSION_DRIVER, SESSION_LIFETIME	Con SESSION_DRIVER=database Laravel guarda las sesiones en la tabla sessions. SESSION_LIFETIME define cuántos minutos puede permanecer válida una sesión inactiva según la configuración del framework.
Colas	QUEUE_CONNECTION	Con database, los Jobs pendientes se almacenan en la infraestructura de colas de MySQL. ModerateComment usa esta conexión y un worker los consume después.
Caché	CACHE_STORE	Con database, Laravel utiliza las tablas de caché para valores temporales y locks que no necesitan guardarse como información editorial permanente.
Correo	MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_ENCRYPTION, MAIL_FROM_ADDRESS, MAIL_FROM_NAME, TRAMA_CONTACT_EMAIL	Configuran SMTP, el remitente y el destinatario interno de Contacto. Registro, recuperación de contraseña y Contacto usan el mailer configurado.
Reloj editorial	TRAMA_REFERENCE_DATE, TRAMA_REFERENCE_TIME	Fijan el punto inicial de TramaClock para mantener coherente el escenario de demostración. No reemplazan APP_TIMEZONE: resuelven otra responsabilidad.
Logs	LOG_CHANNEL, LOG_STACK, LOG_LEVEL, TRAMA_LOG_PATH	Controlan la infraestructura de logging. TRAMA_LOG_PATH participa en el canal diario trama_unexpected explicado en la sección 29.
Archivos	TRAMA_ARTICLE_COVERS_PATH, TRAMA_BANNERS_PATH, FILESYSTEM_DISK	Definen las ubicaciones físicas administradas por el proyecto y el disco predeterminado de Laravel. config/trama.php normaliza las carpetas de portadas y banners.

Grupo	Variables principales	Qué controlan en TRAMA
Node.js	TRAMA_NODE_BINARY	Indica el ejecutable de Node que PHP inicia con <code>Process::run()</code> para moderación de comentarios y procesamiento/clasificación de imágenes.
SSR	INERTIA_SSR_ENABLED, INERTIA_SSR_URL, INERTIA_SSR_ENSURE_BUNDL E_EXISTS, INERTIA_SSR_BUNDLE	Activan SSR, indican dónde escucha el proceso Node y qué bundle compilado debe existir. TRAMA aplica SSR únicamente a la portada.
Frontend	VITE_APP_NAME	Vite expone este valor al JavaScript del cliente y al entry SSR. Las variables <code>VITE_*</code> pueden terminar en el navegador y nunca deben contener secretos.

Una variable no “hace” nada por el simple hecho de existir en `.env`. Laravel debe leerla desde un archivo de `config/` y convertirla en una opción que después consumen Controllers, Services, Jobs, Middleware o el frontend. Por eso conviene seguir el recorrido completo y no memorizar nombres aislados.

Cómo llegan las variables al código

Laravel carga las variables de entorno al iniciar la aplicación. En TRAMA, los archivos de `config/` utilizan `env()` para convertir esos valores en configuración de la aplicación. Después, el resto del código consulta `config()` en lugar de depender directamente del archivo `.env`. Esta separación permite gestionar esos valores desde los archivos de configuración, definir valores por defecto y utilizar la caché de configuración de Laravel.

```
.env
↓
config/*.php -> env('NOMBRE_VARIABLE')
↓
config('grupo.clave')
↓
Controllers / Services / Jobs / Middleware
```

Por ejemplo, `config/trama.php` lee `TRAMA_NODE_BINARY` y crea la clave `trama.node_binary`. `ModerateComment` y los servicios de imágenes consultan después `config('trama.node_binary')` sin depender directamente del archivo `.env`. Si la configuración está cacheada, Laravel continúa usando los valores guardados en esa caché hasta que se actualice.

Una caché es un almacenamiento temporal que conserva un resultado ya preparado para evitar repetir trabajo innecesario. En este caso, la caché de configuración permite que Laravel reutilice la configuración procesada. Por eso, si Carlos cambia `.env` mientras esa caché está activa, puede ser necesario limpiarla o reconstruirla para que el nuevo valor se lea.

Archivos de configuración que conviene ubicar

Archivo	Qué conecta	Uso concreto en TRAMA
<code>config/trama.php</code>	Reloj editorial, rutas de imágenes, Node.js y política SSR	Define la configuración propia del proyecto.
<code>config/inertia.php</code>	<code>INERTIA_SSR_*</code>	Configura la URL y el bundle que utiliza el proceso SSR.

Archivo	Qué conecta	Uso concreto en TRAMA
config/database.php	DB_*	Construye la conexión MySQL usada por Eloquent y Query Builder.
config/mail.php	MAIL_* y TRAMA_CONTACT_EMAIL	Configura SMTP y el destinatario interno de Contacto.
config/logging.php	LOG_* y TRAMA_LOG_PATH	Configura los logs generales y el canal trama_unexpected.
config/fortify.php	guard, broker y features	Define la base de autenticación que TRAMA complementa con flujos propios.
vite.config.js	Entradas del cliente y SSR	Conecta Laravel, Vue y el proceso de build de Vite.

Las variables DB_*, MAIL_*, SESSION_*, QUEUE_* y otras opciones generales siguen la configuración habitual de Laravel. TRAMA añade sobre esa base variables propias para su reloj, rutas físicas, Node.js, Contacto y SSR. Que .env.example incluya opciones como Redis, Memcached o AWS no significa que formen parte del entorno local actual: quedan disponibles como alternativas del framework.

Ejemplo concreto: cómo TRAMA construye la conexión MySQL

Carlos no escribe el puerto, usuario o contraseña dentro de cada Model. Los define una sola vez en .env. config/database.php lee DB_CONNECTION y DB_* y prepara la conexión mysql que después reutiliza toda la capa de persistencia.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=<puerto-mysql>
DB_DATABASE=<base-de-datos>
DB_USERNAME=<usuario>
DB_PASSWORD=<contraseña>
↓
config/database.php
↓
conexión mysql
↓
Eloquent / Query Builder
↓
MySQL
```

DB_HOST identifica el servidor donde escucha MySQL; DB_PORT indica el puerto de ese servicio; DB_DATABASE el nombre de la base; y DB_USERNAME/DB_PASSWORD las credenciales. Si alguno de esos datos no coincide con la instalación real, Laravel no puede abrir la conexión aunque el código de los Models sea correcto.

Ejemplo concreto: cómo la configuración de correo llega a Contacto

config/mail.php toma las variables MAIL_* para construir el mailer SMTP. MAIL_FROM_ADDRESS y MAIL_FROM_NAME definen el remitente general, mientras TRAMA_CONTACT_EMAIL alimenta la clave mail.contact_address que usa el formulario público de Contacto como destinatario interno.

```
MAIL_MAILER=smtp
MAIL_HOST=<servidor-smtp>
MAIL_PORT=<puerto-smtp>
MAIL_USERNAME=<usuario-smtp>
MAIL_PASSWORD=<contraseña-smtp>
MAIL_ENCRYPTION=tls
TRAMA_CONTACT_EMAIL=<correo-contacto>
↓
config/mail.php
↓
mailer SMTP + mail.contact_address
↓
registro / recuperación / Contacto
```

Ejemplo concreto: qué controla la configuración SSR

Las variables `INERTIA_SSR_*` tampoco arrancan un servidor por sí solas. `config/inertia.php` las utiliza para saber si SSR está habilitado, dónde se encuentra el proceso Node y qué bundle compilado debe existir. El proceso se inicia después con `php artisan inertia:start-ssr`, como se explica en las secciones 9 y 31.

```
INERTIA_SSR_ENABLED=true
INERTIA_SSR_URL=http://127.0.0.1:13714
INERTIA_SSR_ENSURE_BUNDLE_EXISTS=true
INERTIA_SSR_BUNDLE=bootstrap/ssr/ssr.js
↓
config/inertia.php
↓
Laravel conoce estado, URL y bundle de SSR
↓
php artisan inertia:start-ssr
```

Ejemplo concreto: cómo TRAMA encuentra Node.js

`TRAMA_NODE_BINARY` muestra el recorrido completo sin necesidad de repetir el mismo esquema para cada variable. El valor se define en `.env`, `config/trama.php` lo convierte en `trama.node_binary` y las clases PHP lo consultan cuando necesitan lanzar los scripts internos de moderación o de imágenes. Esta variable no configura Vite ni el servidor SSR; esos procesos tienen su propia configuración.

```
TRAMA_NODE_BINARY=<ruta-al-ejecutable-node>
↓
config/trama.php
↓
config('trama.node_binary')
↓
ModerateComment / servicios de imágenes
↓
scripts de Node.js
```

Variables que llegan al navegador

Vite puede exponer al frontend las variables cuyo nombre comienza con `VITE_`. En TRAMA, `VITE_APP_NAME` se lee desde `resources/js/app.js` y `resources/js/ssr.js` mediante `import.meta.env.VITE_APP_NAME`. Esta regla es importante desde el punto de vista de seguridad: una variable `VITE_` puede terminar dentro del JavaScript entregado al navegador, por lo que nunca debe contener contraseñas, `APP_KEY`, credenciales SMTP o secretos de base de datos.

Seguridad de la configuración

- `APP_KEY`, `DB_PASSWORD` y `MAIL_PASSWORD` pertenecen a la configuración del servidor y no se envían al frontend.
- `APP_KEY` es la clave que Laravel utiliza para sus funciones criptográficas. El código la obtiene desde la configuración y no la escribe de forma fija dentro de una clase.

- APP_DEBUG debe permanecer desactivado en producción para que una excepción no muestre información interna al visitante.
- Las variables VITE_* pueden incorporarse al JavaScript que recibe el navegador; por eso sólo deben contener datos que puedan ser visibles en el cliente.
- Rutas como TRAMA_NODE_BINARY, TRAMA_ARTICLE_COVERS_PATH y TRAMA_BANNERS_PATH se ajustan desde la configuración del entorno y no se escriben dentro de controladores o servicios.

La idea es simple: .env guarda la configuración que TRAMA usa en ese entorno, config/ la organiza y el código de la aplicación la consulta mediante config(). Así una misma base de código puede ejecutarse con distintas conexiones, rutas o servicios sin modificar controladores, servicios o jobs.

Comprobación de existencia del bundle SSR

TRAMA incorpora una protección adicional para SSR. La variable INERTIA_SSR_ENSURE_BUNDLE_EXISTS indica a Inertia que debe comprobar que el archivo compilado del servidor exista antes de intentar utilizarlo. Esto evita solicitar renderizado a Node.js cuando todavía no se generó el bundle.

```
INERTIA_SSR_ENSURE_BUNDLE_EXISTS=true
```

El archivo fuente sigue siendo resources/js/ssr.js. El bundle configurado en INERTIA_SSR_BUNDLE es el resultado compilado que Node utiliza durante el renderizado del servidor.

Paso a paso: Carlos cambia una variable y sigue su recorrido hasta el código

Las variables de entorno sólo resultan útiles cuando se entiende cómo pasan de .env a las clases que finalmente las consumen.

Paso	Qué hace la persona y qué ocurre internamente
1	Carlos define un valor en .env, por ejemplo TRAMA_NODE_BINARY o una credencial de MySQL. Ese archivo no se publica en Git.
2	config/trama.php o el archivo de configuración correspondiente lee el valor mediante env() y lo convierte en una opción de configuración de Laravel.
3	El resto del código consulta config(...) en lugar de volver a leer .env directamente en cada clase.
4	Cuando una clase necesita iniciar Node.js, usa la ruta ya normalizada de configuración. Cuando Eloquent necesita la base, Laravel usa las opciones de config/database.php.
5	Si una variable se comparte con el navegador, debe pasar explícitamente por una configuración o prop diseñada para ser pública; los secretos permanecen en el servidor.
6	En producción, cambiar .env puede requerir limpiar o reconstruir la caché de configuración para que Laravel vuelva a leer los valores correctos.

6. Base de datos MySQL: tablas y relaciones

TRAMA guarda la información permanente en MySQL. MySQL es una base de datos relacional: la información se divide en tablas y esas tablas se conectan mediante identificadores. De esta forma un usuario, una noticia, una categoría o un comentario pueden guardarse por separado sin perder la relación que existe entre ellos.

Ejemplo básico: una noticia, su autor y su categoría

```

users
id | name
2  | Mateo Ledesma

categories
id | name
1  | Política

articles
id | author_id | category_id | title
1  | 2         | 1           | Congreso acelera el debate por la reforma digital

```

La noticia 1 tiene `author_id = 2`, por lo que su autor es Mateo Ledesma, la fila `users.id = 2`. También tiene `category_id = 1`, por lo que pertenece a Política, la fila `categories.id = 1`. Este ejemplo corresponde a datos cargados por los seeders del proyecto.

`id` es la clave primaria: identifica de manera única una fila. `author_id` y `category_id` son claves foráneas: guardan identificadores que apuntan a filas de otras tablas. MySQL puede comprobar que esas referencias sean válidas.

Migraciones: cómo se crea la estructura

Laravel llama migraciones a los archivos que describen cómo crear o modificar la estructura de la base. Están en `database/migrations`. Una migración puede crear una tabla, agregar una columna, crear un índice o definir una relación. Esto permite reconstruir la estructura sin tener que crear cada tabla manualmente desde MySQL.

```

$table->foreignId('author_id')
    ->constrained('users')
    ->restrictOnDelete();

```

Ese fragmento crea `author_id`, indica que debe apuntar a `users` y establece qué debe pasar si alguien intenta eliminar al usuario relacionado.

Tablas actuales

Grupo	Tablas y función principal
Usuarios y acceso	<code>users</code> · <code>password_reset_tokens</code> · <code>sessions</code>
Noticias y clasificación	<code>articles</code> · <code>categories</code> · <code>tags</code> · <code>article_tag</code>
Historial editorial	<code>article_revisions</code> · <code>article_review_feedback</code> · <code>media_assets</code>
Participación	<code>comments</code> · <code>comment_likes</code> · <code>comment_reports</code> · <code>user_moderation_reviews</code>
Lecturas	<code>article_views</code>
Publicidad	<code>advertisements</code> · <code>advertisement_daily_metrics</code>
Contacto	<code>contact_messages</code>
Reloj de TRAMA	<code>trama_clock_state</code>
Trabajos en segundo plano	<code>jobs</code> · <code>failed_jobs</code> · <code>job_batches</code>
Caché	<code>cache</code> · <code>cache_locks</code>
Control de migraciones	<code>migrations</code>

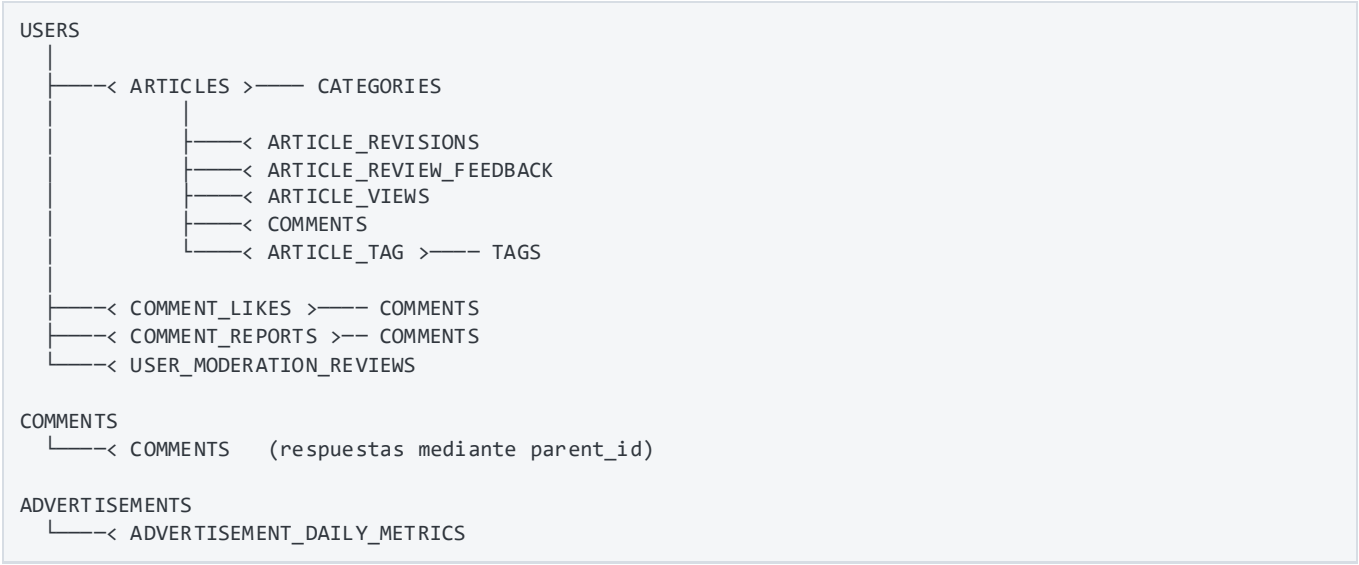
Infraestructura interna de Laravel

Además de las tablas que representan contenido propio de TRAMA, Laravel mantiene tablas internas para sesiones (sessions), recuperación de contraseñas (password_reset_tokens), colas (jobs, failed_jobs y job_batches), caché (cache y cache_locks) y control de migraciones (migrations). Participan en el funcionamiento del sitio, pero no representan información editorial del portal.

Tablas del dominio que más importan

Tabla	Qué guarda
users	Cuentas públicas e internas, rol, perfil, estado de acceso y datos administrativos actuales.
articles	Estado actual de cada noticia: autor, categoría, contenido, portada, estado, marcas, vistas y fechas.
categories	Secciones del diario y sus datos públicos.
tags	Temas reutilizables y la referencia a una etiqueta principal cuando hubo una fusión.
article_tag	Cada combinación entre una noticia y una etiqueta.
article_revisions	Versiones históricas y acciones registradas sobre una noticia.
article_review_feedback	Observaciones que un editor deja al devolver una noticia.
media_assets	Archivos visuales relacionados con artículos y datos sobre quién los cargó.
comments	Comentarios principales y respuestas, autor y estado de moderación.
comment_likes	Me gusta de usuarios sobre comentarios.
comment_reports	Reportes de comentarios y su resolución.
user_moderation_reviews	Casos en los que un editor derivó una cuenta al administrador.
article_views	Cada carga registrada de una noticia.
advertisements	Campañas publicitarias.
advertisement_daily_metrics	Impresiones y clics por publicidad y fecha.
contact_messages	Consultas enviadas desde Contacto.
trama_clock_state	Estado necesario para continuar TramaClock entre sesiones.

Mapa simplificado de relaciones



Relaciones editoriales principales

Relación	Qué representa	Al eliminar
articles.author_id → users.id	Autor de la noticia	RESTRICT
articles.category_id → categories.id	Categoría de la noticia	RESTRICT
article_tag.article_id → articles.id	Noticia relacionada con una etiqueta	CASCADE
article_tag.tag_id → tags.id	Etiqueta relacionada con una noticia	CASCADE
media_assets.article_id → articles.id	Archivo visual asociado a una noticia	SET NULL
media_assets.uploaded_by → users.id	Cuenta que cargó el archivo	SET NULL
article_revisions.article_id → articles.id	Revisiones de una noticia	CASCADE
article_revisions.user_id → users.id	Cuenta que realizó la revisión	RESTRICT
article_revisions.restored_from_revision_id → article_revisions.id	Revisión usada como origen de una restauración	SET NULL
article_review_feedback.article_id → articles.id	Devoluciones de una noticia	CASCADE
article_review_feedback.returned_by → users.id	Editor que devolvió la noticia	RESTRICT
article_review_feedback.resolved_by → users.id	Cuenta que resolvió la devolución	SET NULL
article_views.article_id → articles.id	Vista registrada de una noticia	CASCADE

Relación	Qué representa	Al eliminar
tags.merged_into_id → tags.id	Etiqueta principal después de una fusión	SET NULL

Relaciones de comentarios y administración

Relación	Qué representa	Al eliminar
comments.article_id → articles.id	Comentario de una noticia	CASCADE
comments.user_id → users.id	Cuenta que escribió el comentario	SET NULL
comments.parent_id → comments.id	Respuesta a un comentario principal	CASCADE
comment_likes.comment_id → comments.id	Me gusta sobre un comentario	CASCADE
comment_likes.user_id → users.id	Cuenta que dio Me gusta	CASCADE
comment_reports.comment_id → comments.id	Reporte sobre un comentario	CASCADE
comment_reports.user_id → users.id	Cuenta que hizo el reporte	CASCADE
comment_reports.reviewed_by → users.id	Editor que revisó el reporte	SET NULL
user_moderation_reviews.user_id → users.id	Cuenta revisada por administración	CASCADE
user_moderation_reviews.requested_by_id → users.id	Editor que pidió la revisión	SET NULL
user_moderation_reviews.comment_id → comments.id	Comentario usado como evidencia	SET NULL
user_moderation_reviews.resolved_by_id → users.id	Administrador que resolvió el caso	SET NULL
users.moderation_review_requested_by_id → users.id	Último editor que pidió revisar la cuenta	SET NULL
users.moderation_review_comment_id → comments.id	Comentario asociado al estado administrativo actual	SET NULL
users.moderation_review_resolved_by_id → users.id	Administrador que resolvió el estado actual	SET NULL
advertisement_daily_metrics.advertisement_id → advertisements.id	Métricas de una campaña	CASCADE

Qué significan RESTRICT, CASCADE y SET NULL

RESTRICT impide borrar un registro cuando hacerlo rompería una relación que TRAMA necesita conservar. Por ejemplo, una categoría con noticias no puede eliminarse.

CASCADE elimina datos que ya no tienen sentido por sí solos. Si se elimina un comentario, sus Me gusta y reportes también se eliminan.

SET NULL conserva el registro pero deja vacía la referencia. Eso permite, por ejemplo, mantener un comentario aunque la cuenta registrada que lo escribió haya sido eliminada.

Relación uno-a-muchos y muchos-a-muchos

Un periodista puede escribir muchas noticias, pero cada noticia tiene un solo autor. Esa es una relación uno-a-muchos. En cambio, una noticia puede tener varias etiquetas y una etiqueta puede usarse en muchas noticias. Esa es una relación muchos-a-muchos y necesita la tabla intermedia `article_tag`.

Eloquent: trabajar con esas relaciones desde PHP

MySQL sigue siendo donde se guardan los datos. Eloquent es la herramienta de Laravel que permite trabajar con las tablas como modelos y relaciones de PHP. Por ejemplo, `Article` conoce a su autor mediante el siguiente método:

```
public function author(): BelongsTo
{
    return $this->belongsTo(User::class, 'author_id');
}
```

El nombre técnico para este tipo de herramienta es ORM, abreviatura de Object-Relational Mapping. Eloquent no reemplaza a MySQL: genera las consultas necesarias y devuelve los resultados como objetos que Laravel puede utilizar.

Eloquent y Query Builder: dos formas de trabajar con MySQL

TRAMA utiliza principalmente Eloquent cuando necesita trabajar con Models y relaciones, por ejemplo `Article`, `User` o `Category`. Laravel también ofrece Query Builder, una herramienta más directa para construir operaciones sobre tablas sin cargar necesariamente un Model completo.

Una expresión como `DB::table('articles')->where(...)->update(...)` pertenece al Query Builder. En TRAMA se usa, por ejemplo, cuando conviene actualizar una columna auxiliar como `search_text` sin guardar de nuevo todo el modelo `Article`. Ambas herramientas terminan generando consultas para MySQL; la diferencia está en el nivel de abstracción con el que trabaja el código PHP.

Índices y restricciones únicas

MySQL también puede acelerar búsquedas y evitar combinaciones inválidas. `article_tag` impide repetir la misma combinación noticia-etiqueta. `comment_likes` impide que una misma cuenta tenga dos filas de Me gusta para el mismo comentario. `comment_reports` aplica una regla equivalente para reportes. FULLTEXT acelera la búsqueda interna de noticias.

Datos preparados para consultas rápidas

TRAMA guarda algunas copias calculadas para evitar repetir trabajo en consultas frecuentes. Por ejemplo, `articles.views` conserva el total que necesita la interfaz mientras `article_views` mantiene el detalle de cada carga; `articles` conserva la versión actual y `article_revisions` mantiene las versiones anteriores; `articles.search_text` guarda un texto preparado para buscar. En diseño de bases de datos, guardar deliberadamente una copia de este tipo para resolver una consulta concreta se denomina desnormalización controlada.

Paso a paso: Valentina crea una noticia y MySQL conecta las relaciones

Este ejemplo transforma las claves foráneas y tablas intermedias en una operación cotidiana del CMS.

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina guarda una noticia. La fila principal se inserta en <code>articles</code> y contiene, entre otros datos, <code>author_id</code> y <code>category_id</code> .
2	<code>author_id</code> apunta a <code>users.id</code> y permite saber qué cuenta interna firmó la nota. <code>category_id</code> apunta a <code>categories.id</code> y determina su sección.

Paso	Qué hace la persona y qué ocurre internamente
3	Si Valentina selecciona varias etiquetas, Laravel no guarda una lista dentro de <code>articles</code> : crea relaciones en <code>article_tag</code> .
4	Cuando se registra una revisión, <code>article_revisions</code> conserva el cambio sin duplicar toda la estructura del proyecto en una sola tabla.
5	Si la noticia recibe comentarios o vistas, esas filas guardan <code>article_id</code> para relacionarse con la misma noticia.
6	Las claves foráneas evitan referencias imposibles y las reglas CASCADE, RESTRICT o SET NULL definen qué ocurre cuando se elimina o modifica un registro relacionado.
7	Eloquent permite recorrer esas relaciones desde PHP como objetos y colecciones, aunque por debajo MySQL siga trabajando con identificadores y claves.

7. Cómo se conectan Laravel, Inertia y Vue

Laravel y Vue cumplen tareas diferentes dentro de TRAMA. Laravel recibe la petición, comprueba permisos, valida datos, consulta MySQL y decide qué información necesita la pantalla. Vue toma esa información y construye la interfaz que aparece en el navegador. Inertia es la conexión entre ambas partes: permite que un controlador de Laravel entregue datos a una pantalla Vue sin tener que crear una API separada para cada página.

Qué papel cumple cada pieza

Conceptos de Vue que conviene conocer antes de seguir

TRAMA utiliza la Composition API de Vue 3. No hace falta conocer todo Vue para seguir el proyecto, pero sí conviene ubicar algunas piezas que aparecen repetidamente en las pantallas: props, estado reactivo, valores calculados, observadores y enlaces de formulario. Las definiciones siguientes describen cómo se usan realmente en este repositorio.

Pieza	Qué significa en TRAMA	Ejemplo real
<code>defineProps()</code>	Declara los datos que un componente recibe desde afuera.	<code>Trama.vue</code> recibe <code>screen</code> y <code>screenProps</code> ; <code>Form.vue</code> recibe <code>article</code> , <code>categories</code> , <code>tags</code> y <code>permissions</code> .
<code>ref()</code>	Guarda un valor reactivo individual que puede cambiar durante la interacción.	<code>Form.vue</code> usa <code>ref()</code> para el buscador de etiquetas, los modales y el estado del autoguardado.
<code>computed()</code>	Calcula un valor a partir de otros datos reactivos y se actualiza cuando cambian sus dependencias.	<code>Trama.vue</code> calcula <code>ActiveScreen</code> y <code>screenKey</code> ; <code>Login.vue</code> calcula <code>submitDisabled</code> .
<code>watch()</code>	Ejecuta una reacción cuando cambia un dato observado.	<code>Form.vue</code> observa <code>form.isDirty</code> para ocultar el mensaje de guardado anterior y también controla el bloqueo del scroll cuando hay modales.
<code>v-model</code>	Mantiene sincronizado un control HTML con una propiedad reactiva.	<code>v-model="form.email"</code> escribe directamente sobre el formulario de login; el formulario de noticias hace lo mismo con título, bajada y demás campos.
<code>usePage()</code>	Da acceso al objeto <code>page</code> de Inertia. <code>page.props</code> contiene los datos compartidos y <code>page.url</code> contiene la URL actual.	<code>Trama.vue</code> consulta <code>page.url</code> ; <code>Contact.vue</code> consulta <code>page.props.flash</code> para detectar el mensaje de envío correcto.

```
const page = usePage();
const ActiveScreen = computed(() => screens[props.screen] || HomeScreen);
const screenKey = computed(() => `${props.screen}:${page.url}`);

<component :is="ActiveScreen" :key="screenKey" v-bind="screenProps" />
```

No confundir: screenProps y props del componente

screenProps es el objeto que TramaBridge entrega a la página raíz de Inertia. Trama.vue usa v-bind="screenProps" para convertir sus claves en props del componente concreto. Una prop es, por lo tanto, el dato que termina recibiendo ese componente; screenProps es el paquete que los transporta dentro de la arquitectura propia de TRAMA.

TramaBridge.php: prepara el paso desde Laravel hacia Inertia

TramaBridge es una clase PHP del backend. No es un componente Vue y no muestra nada por sí misma. Un controlador la utiliza cuando ya sabe qué pantalla quiere abrir y qué datos debe recibir esa pantalla.

```
return TramaBridge::render('Public/ArticleShow', [
    'article' => $articleData,
]);
```

En este ejemplo, Public/ArticleShow indica qué pantalla Vue debe mostrarse. El valor \$articleData contiene la noticia y los demás datos que Laravel ya obtuvo y preparó. La llamada puede leerse así: «mostrá la pantalla pública de una noticia y entregale estos datos».

Trama.vue: el componente que elige la pantalla concreta

resources/js/Pages/Trama.vue sí es un componente Vue. Funciona como punto de entrada común para las pantallas de TRAMA: recibe desde Inertia el nombre de la pantalla que Laravel pidió y selecciona el componente Vue correspondiente.

```
screen = Public/ArticleShow

Trama.vue busca:
Public/ArticleShow.vue
```

Trama.vue no representa por sí sola una noticia, un formulario o un listado. Su tarea es decidir qué componente debe cargar en cada respuesta y pasarle los datos recibidos.

Public/ArticleShow.vue: la pantalla que finalmente ve el visitante

Public/ArticleShow.vue es el componente concreto que muestra el detalle de una noticia. Recibe los datos preparados por Laravel y los utiliza para mostrar el título, la bajada, el cuerpo, el autor, la categoría, las etiquetas, los comentarios, las vistas y los espacios publicitarios que correspondan.

```
TramaBridge.php
backend: indica qué pantalla y qué datos enviar
↓
Inertia
transporta esa información entre Laravel y Vue
↓
Trama.vue
frontend: decide qué componente cargar
↓
Public/ArticleShow.vue
muestra concretamente la noticia
```

Ejemplo completo: desde una dirección del portal hasta ArticleShow.vue

Supongamos que un visitante abre la noticia «Congreso acelera el debate por la reforma digital» en esta dirección:

```
/noticias/congreso-acelera-el-debate-por-la-reforma-digital
```

1. Laravel reconoce la dirección. `routes/web.php` reconoce el patrón `/noticias/{slug}`. El valor recibido como slug es `congreso-acelera-el-debate-por-la-reforma-digital` y la ruta envía la petición a `ArticleController::show()`.

2. El controlador delega la búsqueda. `ArticleController::show()` recibe `ArticleQueryService` y llama a `findPublishedBySlug()`. Ese servicio construye la consulta con Eloquent, aplica la visibilidad pública, busca el slug en MySQL y carga las relaciones principales —autor, categoría, etiquetas y comentarios aprobados—. Así el controlador coordina la petición sin concentrar la consulta pública dentro de la misma clase.

```
articles
├── author_id ───> users
├── category_id ───> categories
├── article_tag ───> tags
└── comments
```

3. Laravel prepara los datos. Después de consultar la base, el controlador reúne la información que va a entregar a la pantalla. Para entender el recorrido, puede imaginarse un contenido como el siguiente:

```
$articleData = [
    'title' => 'Congreso acelera el debate por la reforma digital',
    'category' => 'Política',
    'author' => 'Mateo Ledesma',
    'views' => 325,
    // otros datos de la noticia
];
```

4. ArticleController llama a TramaBridge. El controlador no necesita conocer cómo `Trama.vue` selecciona componentes. Sólo indica la pantalla lógica y los datos que debe recibir.

```
return TramaBridge::render('Public/ArticleShow', [
    'article' => $articleData,
]);
```

Qué hace TramaBridge con esa información

`TramaBridge` transforma la llamada anterior en la estructura que utiliza `Inertia`. `TRAMA` usa una única página raíz de `Inertia` llamada `Trama`. Por eso la respuesta incluye dos propiedades: `screen` indica qué pantalla concreta debe mostrarse y `screenProps` contiene los datos que se entregarán a esa pantalla.

```
Inertia::render('Trama', [
    'screen' => 'Public/ArticleShow',
    'screenProps' => [
        'article' => $articleData,
    ],
]);
```

screen vale `Public/ArticleShow` y significa «mostrar el detalle público de una noticia». **screenProps.article** contiene los datos de la noticia preparados por Laravel.

Qué hace Trama.vue al recibir la respuesta

`Trama.vue` recibe `screen` y `screenProps`. Cuando `screen` vale `Public/ArticleShow`, selecciona `Public/ArticleShow.vue` y le pasa los valores contenidos en `screenProps`. Conceptualmente, el componente realiza una operación equivalente a la siguiente:

```
<component
  :is="ActiveScreen"
  v-bind="screenProps"
/>
```

En palabras simples: «mostrá el componente que corresponde a esta pantalla y entregue los datos que llegaron desde Laravel».

Recorrido completo

```

Visitante abre la URL
/noticias/congreso-acelera-el-debate-por-la-reforma-digital
↓
routes/web.php
↓
ArticleController::show()
↓
ArticleQueryService::findPublishedBySlug()
↓
Eloquent consulta MySQL
↓
Laravel prepara $articleData
↓
TramaBridge.php
↓
Inertia envía screen + screenProps
↓
Trama.vue selecciona el componente
↓
Public/ArticleShow.vue
↓
Noticia visible en el navegador

```

Por qué existe TramaBridge

Sin TramaBridge, distintos controladores tendrían que repetir la misma estructura de Inertia cada vez que quieren mostrar una pantalla. TRAMA utiliza esta clase para que el controlador sólo indique qué pantalla necesita y qué datos debe recibir; TramaBridge construye después la respuesta con el formato que espera Trama.vue.

Carga de pantallas sólo cuando hacen falta

TRAMA tiene muchas pantallas y no es necesario descargar el código JavaScript de todas ellas al abrir una noticia. Por ejemplo, un visitante que está leyendo el portal no necesita cargar en ese momento el código del panel de administración. Vue permite cargar algunos componentes únicamente cuando van a utilizarse; TRAMA realiza esa carga con `defineAsyncComponent()`.

Laravel sigue controlando las direcciones y los permisos

TRAMA no utiliza Vue Router para decidir las direcciones del sitio. Las rutas continúan definidas en Laravel. Si alguien escribe manualmente una dirección privada, la petición vuelve a pasar por `routes/web.php`, los `middleware` y los controles de autorización correspondientes. Ocultar un botón en Vue mejora la interfaz, pero no reemplaza la comprobación de permisos del servidor.

Datos que muchas pantallas reciben automáticamente

`HandleInertiaRequests` comparte información que distintas pantallas necesitan de forma habitual, por ejemplo la cuenta autenticada, mensajes temporales después de una acción, categorías públicas, banners del encabezado, las rutas que Ziggy permite usar desde Vue y la fecha que está usando TRAMA. Los datos específicos de cada pantalla continúan dentro de `screenProps`.

Primera carga y navegación con Inertia

Cuando una persona entra al sitio mediante una carga completa, el navegador solicita el documento a Laravel. Laravel resuelve la ruta, prepara la respuesta de Inertia y entrega el documento base que inicia la aplicación Vue. A partir de ese momento, `app.js` monta o hidrata la aplicación según el tipo de respuesta recibida.

Después, las navegaciones realizadas mediante Inertia no necesitan reconstruir todo el documento HTML desde cero. Laravel sigue siendo responsable de las rutas, los permisos y los datos de cada pantalla, pero Inertia obtiene la nueva respuesta y Vue cambia el componente activo dentro de la aplicación ya cargada.

Esta diferencia es importante para entender la sección siguiente: en la mayoría de las pantallas Vue construye la interfaz en el navegador; en la portada, TRAMA agrega SSR para que Vue también pueda generar el HTML inicial desde Node.js antes de que la respuesta llegue al navegador.

Código fuente real de TramaBridge::render()

El puente no es sólo una idea arquitectónica: la clase `app/Support/TramaBridge.php` concentra la forma exacta de construir las respuestas Inertia. El método recibe la pantalla lógica y sus datos, limita SSR a la pantalla configurada y envía todo a la única página raíz Trama.

```
public static function render(string $screen, array $props = []): Response
{
    $ssrScreen = (string) config('trama.ssr.screen', 'Public/Home');

    config([
        'inertia.ssr.enabled' => (bool) config('inertia.ssr.enabled')
        && $screen === $ssrScreen,
    ]);

    return Inertia::render('Trama', [
        'screen' => $screen,
        'screenProps' => $props,
    ]);
}
```

`screen` no es una URL ni una ruta de Vue Router. Es una clave lógica acordada entre Laravel y Trama.vue. `screenProps` contiene únicamente los datos específicos que el controlador ya preparó para esa pantalla.

El resolver único de Inertia

`resources/js/inertia-pages.js` refuerza esta arquitectura: Inertia sólo admite la página raíz Trama. Las pantallas `Public/`, `Admin/` y `Auth/` no se registran como páginas Inertia independientes; Trama.vue las resuelve internamente.

```
export async function resolveInertiaPage(name) {
    if (name !== 'Trama') {
        throw new Error(`Página Inertia no reconocida: ${name}`);
    }

    const page = await import('./Pages/Trama.vue');
    page.default.layout = PersistentInertiaLayout;

    return page;
}
```

Cómo Trama.vue traduce screen en un componente

Trama.vue mantiene un mapa cerrado de pantallas. La función `lazy()` envuelve `defineAsyncComponent()` para cargar de forma diferida las pantallas que no necesitan formar parte de la carga inicial. Después, `ActiveScreen` toma el componente asociado a `screen` y `screenKey` fuerza un montaje nuevo cuando cambia la URL.

```
const lazy = (loader) => defineAsyncComponent(loader);

const AdminEditorialLayout = lazy(() => import('@Layouts/AdminEditorialLayout.vue'));
const AuthNewsLayout = lazy(() => import('@Layouts/AuthNewsLayout.vue'));

const screens = Object.freeze({
    'Public/Home': Home,
    'Public/ArticleShow': lazy(() => import('./Public/ArticleShow.vue')),
    'Public/CategoryShow': CategoryShow,
    'Public/JournalistShow': lazy(() => import('./Public/JournalistShow.vue')),
    'Public/Search': lazy(() => import('./Public/Search.vue')),
});
```

Una vez definido el mapa, `ActiveScreen` selecciona el componente asociado a `screen` y `screenKey` cambia con la URL. El fragmento siguiente muestra esa selección sin repetir el mapa completo.

```
const ActiveScreen = computed(
  () => screens[props.screen] || HomeScreen
);

const screenKey = computed(
  () => `${props.screen}:${page.url}`
);

<component
  :is="ActiveScreen"
  :key="screenKey"
  v-bind="screenProps"
/>
```

Layout público persistente

`PersistentInertiaLayout.vue` envuelve las pantallas `Public/` con `PublicNewsLayout` y permanece montado durante las navegaciones `Inertia` que utilizan la misma página raíz. De esta forma el encabezado, la navegación y el pie públicos no tienen que reiniciarse en cada cambio de pantalla. Las pantallas `Admin/` y `Auth/` se envuelven dentro de `Trama.vue` con sus propios layouts.

useForm: formularios Vue conectados con Laravel

`useForm()` es una función que ofrece el adaptador `Vue` de `Inertia` para trabajar con formularios. Su objetivo es reunir en un mismo objeto los datos que escribe el usuario y todo el estado necesario para enviar esos datos al servidor. En lugar de crear una variable independiente para cada campo y gestionar manualmente la petición HTTP, los errores y el estado de carga, `useForm()` permite trabajar con todo eso desde un único objeto.

Por ejemplo, un formulario puede guardar valores como `title`, `excerpt` o `body`, pero además permite saber si se está enviando (`processing`), consultar los errores de validación que devuelve `Laravel` (`errors`) y ejecutar operaciones como limpiar, transformar o restablecer los datos después de una acción correcta.

En `TRAMA` se utiliza para conectar los formularios `Vue` con `Laravel` mediante `Inertia`. El usuario completa los campos en la interfaz, `Vue` mantiene esos valores dentro del objeto creado con `useForm()`, `Inertia` los envía a `Laravel` y, si el servidor detecta un error de validación, esos errores regresan al mismo formulario para que `Vue` pueda mostrarlos junto al campo correspondiente.

```
const form = useForm({
  email: '',
  password: '',
  remember: false,
  redirect: props.redirect || '',
});

form.post(route('login'), {
  onFinish: () => form.reset('password'),
});
```

Elemento real	Para qué lo usa TRAMA
<code>form.processing</code>	Indica que <code>Inertia</code> está enviando el formulario. Se usa para bloquear botones y mostrar estados como “Ingresando...” o “Guardando...”.
<code>form.errors</code>	Contiene los errores que vuelven desde <code>Laravel</code> y también errores locales agregados con <code>setError()</code> .
<code>form.setError()</code> / <code>clearErrors()</code>	Permiten validar inmediatamente algunos campos en <code>Vue</code> sin reemplazar la validación definitiva del servidor.

Elemento real	Para qué lo usa TRAMA
<code>form.reset()</code>	Restablece uno o más campos. Login.vue, por ejemplo, limpia la contraseña cuando termina el intento.
<code>form.isDirty</code>	Indica que los valores actuales difieren de la última referencia guardada. El formulario de noticias lo usa para detectar cambios pendientes.
<code>form.defaults()</code>	Convierte los valores actuales en la nueva referencia guardada. Después de un guardado manual, Form.vue lo utiliza para que <code>isDirty</code> vuelva a false.
<code>form.transform()</code>	Modifica el payload antes de enviarlo. La edición de noticias envía un POST con <code>_method="put"</code> para soportar correctamente FormData y archivos.

No confundir: useForm, router y Axios

`useForm` administra un formulario y su estado. `router` ejecuta navegaciones o acciones Inertia que no necesitan conservar un formulario completo. `Axios` se usa para peticiones auxiliares y silenciosas, como el autoguardado del artículo. Los tres pueden hablar con Laravel, pero resuelven necesidades diferentes.

Paso a paso: Candela navega con Inertia sin convertir TRAMA en dos aplicaciones separadas

El recorrido muestra qué sucede desde un clic hasta que Vue cambia de pantalla.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela pulsa un enlace generado por Inertia. En una navegación posterior, el navegador no necesita descargar otra vez toda la plantilla HTML completa.
2	Laravel recibe la URL normal, ejecuta rutas, middleware, controladores y servicios exactamente igual que en una petición web tradicional.
3	El controlador llama a <code>TramaBridge::render()</code> indicando un nombre lógico de pantalla y los datos específicos de esa pantalla.
4	Inertia devuelve una respuesta con el componente raíz y las props en formato que su cliente Vue entiende.
5	Trama.vue recibe <code>screen</code> y <code>screenProps</code> . Un computed busca qué componente real corresponde a ese nombre lógico.
6	Vue reemplaza la zona de contenido con la pantalla correcta y conserva las partes persistentes previstas por la aplicación.
7	La URL y la seguridad siguen perteneciendo a Laravel; Vue se ocupa de la experiencia de interfaz, no de sustituir el router del servidor.

8. Fecha que usa TRAMA y TramaClock

TRAMA no guarda sus acciones usando directamente la fecha actual de la computadora. El proyecto tiene su propio reloj para que las noticias, comentarios, publicidades y demás datos de prueba mantengan las fechas para las que fueron preparados, aunque la aplicación se vuelva a abrir semanas o meses después.

La fecha inicial del proyecto

En el archivo `.env` se define que TRAMA comienza el 19/07/2026 a las 15:30. La computadora puede tener otra fecha completamente distinta.

```
TRAMA_REFERENCE_DATE=2026-07-19
TRAMA_REFERENCE_TIME=15:30:00
```

config/trama.php lee esos valores y los pone a disposición del código. La clase TramaClock utiliza esa fecha y esa hora como punto de partida.

Ejemplo: la fecha real puede ser distinta

Fecha real de la computadora:

01/09/2026 10:20

Hora que comienza usando TRAMA:

19/07/2026 15:30

Si en ese momento se crea una noticia o un comentario, TRAMA utiliza su propia hora del proyecto. No guarda 01/09/2026 sólo porque esa sea la fecha real del equipo.

Qué hace TramaClock

TramaClock es la clase encargada de calcular qué fecha y hora considera actuales TRAMA. La hora inicial no queda congelada en las 15:30: mientras el sitio está siendo utilizado, el reloj avanza prácticamente al mismo ritmo que el tiempo real.

TRAMA comienza: 19/07/2026 15:30

10 minutos de uso: 19/07/2026 15:40

20 minutos de uso: 19/07/2026 15:50

Qué ocurre cuando TRAMA deja de usarse

La diferencia importante aparece cuando no hay actividad. Supongamos que la última acción se realizó a las 15:42:18 y después se cierra el proyecto durante dos días. TRAMA no suma esas 48 horas al reloj del proyecto.

Última hora usada por TRAMA:

19/07/2026 15:42:18

El proyecto queda cerrado durante dos días.

Al volver a utilizarlo:

aproximadamente 19/07/2026 15:42:19

De esta forma, dejar el proyecto cerrado no hace que los datos de prueba salten automáticamente varios días hacia adelante.

Por qué aparecen 30 minutos de inactividad

TRAMA considera que una etapa de uso sigue activa mientras no pasen más de 30 minutos sin peticiones web. Si el usuario navega, guarda una noticia o realiza otra acción antes de ese límite, el reloj continúa avanzando. Si pasan más de 30 minutos sin actividad, esa etapa se considera pausada y la siguiente petición continúa desde la última hora guardada, en vez de sumar todo el tiempo de inactividad.

Cómo se activa el reloj en una petición

Todas las peticiones web pasan por AdvanceTramaClock. Es un middleware, es decir, una clase que Laravel ejecuta automáticamente alrededor de una petición para aplicar una regla común. Al comenzar la petición activa el reloj y, cuando la respuesta termina, registra el final de esa actividad.

El usuario abre una noticia

↓

Laravel recibe la petición

```

↓
AdvanceTramaClock
↓
TramaClock::activate()
↓
se ejecuta el controlador
↓
se genera la respuesta
↓
TramaClock::finishWebRequest()

```

Gracias a este middleware, ArticleController, UserController y los demás controladores no tienen que avanzar el reloj manualmente en cada método.

Dónde recuerda la última hora

TRAMA necesita recordar hasta qué hora llegó incluso si el servidor se detiene. Para eso utiliza la tabla `trama_clock_state` de MySQL. Allí conserva la hora del proyecto que había alcanzado, un valor del reloj real que utiliza sólo para medir cuánto tiempo transcurrió y la última actividad registrada.

```

trama_clock_state
↓
última hora usada por TRAMA
última actividad
referencia del reloj real para medir el intervalo

```

Qué registros utilizan esta hora

Los modelos principales del proyecto utilizan un comportamiento llamado `UsesTramaClock`. Cuando Laravel completa campos automáticos como `created_at` y `updated_at`, esos modelos toman la hora de TramaClock en lugar de pedir directamente la fecha actual del sistema.

```

Fecha real del equipo:
03/09/2026 09:15:00

Hora de TramaClock:
19/07/2026 16:03:27

Nuevo comentario → created_at = 19/07/2026 16:03:27

```

Este comportamiento se aplica a noticias, comentarios, revisiones, categorías, etiquetas, publicidades y otros registros que necesitan mantener la línea temporal del proyecto.

Qué pasa si llegan dos peticiones al mismo tiempo

Dos peticiones pueden intentar actualizar el estado del reloj casi al mismo tiempo. Para evitar que ambas calculen y escriban sobre el mismo valor de forma desordenada, Laravel bloquea durante unos instantes la fila de `trama_clock_state` mientras una de ellas termina la actualización. El método utilizado es `lockForUpdate()`.

```

Petición A lee el reloj
↓
bloquea momentáneamente la fila
↓
actualiza trama_clock_state
↓
libera la fila
↓
Petición B puede continuar

```

TramaClock y Scheduler no son lo mismo

Diferencia importante

TramaClock determina qué fecha y hora está usando TRAMA.

Laravel Scheduler ejecuta tareas periódicas, por ejemplo comprobar si una noticia programada ya debe publicarse.

TramaClock funciona cuando se utiliza el sitio aunque el Scheduler no esté activo. El Scheduler puede consultar la hora de TramaClock para decidir si una tarea programada ya corresponde, pero no es quien hace avanzar el reloj.

Límite actual del reloj

La versión actual de TramaClock está preparada para mantenerse dentro del 19/07/2026. Si llegara al final de ese día, TramaClock::clamp() lo detiene en 23:59:59 y no permite que continúe al 20/07/2026.

Fechas posteriores al 19/07/2026

La validación de noticias programadas permite elegir fechas de hasta 30 días después de la fecha base, pero TramaClock no puede avanzar al día siguiente. Por eso una noticia programada para el 20/07/2026 o una fecha posterior puede quedar aceptada por el formulario, aunque el reloj actual no pueda alcanzar automáticamente ese momento.

Paso a paso: Carlos vuelve a abrir TRAMA y TramaClock continúa sin saltar un día entero

El reloj editorial existe para que el escenario de demostración mantenga una cronología coherente aunque el proyecto se abra en días reales distintos.

Paso	Qué hace la persona y qué ocurre internamente
1	Carlos abre el portal y una petición web ejecuta el middleware que activa TramaClock.
2	En la primera activación, TramaClock parte de la fecha y hora de referencia configuradas y crea el estado persistente en trama_clock_state.
3	Mientras hay actividad, calcula cuánto tiempo real transcurrió desde el ancla y hace avanzar la hora editorial en la misma cantidad de segundos.
4	Si pasan más de 30 minutos sin actividad, la siguiente sesión no suma toda la pausa real. Continúa un segundo después del último instante editorial alcanzado.
5	La actualización del estado se realiza dentro de una transacción y usa lockForUpdate() para que dos peticiones simultáneas no hagan retroceder el reloj.
6	Los controladores y comandos consultan TramaClock::now() o today() para usar una misma cronología editorial en publicaciones, vistas y demás registros temporales.

9. Server-Side Rendering (SSR) e hidratación

La portada de TRAMA utiliza Server-Side Rendering (SSR) para que su primer HTML pueda generarse con Vue antes de llegar al navegador. Para entender la implementación conviene separar tres momentos: Laravel prepara los datos, Node.js genera el HTML inicial y, finalmente, Vue activa esa interfaz en el navegador mediante hidratación.

Qué es Server-Side Rendering (SSR)

Server-Side Rendering significa renderizado del lado del servidor. En una pantalla Vue sin SSR, Laravel prepara la respuesta de Inertia y el navegador descarga JavaScript; después Vue construye allí la interfaz. Con SSR aparece un paso adicional: antes de enviar la respuesta final, Vue también se ejecuta en Node.js y transforma los componentes en HTML.

En TRAMA esto se utiliza deliberadamente sólo para Public/Home. Laravel sigue siendo el servidor principal: resuelve GET /, consulta MySQL, aplica reglas de negocio y prepara las secciones de la portada. Node.js recibe una página de Inertia ya armada y la convierte en HTML; no elige noticias, no consulta la base y no reemplaza a Laravel.

Pantalla normal frente a portada con SSR

Pantalla Inertia/Vue normal	Portada Public/Home con SSR
Laravel prepara los datos y la página Inertia.	Laravel prepara los mismos tipos de datos y la página Inertia.
El navegador recibe la aplicación sin HTML Vue pre-renderizado.	La página puede enviarse primero al proceso SSR de Node.js.
<code>resources/js/app.js</code> usa <code>createApp()</code> .	<code>resources/js/ssr.js</code> usa <code>createSSRApp()</code> y <code>renderToString()</code> .
Vue construye la interfaz en el navegador.	Node genera el HTML inicial y Laravel lo incorpora a la respuesta.
No hay hidratación de HTML Vue previo.	<code>app.js</code> reconoce el HTML existente y Vue lo hidrata.

Qué es la hidratación

El HTML producido por SSR puede mostrar el encabezado, la navegación, la noticia principal y otros bloques desde el primer momento, pero ese HTML todavía no está conectado con toda la lógica reactiva de Vue. La hidratación es el proceso mediante el cual Vue toma esa estructura ya existente y la asocia con sus componentes, estado y eventos.

Idea clave

Hidratar no significa dibujar toda la portada de nuevo. Vue parte del HTML que ya recibió y lo conecta con la aplicación JavaScript. Un botón puede verse antes de la hidratación; después queda unido a su evento de clic y al estado reactivo del componente.

```

Node.js genera HTML
    ↓
El navegador muestra ese HTML
    ↓
resources/js/app.js carga Vue
    ↓
createSSRApp()
    ↓
Hidratación
    ↓
La portada queda interactiva

```

Una misma interfaz, dos momentos de ejecución

TRAMA no mantiene una portada separada para el servidor y otra para el navegador. `Public/Home.vue` forma parte de la misma aplicación. Durante el SSR puede ejecutarse en Node.js para producir HTML y, después, el navegador continúa trabajando sobre esa interfaz mediante hidratación.

Cómo app.js decide entre montar e hidratar

La entrada del navegador inspecciona el contenedor generado por Inertia. Si ya contiene HTML, la respuesta fue pre-renderizada y utiliza `createSSRApp()`; si está vacío, realiza un montaje normal con `createApp()`.

```

setup({ el, App, props, plugin }) {
  const createVueApp = el.innerHTML.trim()
    ? createSSRApp
    : createApp;

  return createVueApp({
    render: () => h(App, props),
  })
    .use(plugin)
    .use(ZiggyVue)
    .mount(el);
}

```

Recorrido completo de la portada con SSR

Paso 1. El navegador solicita la portada. Se realiza GET `/`. `routes/web.php` envía la petición a `HomeController`.

Paso 2. Laravel prepara los datos. `HomeController` consulta las secciones mediante `ArticleQueryService` y prepara la información que necesita `Public/Home`.

Paso 3. El controlador utiliza el puente. `HomeController` llama a `TramaBridge::render('Public/Home', ...)` con los datos ya preparados.

Paso 4. Se aplican las barreras SSR. `HandleInertiaRequests` comprueba la ruta `home` y `TramaBridge` vuelve a comprobar que la pantalla lógica sea `Public/Home`.

Paso 5. Inertia arma la página. La raíz es `Trama`; `screen` vale `Public/Home` y `screenProps` contiene los datos específicos. También se agregan las props compartidas, entre ellas `Ziggy`.

Paso 6. Node recibe la página. El proceso iniciado con `php artisan inertia:start-ssr` ejecuta el bundle SSR compilado.

Paso 7. Vue se ejecuta en el servidor. `resources/js/ssr.js` crea la aplicación con `createSSRApp()`; `Trama.vue` resuelve `Public/Home.vue`.

Paso 8. Se genera HTML. `renderToString()` convierte el árbol de Vue en una cadena HTML.

Paso 9. Laravel responde. El HTML se incorpora al documento base `resources/views/app.blade.php` junto con las props de Inertia y los recursos JavaScript/CSS.

Paso 10. Vue hidrata. Cuando `resources/js/app.js` termina de cargar, detecta el HTML existente y activa la interacción mediante `createSSRApp()`.

Qué hace ssr.js

`resources/js/ssr.js` es la entrada Vue que utiliza Node. `createServer()` recibe la página Inertia, `renderToString()` produce el HTML y `createSSRApp()` crea la aplicación del servidor. No se llama a `mount()` porque Node no dispone de un elemento DOM real.

```

createServer((page) => createInertiaApp({
  page,
  render: renderToString,
  resolve: resolveInertiaPage,
  setup({ App, props, plugin }) {
    return createSSRApp({ render: () => h(App, props) })
      .use(plugin)
      .use(ZiggyVue, ziggy);
  },
}));

```

Por qué sólo la portada

La portada es la principal página de entrada y es la pantalla donde TRAMA aprovecha el HTML inicial generado en el servidor. El login, el panel editorial y las demás páginas Inertia/Vue usan montaje normal en el navegador. Las vistas Blade independientes, como la contingencia de MySQL o las páginas ficticias de anunciantes, siguen un flujo diferente.

Dos barreras para limitar SSR

INERTIA_SSR_ENABLED no es el único control. HandleInertiaRequests restringe SSR a la ruta nombrada home. Luego TramaBridge compara la pantalla lógica con Public/Home. La segunda comprobación evita que una respuesta distinta producida accidentalmente dentro de / herede SSR sólo por encontrarse en esa ruta.

Petición	Comprobación	Resultado
GET /	Ruta home + screen Public/Home	Puede utilizar SSR si la configuración general está activa.
GET /login	La ruta no es home	Montaje normal de Vue en el navegador.
GET /noticias/{slug}	La ruta no es home	La noticia individual no usa el SSR selectivo de portada.
Otra respuesta dentro de /	Ruta home, pero screen distinto	TramaBridge desactiva SSR para esa respuesta.

Qué papel cumple app.blade.php

resources/views/app.blade.php es el documento HTML base de la aplicación Inertia. No consulta noticias y no reemplaza a Public/Home.vue. Contiene el punto donde Inertia inserta la aplicación y, cuando corresponde, recibe el HTML generado por Vue antes de que la respuesta llegue al navegador. También incluye CSS crítico y precargas de imágenes importantes para mantener estable el primer pantallazo mientras terminan de cargar los recursos completos.

Código fuente, bundle y servidor SSR

```
resources/js/ssr.js
↓ npm run build
bootstrap/ssr/ssr.js
↓ php artisan inertia:start-ssr
Servidor SSR de Inertia / Node.js
↓
HTML inicial de Public/Home
```

Compilar y ejecutar son operaciones distintas. npm run build genera tanto los recursos del cliente como el bundle del servidor. php artisan inertia:start-ssr inicia el proceso que utiliza ese bundle para atender solicitudes de renderizado.

Cómo se inicia SSR en TRAMA

Tener INERTIA_SSR_ENABLED=true permite que Laravel utilice SSR cuando la petición corresponde a Public/Home, pero esa variable no inicia por sí sola ningún proceso. Para que Node pueda renderizar la portada debe iniciarse el servidor SSR desde una terminal situada en la raíz del proyecto.

```
php artisan inertia:start-ssr
```

La terminal debe permanecer abierta mientras se utiliza SSR. Este comando ejecuta el bundle configurado en INERTIA_SSR_BUNDLE; no compila resources/js/ssr.js. Si bootstrap/ssr/ssr.js no existe o cambia código que participa en SSR, npm run dev no recompila ese bundle: debe ejecutarse npm run build:ssr y reiniciarse php artisan inertia:start-ssr para que Node utilice la versión actualizada.

Habilitar, compilar y ejecutar son cosas distintas

INERTIA_SSR_ENABLED=true habilita SSR en la configuración. npm run build:ssr genera el bundle cuando falta o debe recompilarse. php artisan inertia:start-ssr es el comando que inicia realmente el servidor SSR.

Variables SSR utilizadas por TRAMA

```
INERTIA_SSR_ENABLED=true
INERTIA_SSR_URL=http://127.0.0.1:13714
INERTIA_SSR_ENSURE_BUNDLE_EXISTS=true
INERTIA_SSR_BUNDLE=bootstrap/ssr/ssr.js
```

Variable	Función
INERTIA_SSR_ENABLED	Interruptor general del SSR. Las barreras de ruta y screen siguen limitando su alcance.
INERTIA_SSR_URL	Dirección donde Laravel contacta al proceso SSR.
INERTIA_SSR_ENSURE_BUNDLE_EXISTS	Obliga a comprobar que exista el bundle antes de intentar SSR.
INERTIA_SSR_BUNDLE	Ruta al archivo compilado que utiliza Node; no es el fuente resources/js/ssr.js.

Paso a paso: Candela abre la portada con SSR y después Vue toma el control

La primera carga y las navegaciones posteriores usan las mismas pantallas Vue, pero no empiezan exactamente del mismo lugar.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela escribe la dirección de la portada. Laravel resuelve Public/Home y prepara las props de la página.
2	Como la portada es una de las pantallas habilitadas para SSR, Inertia envía esos datos al proceso Node que ejecuta el bundle generado desde resources/js/ssr.js.
3	Node crea la aplicación Vue en el servidor y devuelve HTML ya renderizado para la primera respuesta.
4	El navegador recibe contenido visible antes de ejecutar el JavaScript del cliente.
5	app.js detecta que ya existe HTML generado y usa hidratación en lugar de montar una interfaz vacía desde cero.
6	Vue conecta eventos, estado y componentes sobre ese HTML sin reemplazar innecesariamente toda la estructura inicial.
7	Cuando Candela navega después dentro del sitio, Inertia puede resolver nuevas pantallas como navegación cliente sin repetir siempre el mismo proceso SSR.

10. Portada pública: cómo se seleccionan y organizan las noticias

La portada no recibe una lista única de noticias para que Vue decida qué hacer con ella. Laravel construye cada bloque editorial por separado. GET / llega a HomeController, que solicita las secciones a ArticleQueryService, convierte los modelos mediante Resources y entrega a Public/Home.vue propiedades independientes para la noticia principal, urgentes, destacadas, últimas, más leídas, categorías y bloques por categoría.

```

GET /
↓
HomeController
↓
ArticleQueryService::getHomePageSections()
↓
ArticleCardResource / CategoryResource
↓
TramaBridge::render('Public/Home')
↓
Public/Home.vue

```

Ventana de actualidad de siete días

Para ocupar espacios de portada, una noticia debe seguir siendo públicamente visible y haber sido publicada desde el comienzo del día de siete días atrás hasta el momento editorial actual. Esta regla no despublica la noticia: cuando supera esa ventana continúa accesible por su URL, por la búsqueda o desde otras páginas, pero deja de competir por la noticia principal, la cinta urgente, destacadas, últimas, más leídas y los bloques recientes de categorías.

Bloque	Cómo se selecciona	Cantidad máxima
Noticia principal (hero)	Primera noticia reciente marcada <code>is_featured=true</code> , ordenada por <code>published_at</code> descendente.	1
Urgentes (breaking)	Noticias recientes con <code>is_breaking=true</code> .	4
Destacadas	Noticias recientes con <code>is_featured=true</code> , omitiendo la principal.	6
Últimas	Noticias recientes ordenadas por fecha de publicación.	8
Más leídas	Misma ventana reciente, reordenada por <code>views</code> de mayor a menor.	8
Bloques por categoría	Categorías públicas con contenido reciente; dentro de cada una se toman las noticias más nuevas.	4 por categoría

Ejemplo: qué entra y qué no entra en la portada

Supongamos que TramaClock marca 19/07/2026 15:30. Una nota publicada el 18/07 y marcada como Destacada puede convertirse en la principal. Otra noticia del 19/07 marcada como Urgente puede aparecer en la cinta superior. Una nota publicada el 10/07 sigue siendo una noticia publicada válida, pero ya quedó fuera de la ventana de portada y no participa de estos bloques. Si una misma noticia es urgente, destacada y además está entre las más leídas, Laravel puede enviarla en más de una colección porque cada bloque responde a una regla editorial distinta.

`Public/Home.vue` evita repeticiones dentro de la cinta móvil mediante un Set de IDs. La cinta combina una referencia de Economía/Mercados, urgentes, últimas y más leídas; si una noticia ya fue agregada no vuelve a aparecer en esa misma vuelta. Cuando una noticia tiene `is_breaking=true`, la etiqueta Urgente tiene prioridad sobre cualquier etiqueta secundaria.

Código fuente real: cómo Laravel construye los bloques de portada

Las reglas anteriores no se aplican en Vue. `app/Services/ArticleQueryService.php` construye una consulta base de noticias visibles y recientes y después la clona para cada bloque. El fragmento siguiente es código real del proyecto; se omiten comentarios para concentrar la explicación en la consulta:

```
$base = Article::query()
->publiclyVisible()
->where('published_at', '>=', $frontPageStartsAt)
->with(['author', 'category', 'tags'])
->latest('published_at');

return [
    'hero' => (clone $base)->where('is_featured', true)->first(),
    'breaking' => (clone $base)->where('is_breaking', true)->take(4)->get(),
    'featured' => (clone $base)->where('is_featured', true)->skip(1)->take(6)->get(),
    'latest' => (clone $base)->take(8)->get(),
    'mostRead' => (clone $base)->reorder()->orderByDesc('views')->take(8)->get(),
];
```

`publiclyVisible()` aplica las condiciones necesarias para que la noticia y su categoría puedan mostrarse públicamente. `published_at >= $frontPageStartsAt` introduce la ventana reciente de siete días. `clone` permite reutilizar la misma consulta base sin mezclar las condiciones de un bloque con otro. En Más leídas, `reorder()` elimina el orden por fecha de la consulta base y lo sustituye por `views` descendente.

Paso a paso: Candela abre la portada y Laravel decide qué noticia ocupa cada bloque

La portada no es una lista estática: se arma a partir del estado editorial, la fecha de publicación y las marcas configuradas por el equipo.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela abre la portada y <code>HomeController</code> solicita a <code>ArticleQueryService</code> las secciones públicas necesarias.
2	El servicio toma como referencia <code>TramaClock</code> y calcula el inicio de la ventana de actualidad de siete días.
3	La consulta base sólo incluye noticias publicadas, con categoría pública y fecha dentro de esa ventana reciente.
4	La noticia principal sale de las destacadas; la cinta urgente usa <code>is_breaking</code> ; otros bloques toman destacadas restantes, últimas noticias y más leídas.
5	El ranking de lectura reordena por <code>views</code> , mientras los bloques de actualidad normalmente priorizan <code>published_at</code> .
6	Las categorías públicas se cargan en su orden editorial y cada una puede aportar una pequeña muestra de noticias recientes.
7	<code>TramaBridge</code> envía el conjunto a <code>Public/Home.vue</code> y la portada pinta cada bloque sin que Vue tenga que decidir reglas editoriales por su cuenta.

11. Registro, autenticación, recuperación y sesiones

Autenticarse significa comprobar quién intenta entrar. Autorizar significa decidir qué puede hacer esa cuenta después de haber iniciado sesión. Laravel Fortify aporta la base para el inicio de sesión y la recuperación de contraseña; TRAMA agrega el registro, la verificación del correo, el bloqueo de cuentas y otras reglas propias.

Service Provider: configuración que Laravel registra al iniciar

Un Service Provider es una clase que Laravel utiliza durante el arranque de la aplicación para registrar o configurar comportamientos que deben quedar disponibles para el resto del sistema. No representa una pantalla ni una tabla: prepara servicios y reglas globales antes de que lleguen las peticiones de los usuarios.

TRAMA utiliza FortifyServiceProvider para personalizar partes de la autenticación y recuperación de contraseña. AppServiceProvider, por su parte, registra configuraciones generales de la aplicación, entre ellas el Observer que mantiene sincronizado el orden editorial de las categorías cuando cambian noticias relevantes.

Ejemplo: registrar una cuenta

Paso a paso: Candela crea la cuenta y verifica el correo

Formulario de registro. Los mensajes junto a cada campo corresponden al mismo objeto useForm.

Register.vue combina dos capas de validación. Vue hace comprobaciones inmediatas con `form.setError()` para nombre, email y contraseña; al enviar, Laravel vuelve a validar todo mediante `CreateNewUser`. La comprobación del navegador mejora la experiencia, pero no reemplaza las reglas del servidor.

```
const form = useForm({
  name: '',
  email: '',
  password: '',
  password_confirmation: '',
  redirect: props.redirect || '',
});

form.setError('email', 'Ingresá un correo electrónico válido.');
```

Paso	Resultado
1	Candela completa nombre, email, contraseña y confirmación.
2	Vue normaliza y comprueba los datos básicos. Si encuentra un problema, usa <code>setError()</code> y no envía todavía.
3	<code>useForm</code> envía el POST a <code>/register</code> .
4	<code>RegisteredUserController</code> y <code>CreateNewUser</code> vuelven a validar y crean users con <code>role=reader</code> .
5	TRAMA intenta enviar el correo de verificación y cierra la sesión de registro.

Paso	Resultado
------	-----------

6

Candela abre el enlace firmado, verifica el correo y recién entonces puede iniciar sesión normalmente.



Confirmación mostrada después de crear la cuenta y antes de verificar el correo.

Candela completa nombre, correo, contraseña y confirmación y pulsa Crear cuenta. Vue envía el formulario a Laravel. RegisteredUserController::store() utiliza CreateNewUser, que vuelve a validar todos los datos en el servidor aunque el navegador ya haya hecho controles previos.



Formulario público de registro utilizado en TRAMA.

Paso	Qué ocurre
1	Candela envía el formulario.
2	RegisteredUserController::store() recibe la petición.

Paso	Qué ocurre
3	CreateNewUser valida nombre, correo y contraseña y crea users con role=reader e is_active=true.
4	Laravel intenta enviar el correo de verificación.
5	La sesión se cierra: registrarse no inicia sesión automáticamente.
6	Candela abre el enlace recibido, verifica su correo y después puede iniciar sesión.

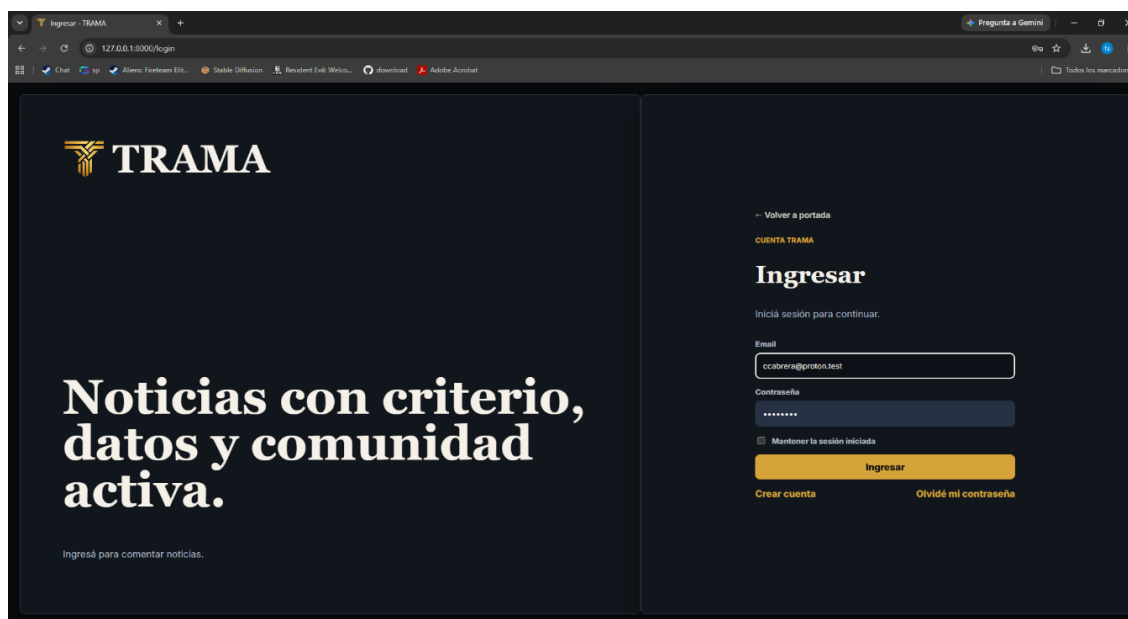
Qué es SMTP y qué pasa si falla

SMTP es el protocolo que utiliza una aplicación para enviar correo electrónico. Si TRAMA crea la cuenta pero el servicio de correo falla antes de enviar el enlace de verificación, RegisteredUserController registra el error y elimina la cuenta recién creada. Así no queda en MySQL una cuenta que el usuario no puede terminar de verificar.

Ejemplo: iniciar sesión

Paso a paso: Candela inicia sesión desde Login.vue

Candela abre Ingresar, escribe su correo y contraseña y pulsa el botón. En esta pantalla el objeto form fue creado con useForm(), por lo que Vue conoce los valores actuales, Inertia conoce si la petición está procesándose y Laravel puede devolver errores asociados a email o password.



Pantalla pública de inicio de sesión. La captura usa credenciales de prueba; el recorrido técnico es el mismo que sigue Candela.

```
const form = useForm({
  email: '',
  password: '',
  remember: false,
  redirect: props.redirect || '',
});

form.post(route('login'), {
  onFinish: () => {
    form.reset('password');
    isSubmitting.value = false;
  },
});
```

Paso	Qué ocurre
1	v-model mantiene form.email y form.password sincronizados con los inputs.
2	submit() bloquea el doble clic y llama form.post(route("login")).
3	La petición llega a las rutas de autenticación configuradas por Fortify.
4	Laravel valida credenciales, límites y estado de la cuenta.
5	Si el acceso falla, Inertia devuelve errores y Login.vue los lee desde form.errors.
6	Si el acceso funciona, Laravel regenera la sesión y redirige al destino correspondiente.
7	onFinish limpia la contraseña y vuelve a habilitar el botón.

TRAMA no guarda la contraseña escrita por el usuario tal como fue ingresada. Guarda un valor producido por un proceso de seguridad llamado hash. Al iniciar sesión, Hash::check() permite comprobar si la contraseña ingresada corresponde a ese valor sin necesitar guardar la contraseña original.

Paso	Qué ocurre
1	Candela escribe su correo y contraseña.
2	Fortify busca la cuenta por correo.
3	Laravel comprueba la contraseña.
4	TRAMA comprueba que la cuenta esté activa, que el correo esté verificado y que no exista un restablecimiento obligatorio pendiente.
5	Si todo es correcto, Laravel crea la sesión y permite continuar.

Situación	Qué ocurre
Correo o contraseña incorrectos	No se inicia sesión y se muestra un mensaje de credenciales incorrectas.
Cuenta bloqueada	Se rechaza el acceso.
Correo no verificado	El usuario debe completar primero la verificación.
Cambio obligatorio de contraseña pendiente	Debe utilizarse el enlace enviado por correo antes de entrar.

Situación	Qué ocurre
Todo correcto	Laravel crea la sesión.

El inicio de sesión admite hasta 5 intentos por minuto para la combinación de correo normalizado e IP. La IP es la dirección de red desde la que llega la petición. El control se ejecuta en el servidor, por lo que no depende de que JavaScript esté activo.

Qué es una sesión

Después de un inicio correcto, Laravel guarda la sesión en la tabla `sessions` de MySQL. El navegador conserva una cookie, es decir, un pequeño dato guardado localmente, con el identificador necesario para reconocer esa sesión en las siguientes peticiones. En Laravel esta configuración se conoce como driver de sesión `database`.

`EnsureActiveAccount` vuelve a comprobar el estado de la cuenta en peticiones posteriores. Si Marina, la administradora, bloquea una cuenta que ya estaba conectada, la pantalla que esa cuenta tiene abierta puede seguir visible hasta que haga otra acción. En la siguiente petición, la sesión se invalida y el usuario vuelve al login.

Recuperación de contraseña

Conceptos del flujo: token, Password Broker y Notification

Un token de recuperación es un valor temporal asociado al proceso de restablecimiento. No es la contraseña del usuario: sirve para demostrar que quien abre el enlace posee una referencia válida emitida por el sistema. Laravel conserva la información necesaria en `password_reset_tokens` y comprueba el token antes de aceptar una contraseña nueva.

El Password Broker es el componente de Laravel que coordina la creación y validación de esos tokens. Una Notification es una clase utilizada para representar una notificación destinada a un usuario; en este flujo, TRAMA utiliza `TramaResetPassword` para construir y enviar el correo con el enlace de recuperación.

Paso a paso: Candela recupera el acceso

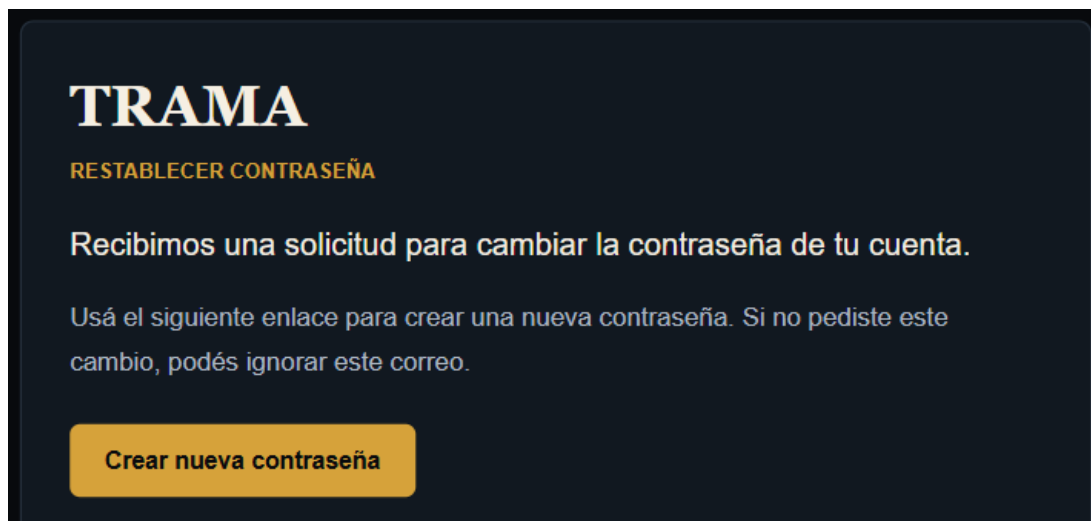
Paso	Qué hace la persona y qué ocurre internamente
1	Candela abre «Olvidé mi contraseña». Laravel muestra <code>Auth/ForgotPassword.vue</code> mediante Fortify y TramaBridge. La pantalla crea <code>useForm({ email: "" })</code> para conservar el correo, los errores y el estado <code>processing</code> .
2	Candela escribe su email y pulsa Enviar enlace. <code>submit()</code> ejecuta <code>form.post(route("password.email"))</code> , que corresponde a <code>POST /forgot-password</code> . Antes de Fortify, <code>ValidateAuthInputLimits</code> valida el formato, normaliza el email y aplica los límites: un intento real por email+IP cada 5 minutos y hasta 6 por IP en una hora.
3	Si la petición puede continuar, Fortify utiliza el Password Broker configurado como <code>users</code> . Cuando la cuenta existe, Laravel genera el token de recuperación en <code>password_reset_tokens</code> y <code>User::sendPasswordResetNotification()</code> envía la notificación personalizada <code>TramaResetPassword</code> . La respuesta visible sigue siendo neutra exista o no la cuenta.
4	Candela abre el enlace recibido. La ruta <code>/reset-password/{token}</code> muestra <code>Auth/ResetPassword.vue</code> . <code>FortifyServiceProvider</code> entrega email y token y además calcula <code>resetLinkError</code> con <code>Password::broker(...)->tokenExists()</code> , de modo que un enlace vencido, inválido o ya utilizado se detecta antes de permitir el envío.

Paso	Qué hace la persona y qué ocurre internamente
5	ResetPassword.vue crea un useForm con token, email, password y password_confirmation. Vue comprueba localmente que la contraseña tenga entre 8 y 20 caracteres, al menos una letra y un número, y que la confirmación coincida. Estas comprobaciones ayudan al usuario, pero no sustituyen la validación del servidor.
6	Candela pulsa Actualizar contraseña. form.post(route("password.update")) envía POST /reset-password. ValidateAuthInputLimits vuelve a validar email, contraseña y confirmación en Laravel antes de que Fortify procese el restablecimiento.
7	Fortify valida el token y el email mediante el Password Broker y llama a App\Actions\Fortify\ResetUserPassword. Esa acción vuelve a aplicar la política de contraseña, guarda el nuevo hash con Hash::make() y deja password_reset_required_at en null si la cuenta tenía un cambio obligatorio pendiente.
8	Cuando el restablecimiento termina correctamente, la configuración de Fortify redirige a /login y muestra el mensaje de estado. Candela ya puede iniciar sesión con la contraseña nueva; el enlace anterior deja de ser válido como token de recuperación.



Pantalla para solicitar el enlace de recuperación.

ForgotPassword.vue también utiliza useForm. El formulario envía solamente el email y permanece bloqueado mientras form.processing es true. Laravel responde con un mensaje neutro exista o no la cuenta, para no revelar qué direcciones están registradas.



Correo de recuperación. El enlace conduce al formulario `ResetPassword.vue`.

En Olvidé mi contraseña, el usuario escribe su correo y solicita un enlace. La pantalla muestra el mismo mensaje tanto si ese correo existe como si no existe. Esto evita que alguien utilice el formulario para averiguar qué direcciones están registradas.

Para un mismo correo y una misma IP se realiza como máximo un intento real cada 5 minutos. Además, una IP puede generar hasta 6 intentos reales por hora. Si se repite antes de tiempo, la pantalla mantiene el mismo mensaje pero no envía otro correo. El enlace de recuperación dura 60 minutos.

Código fuente real: controles adicionales durante el inicio de sesión

Fortify aporta el mecanismo de autenticación, pero TRAMA agrega reglas propias antes de aceptar la sesión. `app/Providers/FortifyServiceProvider.php` registra una función de autenticación que comprueba la contraseña y después revisa el estado de la cuenta. El fragmento siguiente conserva el código ejecutable de esa función y omite únicamente los comentarios explicativos del archivo fuente:

```

Fortify::authenticateUsing(function (Request $request): ?User {
    $user = User::query()
        ->where('email', $request->input(Fortify::username()))
        ->first();

    if (! $user || ! Hash::check((string) $request->input('password'), $user->password)) {
        return null;
    }

    if (! $user->is_active) {
        throw ValidationException::withMessages([
            Fortify::username() => 'Tu cuenta se encuentra bloqueada y no puede acceder a TRAMA. Si
consideras que se trata de un error, contacta al equipo de TRAMA.',
        ]);
    }

    if ($user->password_reset_required_at !== null) {
        throw ValidationException::withMessages([
            Fortify::username() => 'Debes restablecer la contraseña desde el enlace enviado a tu correo.',
        ]);
    }

    if ($user instanceof MustVerifyEmail && ! $user->hasVerifiedEmail()) {
        throw ValidationException::withMessages([
            Fortify::username() => __('auth.email_not_verified'),
        ]);
    }

    if (Hash::needsRehash($user->password)) {
        $user->forceFill(['password' => Hash::make((string) $request->input('password'))])>save();
    }

    return $user;
});

```

Devolver null deja que Fortify produzca el mensaje normal de credenciales incorrectas. Las otras situaciones se convierten en errores de validación controlados: una contraseña correcta no basta si la cuenta está bloqueada, si tiene un restablecimiento obligatorio pendiente o si todavía no verificó el correo. Al final, Hash::needsRehash() comprueba si el hash almacenado utiliza una configuración antigua; si es necesario, Laravel vuelve a generarlo con la configuración actual después de autenticar correctamente al usuario. Estas comprobaciones se ejecutan en Laravel y no dependen de que la interfaz Vue oculte botones.

12. Roles, paneles y permisos

TRAMA tiene cuatro tipos de cuenta con permisos diferentes: usuario registrado, periodista, editor y administrador. El menú cambia según el rol, pero los permisos importantes también se comprueban en Laravel. Conocer una dirección privada no permite saltarse esas reglas.

PERIODISTA	EDITOR	ADMINISTRADOR
Panel	Panel	Panel
Noticias	Noticias	Categorías · Etiquetas
Ver portal	Comentarios	Publicidades
	Ver portal	Empleados · Usuarios
		Ver portal

Periodista

El periodista crea borradores, completa noticias, las envía a revisión, consulta las observaciones del editor y corrige las notas devueltas. No puede publicar directamente una noticia ni abrir el módulo general de moderación.

Editor

El editor también puede escribir noticias. Además, revisa las notas enviadas por periodistas, puede devolverlas con observaciones, programarlas, publicarlas, archivarlas, modificar marcas de portada y moderar comentarios.

Administrador

El administrador utiliza un panel separado para categorías, etiquetas, publicidades, empleados y usuarios registrados. No reemplaza al editor: no revisa ni publica noticias y no escribe comentarios públicos.

Permisos principales

Acción	Usuario registrado	Periodista	Editor	Administrador
Escribir comentario principal	Sí	No	No	No
Responder un comentario	Sí	Sólo en sus noticias	Sí, como Equipo TRAMA	No
Dar Me gusta	Sí	No	No	No
Reportar comentario	Sí	No	No	No
Crear noticias	No	Sí	Sí	No
Revisar noticias de periodistas	No	No	Sí	No
Publicar o programar	No	No	Sí	No
Moderar comentarios	No	No	Sí	No
Categorías, etiquetas y publicidad	No	No	No	Sí
Empleados y usuarios registrados	No	No	No	Sí

Ejemplo de un permiso comprobado en el servidor

Si un periodista escribe manualmente una dirección reservada al administrador, Laravel vuelve a comprobar el rol antes de ejecutar el controlador. Que la opción no aparezca en el menú mejora la interfaz, pero la seguridad real está en el servidor.

Empleados y usuarios registrados son módulos distintos

Empleados contiene las cuentas internas de periodista, editor y administrador. Usuarios registrados contiene las cuentas creadas desde el portal. El administrador puede bloquearlas, reactivarlas, iniciar un restablecimiento de acceso y resolver revisiones derivadas desde Moderación.

Código fuente real: el permiso se comprueba antes de ejecutar el controlador

`routes/web.php` agrupa los módulos de estructura y cuentas bajo `EnsureAdminRole`. Conocer manualmente una URL como `/admin/users/employees` no evita este middleware:

```
Route::middleware(EnsureAdminRole::class)->group(function (): void {
    Route::get('users/employees', [AdminUserController::class, 'employees'])
        ->name('users.employees');

    Route::get('users/readers', [AdminUserController::class, 'readers'])
        ->name('users.readers');

    Route::get('categories', [AdminCategoryController::class, 'index'])
        ->name('categories.index');
});
```

El middleware `app/Http/Middleware/EnsureAdminRole.php` consulta la cuenta autenticada antes de dejar continuar la petición:

```
public function handle(Request $request, Closure $next): Response
{
    if (! $request->user()?->canManageSystem()) {
        abort(403, 'Solo un administrador puede acceder a esta sección.');
```

`abort(403, ...)` termina la petición con el estado HTTP 403, que significa que la cuenta fue reconocida pero no está autorizada para esa acción. Sólo si `canManageSystem()` devuelve verdadero se ejecuta `$next($request)` y Laravel continúa hacia la ruta y su controlador.

Paso a paso: Valentina intenta abrir un módulo reservado para la administradora

Este caso muestra por qué ocultar un botón no es suficiente para aplicar permisos.

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina inicia sesión con rol de periodista. Su menú no muestra Categorías, Etiquetas, Publicidades ni Usuarios registrados.
2	Aunque escriba manualmente una URL de administración, la petición igualmente entra por <code>routes/web.php</code> .
3	El grupo de rutas ejecuta <code>EnsureAdminRole</code> antes del controlador.
4	El middleware comprueba el rol real guardado en la sesión y en la cuenta autenticada.
5	Como Valentina no es administradora, Laravel detiene la petición y no ejecuta la acción protegida.
6	Marina, con rol de administradora, atraviesa el mismo middleware correctamente y recién entonces llega al controlador del módulo.

13. Dashboard del CMS y métricas según el rol

Después del login, una cuenta interna llega al dashboard del CMS. `DashboardController` no envía el mismo contenido a todos: consulta los permisos del usuario y combina `EditorialStatsService` con listados de actividad reciente. El objetivo es que cada rol vea información relacionada con las tareas que realmente puede realizar.

Rol	Indicadores principales	Actividad y listados
Periodista	Estados de sus noticias y vistas acumuladas de sus propias publicaciones.	Sus noticias más vistas, sus notas en revisión, sus últimas revisiones y sus publicaciones recientes.

Rol	Indicadores principales	Actividad y listados
Editor	Estados editoriales generales, Vistas hoy, comentarios pendientes y autores activos.	Ranking de hasta seis noticias publicadas que el editor puede consultar, notas en revisión, últimas revisiones, publicaciones recientes y comentarios pendientes.
Administrador	Empleados activos, usuarios registrados, categorías activas, etiquetas activas y publicidades activas.	Últimos usuarios registrados y días recientes con clics de banners. No recibe actividad editorial ni ranking de noticias.

Cómo cambia el cálculo de vistas

Para un periodista, EditorialStatsService suma el campo views de sus propias noticias y muestra el acumulado. Para un editor, el dashboard utiliza ArticleView y cuenta solamente las vistas del día editorial actual. De esa forma el editor ve actividad operativa del día y no un total histórico inflado por todos los datos de prueba.

Código fuente real: el dashboard decide las métricas según el rol

La diferencia entre Valentina, Mateo y Marina no se resuelve ocultando tarjetas en Vue. app/Services/EditorialStatsService.php recibe el User autenticado y construye conjuntos distintos. El fragmento siguiente conserva las bifurcaciones principales del método getDashboardIndicatorsForUser():

```
if ($user->canManageSystem()) {
    return [
        'Empleados activos' => User::query()
            ->whereIn('role', ['admin', 'editor', 'journalist'])
            ->where('is_active', true)
            ->count(),
        'Usuarios registrados' => User::query()
            ->where('role', 'reader')
            ->count(),
        'Categorías activas' => Category::query()
            ->where('is_active', true)
            ->count(),
        'Publicidades activas' => Advertisement::query()
            ->where('is_active', true)
            ->count(),
    ];
}

$articles = $this->visibleArticlesForUser($user);

$indicators = [
    'Publicadas' => (clone $articles)->where('status', 'published')->count(),
    'En revisión' => (clone $articles)->where('status', 'review')->count(),
    'Vistas' => $this->dashboardViewsForUser($user, $articles),
];

if ($user->canReviewArticles()) {
    $indicators['Comentarios pendientes'] = Comment::query()
        ->where('status', 'pending')
        ->count();
}
```

canManageSystem() separa al administrador, que recibe métricas de estructura y cuentas. Para periodista y editor, visibleArticlesForUser() limita primero qué noticias entran en los cálculos: Valentina queda restringida a su autoría; Mateo puede ver el flujo editorial general según las reglas del servicio. Después DashboardController entrega stats, rankings, revisiones y permisos a Admin/Dashboard.vue mediante TramaBridge.

Ejemplo comparativo

Usuario	Qué ve al entrar
Valentina, periodista	Borradores, noticias enviadas a revisión, devueltas, programadas, publicadas y archivadas de su autoría; vistas acumuladas y sus seis noticias más vistas.
Mateo, editor	Estado de la redacción, comentarios pendientes, autores activos, Vistas hoy, hasta seis notas en revisión y las últimas acciones editoriales visibles.
Marina, administradora	Cantidad de empleados y lectores, categorías, etiquetas y publicidades activas; últimas cuentas creadas y actividad reciente de clics publicitarios.

Los permisos también viajan a Admin/Dashboard.vue. Vue puede ocultar botones o bloques que no corresponden, pero el control real sigue estando en Laravel: que una tarjeta no se vea en el dashboard no reemplaza los middleware ni las comprobaciones de autorización de las rutas privadas.

Paso a paso: Valentina y Mateo abren el panel y reciben métricas distintas

El dashboard reutiliza una misma idea de interfaz, pero Laravel prepara datos diferentes según la responsabilidad del rol.

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina entra al panel como periodista. Laravel identifica su cuenta y calcula métricas centradas en sus propias noticias.
2	Las consultas cuentan borradores, revisiones, publicaciones y vistas vinculadas al trabajo de Valentina según las reglas del dashboard.
3	Mateo entra con rol de editor. El mismo acceso al panel necesita una mirada global de la redacción y por eso las consultas cambian de alcance.
4	Mateo puede recibir cantidades de noticias en revisión, programadas, publicadas y otras señales generales que no dependen únicamente de su autoría.
5	El controlador prepara las props adecuadas para cada rol y Vue pinta tarjetas equivalentes con valores distintos.
6	La diferencia se decide en el servidor; no consiste en descargar todos los datos y esconder con CSS lo que cada persona no debería ver.

14. Creación de una noticia: campos y validaciones

El formulario de una noticia tiene las pestañas Contenido, Configuración e Historial. Cuando se está creando una noticia por primera vez, Historial todavía no aparece porque no existe ningún guardado anterior para comparar.

TRAMA no exige que un borrador esté completo. Un periodista puede guardar lo que lleva escrito y continuar después. Las reglas completas se aplican cuando intenta enviar la noticia a revisión, programarla o publicarla.

Campos principales

Campo visible	Qué contiene	Regla al completar la noticia
Título	Encabezado principal	20 a 90 caracteres.

Campo visible	Qué contiene	Regla al completar la noticia
Bajada	Texto que acompaña al título	50 a 140 caracteres.
Resumen	Texto breve usado en listados	50 a 200 caracteres.
Cuerpo	Contenido completo de la noticia	300 a 10.000 caracteres visibles.
Categoría	Sección principal del diario	Obligatoria.
Etiquetas	Temas relacionados	Entre 1 y 5.
Portada	Imagen principal	Obligatoria cuando la noticia debe quedar completa.

En la interfaz se utilizan los nombres Bajada y Resumen. En el código y en MySQL esos mismos campos se llaman subtitle y excerpt.

Cómo se limpia el cuerpo de la noticia

El editor de texto utiliza HTML para representar elementos como párrafos, negritas o enlaces. Antes de validar y guardar el cuerpo, HtmlSanitizer elimina etiquetas o atributos que TRAMA no permite. Así no se guarda código ejecutable introducido dentro del texto.

Ejemplo: guardar un borrador y enviarlo a revisión

Antes de Laravel: cómo Form.vue conserva lo que escribe Valentina

Pestaña Contenido del formulario editorial en una cuenta periodista de prueba.

El formulario editorial es uno de los usos más completos de useForm en TRAMA. El mismo objeto reúne contenido, categoría, portada, estado editorial, marcas de portada, fechas y etiquetas. Al crear una noticia parte de valores vacíos; al editar toma los valores recibidos en props.article.

Las capturas de esta subsección provienen de la guía visual y utilizan una cuenta periodista de prueba distinta. El ejemplo narrativo mantiene a Valentina para conservar el hilo pedagógico; ambas cuentas recorren el mismo Form.vue y las mismas reglas del rol Periodista.

```
const form = useForm({
  category_id: props.article?.category_id || '',
  title: props.article?.title || '',
  subtitle: props.article?.subtitle || '',
  excerpt: props.article?.excerpt || '',
  body: props.article?.body || '',
  cover_image_file: null,
  remove_cover_image: false,
  status: props.article?.status || 'draft',
  is_breaking: props.article?.is_breaking || false,
  is_featured: props.article?.is_featured || false,
  published_at: props.article?.published_at || '',
  scheduled_at: props.article?.scheduled_at || '',
  tag_ids: props.article?.tag_ids || [],
});
```

Título
Nuevas rutas comerciales reordenan la logística regional 56/90

Bajada
La redacción compara proyectos logísticos y acuerdos recientes. 63/140

Resumen
El artículo aún necesita ordenar fuentes y verificar estimaciones antes de revisión. 84/200

Cuerpo de la noticia

Rich Text Editor: P H₂ H₃ B I [List Icons] [Undo] [Redo]

Nuevas rutas comerciales están modificando la planificación logística regional a medida que empresas prueban corredores alternativos y combinaciones distintas de puerto, ferrocarril y transporte por carretera. El borrador busca comparar qué cambios responden a costos coyunturales y cuáles pueden mantenerse en el tiempo.

La redacción analiza proyectos de infraestructura, acuerdos aduaneros y tiempos de tránsito, pero todavía necesita ordenar las fuentes para no mezclar obras anunciadas con capacidad ya disponible. Esa diferencia es clave porque una ruta potencial no puede evaluarse como si estuviera plenamente operativa.

El artículo también debe verificar estimaciones de ahorro y volúmenes antes de pasar a revisión. La versión final debería mostrar con claridad qué cargas están cambiando de recorrido, por qué lo hacen y qué cuellos de botella pueden limitar esa reorganización.

Buttons: Eliminar borrador, Guardar borrador, Enviar a revisión, Volver a noticias

Contenido escrito durante la edición: título, bajada, resumen y cuerpo. La captura pertenece a una cuenta periodista de prueba.

Categoría
Mundo

Etiquetas 2 / 5
Geopolítica x Inversión x
Buscar etiqueta...

Podés elegir hasta 5. La categoría solo ordena las sugerencias y no limita la búsqueda.

ESTADO ACTUAL
En borrador

Imagen de portada
[Image of a port with many containers and ships]
Cambiar imagen
Quitar imagen

Imagen opcional mientras la noticia esté en borrador. JPG, JPEG, PNG o WEBP. Máximo 5 MB.

Buttons: Previsualizar noticia

Bottom Buttons: Eliminar borrador, Guardar borrador, Enviar a revisión, Volver a noticias

Configuración del mismo formulario: categoría, etiquetas e imagen de portada.

Paso a paso: Valentina guarda el primer borrador

Paso	Qué ocurre en el proyecto
1	Valentina escribe. Cada v-model modifica una propiedad del objeto form.
2	form.isDirty pasa a true porque los valores ya no coinciden con la referencia del último guardado.
3	Valentina pulsa Guardar borrador y submit() valida primero los controles que puede comprobar el navegador.
4	En creación, form.post(route("admin.articles.store")) envía FormData a Laravel.
5	ArticleRequest normaliza título, bajada, resumen y cuerpo y aplica las reglas correspondientes al estado draft.
6	ArticleController coordina el procesamiento de portada y delega el flujo editorial que corresponde.
7	MySQL guarda la noticia y las relaciones dentro de las transacciones definidas por el backend.
8	La respuesta vuelve por Inertia. onSuccess limpia el archivo temporal y ejecuta form.defaults(), por lo que isDirty vuelve a false.

Qué pasa si Valentina intenta completar una acción con datos inválidos

Cuando Laravel devuelve errores de validación, Inertia los coloca dentro de form.errors. Form.vue revisa qué campos fallaron: si el problema está en título, bajada, resumen o cuerpo abre Contenido; para categoría, etiquetas, portada o programación abre Configuración. De esta forma el usuario vuelve directamente a la zona que debe corregir.

```
onError: (errors) => {
  const hasContentErrors =
    errors.draft_content || errors.title || errors.subtitle ||
    errors.excerpt || errors.body;

  activeTab.value = hasContentErrors
    ? 'content'
    : 'settings';
},
```

Valentina comienza una noticia y escribe solamente el título y una parte del cuerpo. Puede pulsar Guardar borrador aunque todavía falten categoría, etiquetas, portada o los mínimos de algunos campos. Más tarde completa la noticia y pulsa Enviar a revisión. En ese momento ArticleRequest aplica las reglas completas.

Si, por ejemplo, el resumen tiene sólo 35 caracteres, Laravel rechaza la operación, mantiene la noticia como borrador y muestra el error junto al campo correspondiente.

Paso	Qué ocurre
1	Form.vue reúne los valores escritos en el formulario.
2	ArticleRequest normaliza y valida los datos según el estado solicitado.
3	Si se cargó una portada nueva, ArticleCoverProcessor y SafeImageProcessor la revisan y generan el archivo final.
4	ArticleController llama a ArticleWorkflowService para guardar los cambios relacionados.
5	MySQL actualiza la noticia, sus etiquetas, el texto de búsqueda y el historial dentro de una transacción, es decir, como un grupo de cambios que se confirma o se revierte en conjunto.

Paso	Qué ocurre
6	El formulario recibe el nuevo estado o los mensajes de validación.

Portada

La portada acepta JPG, JPEG, PNG o WebP de hasta 5 MB. La imagen debe tener resolución suficiente para generar 1600 × 900 sin agrandarla. Además, pasa por el análisis local de seguridad explicado en la sección 20.



Ejemplo de una portada rechazada por la validación de seguridad.

Qué es el slug de una noticia

Una noticia publicada necesita una dirección legible. TRAMA genera parte de esa dirección a partir del título; esa parte se llama slug.

```
Título:
Nuevas rutas comerciales reordenan la logística regional

Slug:
nuevas-rutas-comerciales-reordenan-la-logistica-regional

URL:
/noticias/nuevas-rutas-comerciales-reordenan-la-logistica-regional
```

Mientras la noticia sigue como borrador puede utilizar un identificador provisional como borrador-UUID. Cuando se programa o publica por primera vez obtiene el slug definitivo. Después lo conserva aunque el título cambie para no romper enlaces ya compartidos.

Editor de texto e Historial

Deshacer y rehacer dentro del editor sólo afecta a la sesión de escritura abierta en el navegador. Historial es otra cosa: guarda revisiones permanentes en MySQL y sigue existiendo aunque se cierre el navegador.

Código fuente real: las reglas cambian según el estado solicitado

app/Http/Requests/Admin/ArticleRequest.php no aplica exactamente las mismas reglas a un borrador que a una noticia lista para entrar al circuito editorial. Primero determina si el estado solicitado exige una noticia completa:

```
private function requiresCompleteArticle(): bool
{
    return in_array(
        (string) $this->input('status'),
        ['review', 'scheduled', 'published'],
        true,
    );
}
```

Ese resultado se utiliza para construir las reglas. Un borrador puede dejar campos incompletos, mientras que revisión, programación y publicación exigen los mínimos editoriales:

```
$requiresCompleteArticle = $this->requiresCompleteArticle();

$titleRules = $requiresCompleteArticle
    ? ['required', 'string', 'min:20', 'max:90']
    : ['nullable', 'string', 'max:90'];

return [
    'category_id' => $requiresCompleteArticle
        ? ['required', $this->categoryExistsRule()]
        : ['nullable', $this->categoryExistsRule()],

    'title' => $titleRules,

    'tag_ids' => $requiresCompleteArticle
        ? ['required', 'array', 'min:1', 'max:5']
        : ['sometimes', 'array', 'max:5'],
];
```

La validación se decide en el servidor a partir del estado recibido. Por eso modificar el HTML o JavaScript del formulario no permite enviar una noticia incompleta a revisión: `ArticleRequest` vuelve a comprobar las reglas antes de que el controlador continúe.

15. Autoguardado y protección de cambios

Después del primer guardado manual, una noticia editable puede autoguardarse. El autoguardado reduce el riesgo de perder trabajo, pero no crea una revisión histórica cada vez que alguien escribe una palabra.

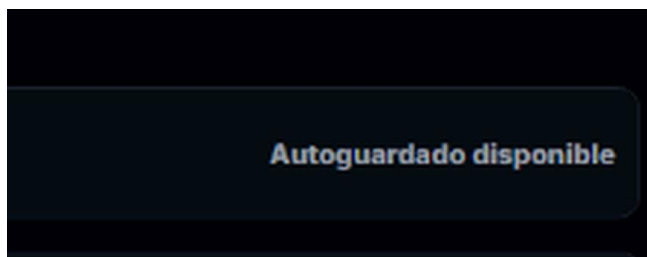
Sólo se utiliza en Borrador (draft) y Devuelta para corrección (needs_changes), que son los estados en los que el autor puede seguir modificando el contenido.

Ejemplo del temporizador de 15 segundos

No confundir: guardado manual con useForm y autoguardado con Axios

El guardado manual necesita manejar el formulario completo, archivos, errores y la navegación Inertia. El autoguardado tiene otra finalidad: enviar silenciosamente sólo un subconjunto de campos de una noticia que ya existe, sin cambiar de pantalla. Por eso `Form.vue` construye `autosavePayload` y usa `window.axios.patch()`.

Guardado manual	Autoguardado
useForm + Inertia	Axios directo
Puede crear una noticia nueva o actualizar la existente	Sólo actualiza una noticia existente
Incluye portada, etiquetas, estado y demás campos según la acción	Envía categoría, título, bajada, resumen y cuerpo
Puede devolver errores dentro de <code>form.errors</code>	Maneja el resultado como petición auxiliar
Al terminar puede ejecutar <code>form.defaults()</code>	Actualiza la referencia del formulario con los valores autoguardados



Estado visible cuando el autoguardado está disponible.



Advertencia mostrada cuando existe trabajo posterior al último guardado.

Valentina cambia una frase. Pasan 8 segundos y vuelve a escribir: el contador vuelve a empezar. Cinco segundos después corrige otra palabra y el contador se reinicia otra vez. Recién cuando pasan 15 segundos sin cambios, Vue envía un único autoguardado.

Si la noticia tiene id 37, la petición se envía a `/admin/articles/37/autosave`. El método HTTP usado es PATCH, que en este caso indica que se actualiza una parte de un registro que ya existe.

```
const autosavePayload = {
  category_id: form.category_id,
  title: form.title,
  subtitle: form.subtitle,
  excerpt: form.excerpt,
  body: form.body,
};

window.axios.patch(
  route('admin.articles.autosave', props.article.id),
  autosavePayload
);
```

Qué guarda el autoguardado

`AutosaveArticleRequest` valida categoría, título, bajada, resumen y cuerpo. `ArticleWorkflowService::autosave()` actualiza esos campos y recalcula el tiempo de lectura. No cambia el estado de la noticia, no cambia etiquetas ni portada y no crea una nueva fila en `article_revisions`.

Qué pasa si coinciden dos guardados

Puede ocurrir que el autoguardado comience justo cuando el periodista pulsa Guardar cambios. Para evitar que ambos intenten escribir la misma noticia al mismo tiempo, Laravel bloquea esa fila durante unos instantes mientras termina una de las operaciones. El método utilizado para hacerlo es `lockForUpdate()`.

Protección al salir de la pantalla

Si quedan cambios posteriores al último guardado y el usuario intenta volver al listado, abrir otra opción del panel o cerrar sesión, TRAMA muestra una confirmación. Si se cierra o recarga la pestaña del navegador, utiliza la advertencia incorporada por el propio navegador; en JavaScript ese evento se llama `beforeunload`.

Paso a paso: Valentina escribe, espera y el autoguardado protege el borrador

El autoguardado es un buen ejemplo de una acción silenciosa que no necesita reemplazar la pantalla completa.

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina modifica título, bajada, resumen o cuerpo de una noticia editable. Vue detecta que existen cambios pendientes.
2	Cada nueva modificación reinicia el temporizador. Mientras Valentina sigue escribiendo no se envía una petición por cada tecla.
3	Cuando pasan 15 segundos sin cambios y el formulario tiene los datos mínimos, Vue construye <code>autosavePayload</code> .
4	En lugar del envío normal de <code>useForm</code> , el componente usa <code>axios.patch()</code> contra la ruta <code>admin.articles.autosave</code> .
5	<code>AutosaveArticleRequest</code> valida únicamente los campos admitidos para esta operación y <code>ArticleWorkflowService::autosave()</code> bloquea la fila con <code>lockForUpdate()</code> .
6	MySQL actualiza el trabajo en curso y recalcula el tiempo de lectura, pero no cambia el estado, las etiquetas, la portada ni crea una revisión histórica.
7	Vue recibe la respuesta y cambia el indicador visual a Cambios autoguardados. Si Valentina intenta salir antes, la interfaz puede advertir que todavía hay cambios pendientes.

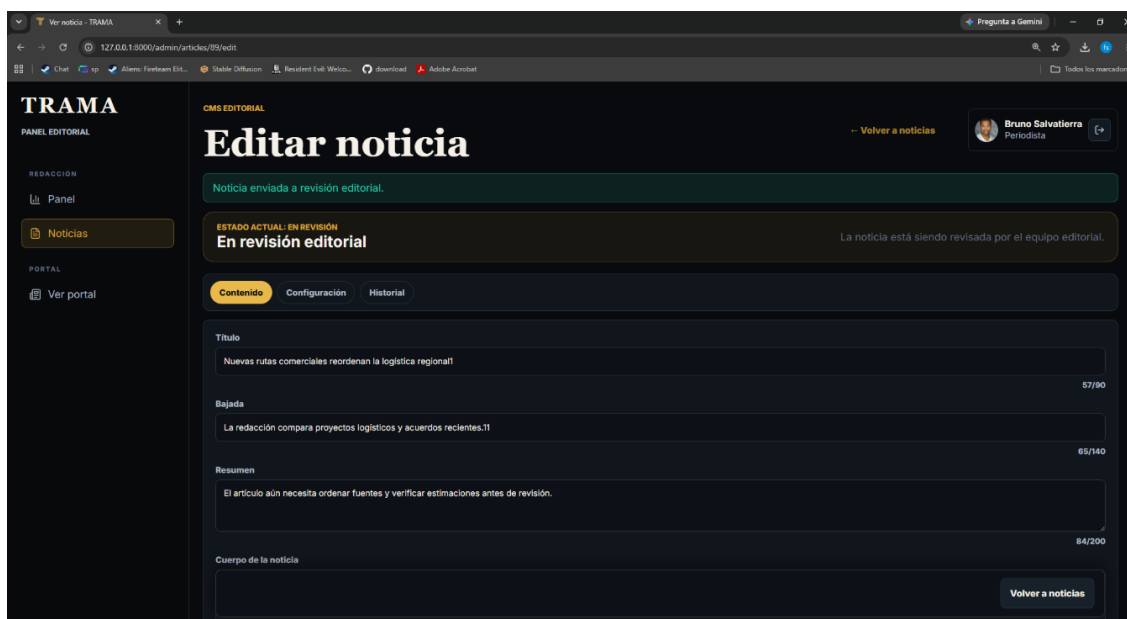
16. Flujo entre periodista y editor

El flujo editorial separa claramente el trabajo del periodista y el del editor. El periodista escribe la noticia y realiza las correcciones que le piden. El editor revisa el contenido y decide si necesita cambios, si se publica en ese momento o si queda programado para más adelante.

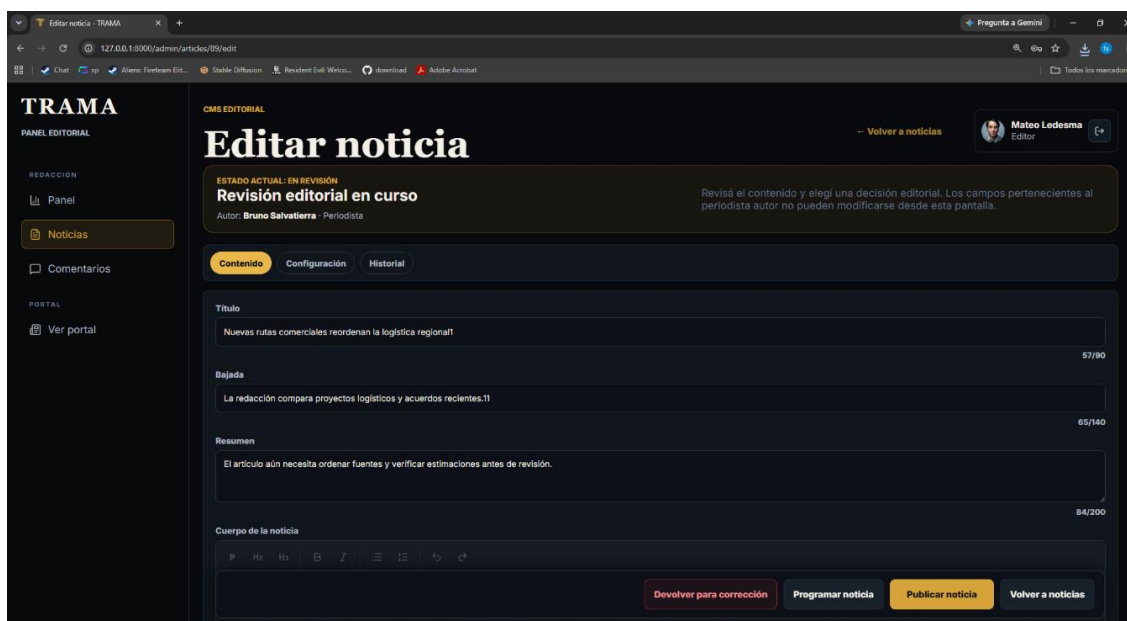
Ejemplo completo: de borrador a publicación

El mismo recorrido visto desde la interfaz



Valentina confirma el envío de la noticia al circuito editorial.

Después del envío, la noticia queda En revisión y el contenido se bloquea para la periodista.



Mateo abre la noticia recibida y toma la decisión editorial.

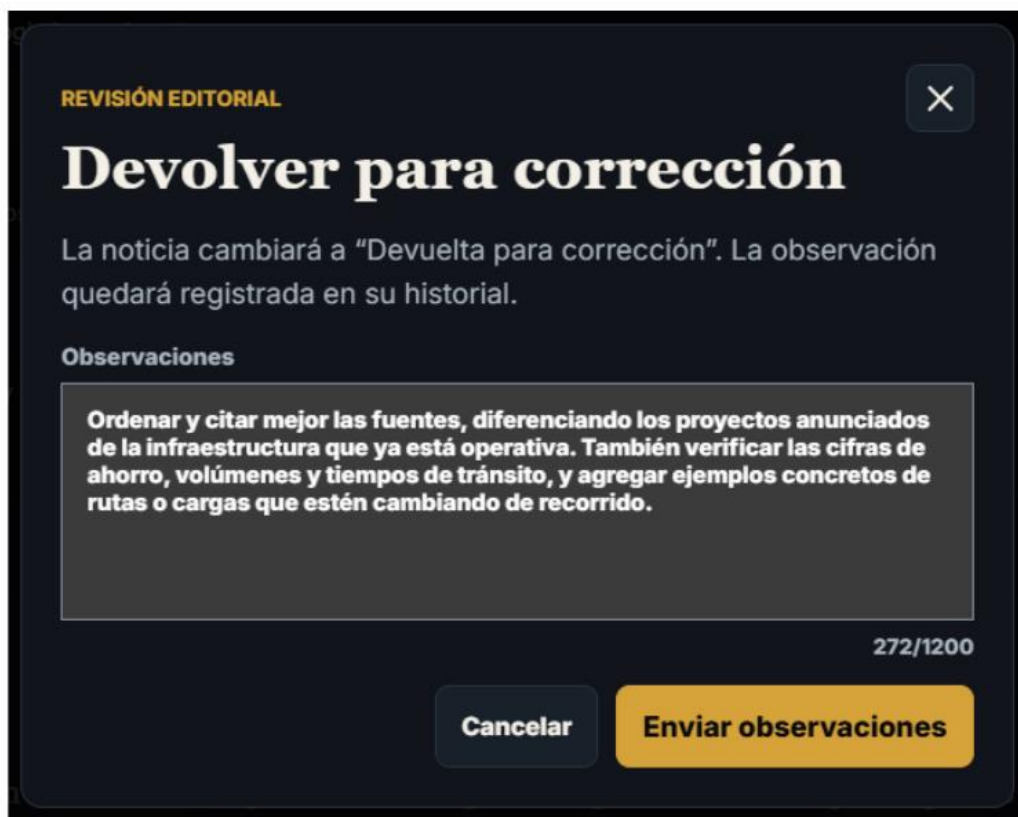
Por qué este ejemplo importa

La misma pantalla Form.vue cambia de comportamiento según autor, rol y estado. Vue puede ocultar o bloquear controles, pero la autorización definitiva continúa en ArticleRequest, los middleware y ArticleWorkflowService. La interfaz explica el estado; el servidor lo hace cumplir.

Valentina termina una noticia y pulsa Enviar a revisión. Antes de aceptarla, TRAMA vuelve a comprobar que tenga los datos necesarios para entrar al circuito editorial. Si la validación es correcta, la noticia deja de ser un borrador y pasa a En revisión.

Desde ese momento Valentina ya no puede modificarla. La nota queda disponible para Mateo, el editor, que puede abrirla desde las noticias pendientes de revisión y consultar su contenido, configuración e historial.

Si Mateo encuentra algo que debe corregirse, escribe una observación y utiliza Devolver para corrección. La noticia pasa a Devuelta para corrección y la observación queda registrada en su historial para que la periodista sepa exactamente qué debe revisar.



REVISIÓN EDITORIAL

Devolver para corrección

La noticia cambiará a “Devuelta para corrección”. La observación quedará registrada en su historial.

Observaciones

Ordenar y citar mejor las fuentes, diferenciando los proyectos anunciados de la infraestructura que ya está operativa. También verificar las cifras de ahorro, volúmenes y tiempos de tránsito, y agregar ejemplos concretos de rutas o cargas que estén cambiando de recorrido.

272/1200

Cancelar **Enviar observaciones**

El editor devuelve la noticia con una observación que queda registrada en el historial.

Valentina vuelve a entrar a la edición y ve el pedido del editor. Puede corregir el contenido y guardar esos cambios sin reenviar todavía la noticia. Cuando termina, pulsa Reenviar a revisión y la nota vuelve al estado En revisión.

Mateo vuelve a recibirla. Puede devolverla otra vez si todavía encuentra problemas o, si ya está lista, programarla para una fecha futura o publicarla. De esta forma la responsabilidad de corregir el contenido sigue siendo de la periodista y la decisión editorial queda en manos del editor.

Resumen del recorrido

```

Valentina - periodista
crea la noticia
↓
Borrador

Valentina completa los campos
y pulsa "Enviar a revisión"
↓
En revisión

Mateo - editor
abre la noticia
↓
detecta una cifra que debe verificarse

Mateo escribe una observación
↓
Devuelta para corrección

Valentina corrige y vuelve a enviar
↓
Mateo vuelve a revisar
↓
Publicar ahora o Programar

```

Estados que aparecen durante el flujo

Texto que ve el usuario	Valor que guarda TRAMA	Qué significa
En borrador	draft	El autor todavía está escribiendo.
En revisión	review	El periodista la envió y espera una decisión del editor.
Devuelta para corrección	needs_changes	El editor dejó una observación y el periodista debe corregir.
Programada	scheduled	Quedó preparada para publicarse en una fecha y hora.
Publicada	published	Puede aparecer en el portal.
Archivada	archived	Se retira del portal sin borrar la noticia ni su historial.

Qué puede cambiar el editor en una nota ajena

Cuando Mateo revisa una noticia escrita por Valentina puede decidir sobre su estado, pero no reemplaza directamente su título, bajada, resumen, cuerpo, categoría, etiquetas o portada. Si necesita correcciones, la devuelve con una observación para que la autora realice los cambios.

El editor sí puede escribir sus propias noticias. En ese caso puede guardarlas como borrador, programarlas o publicarlas sin pasar por la revisión de otro editor. Un periodista no puede publicar directamente.

Código fuente real: devolver una noticia para corrección

ArticleWorkflowService concentra las transiciones que necesitan varios cambios coordinados. El método requestChanges() muestra bien esta responsabilidad: el estado de la noticia, la observación del editor y el historial deben guardarse como una sola operación.

```
return DB::transaction(function () use ($article, $reviewer, $message): Article {
    $locked = Article::query()
        ->with('tags:id,name,slug')
        ->whereKey($article->id)
        ->lockForUpdate()
        ->firstOrFail();

    if ($locked->status !== 'review') {
        throw ValidationException::withMessages([
            'message' => 'Solo se puede devolver una noticia que está en revisión.',
        ]);
    }

    $previousSnapshot = $locked->revisionSnapshot();
    $previousStatus = $locked->status;

    $locked->update([
        'status' => 'needs_changes',
        'published_at' => null,
        'scheduled_at' => null,
        'is_breaking' => false,
        'is_featured' => false,
    ]);

    $locked->reviewFeedback()->create([
        'returned_by' => $reviewer->id,
        'message' => $message,
    ]);

    $this->revisions->record(
        $locked,
        $reviewer,
        'changes_requested',
        $previousStatus,
        $previousSnapshot,
    );

    return $locked->refresh();
}, attempts: 3);
```

DB::transaction() garantiza que los cambios se confirmen juntos. lockForUpdate() vuelve a leer y bloquea la fila antes de comprobar el estado, evitando que otra operación publique o archive la misma noticia mientras el editor la está devolviendo. La observación se crea después del cambio de estado dentro de la misma transacción y ArticleRevisionService registra el resultado en el historial.

Paso a paso: Valentina y Mateo completan una vuelta editorial

La misma noticia cambia de manos sin perder autoría, historial ni reglas de edición.

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina termina el borrador y pulsa Enviar a revisión. Laravel valida los requisitos del estado solicitado.
2	ArticleWorkflowService cambia la noticia a En revisión y registra la acción en el historial.
3	Valentina deja de poder editarla mientras espera una decisión del editor.

Paso	Qué hace la persona y qué ocurre internamente
4	Mateo abre la noticia desde su panel. Puede revisar contenido, configuración, historial y los controles editoriales que su rol habilita.
5	Si detecta un problema, escribe una observación y devuelve la nota para corrección. La noticia pasa a Devuelta para corrección y Valentina recupera la edición.
6	Valentina corrige el dato señalado y vuelve a enviarla. La devolución queda asociada al flujo y puede marcarse como resuelta.
7	Mateo puede programar o publicar la versión válida. La transición se ejecuta dentro de las reglas del servicio editorial y queda registrada como revisión.
8	Cuando la noticia queda Publicada, el portal público ya puede incluirla en sus consultas según fecha, categoría y tratamiento de portada.

17. Programación, publicación, archivo y portada

Programar una noticia

Paso a paso: Mateo programa una publicación



Modal de programación utilizado por el editor.

Paso	Qué ocurre
1	Mateo abre Programar noticia y elige fecha y hora con Vue DatePicker.
2	El valor estable se guarda en form.scheduled_at y el estado solicitado pasa a scheduled.
3	ArticleRequest exige fecha válida, posterior al momento de TRAMA y dentro de la ventana de 30 días.
4	ArticleWorkflowService registra el estado Programada y la revisión correspondiente.

Paso	Qué ocurre
5	La noticia queda bloqueada para edición mientras conserve ese estado.
6	Laravel Scheduler ejecuta <code>articles:publish-scheduled</code> ; al llegar el momento, la publicación automática vuelve a validar el estado actual dentro de una transacción.

Supongamos que la hora que está usando TRAMA es 19/07/2026 16:35. Mateo revisa una noticia y decide que se publique a las 16:40. Al confirmar, la noticia queda en estado Programada y se guarda la fecha y hora en `scheduled_at`. Mientras siga programada, el contenido normal queda bloqueado para evitar cambios silenciosos antes de salir.

Laravel tiene una herramienta llamada Scheduler que revisa tareas periódicas definidas por el proyecto. Cada minuto puede ejecutar el comando `articles:publish-scheduled`. Cuando una noticia ya alcanzó su `scheduled_at`, el comando vuelve a comprobarla dentro de una transacción y la convierte en Publicada. Esa acción se registra en el historial como `published_automatically`. Si se quiere probar localmente esta publicación automática sin esperar al programador de tareas de un servidor, puede mantenerse activo el Scheduler con `php artisan schedule:work`. Este comando sólo sirve para ejecutar las tareas periódicas de Laravel: no hace avanzar TramaClock ni es necesario para navegar por TRAMA.

Para probar localmente la publicación automática, mantener otra terminal abierta con:

```
php artisan schedule:work
```

El proceso revisa las tareas programadas mientras permanece activo. No hace avanzar TramaClock: sólo permite que Laravel ejecute `articles:publish-scheduled` cuando corresponde.

```
19/07/2026 16:35 → Programada
19/07/2026 16:39 → sigue Programada
19/07/2026 16:40 → el Scheduler puede publicarla
```

Limitación

actual

Para fechas posteriores al 19/07/2026 se aplica la limitación explicada en la sección 8: el formulario puede aceptarlas, pero TramaClock no puede avanzar al día siguiente con la configuración actual.

Cancelar una programación

Sólo el editor puede cancelar una programación. Si la noticia fue escrita por ese mismo editor, vuelve a Borrador. Si pertenece a un periodista, vuelve a En revisión. TRAMA elimina las fechas de programación y publicación que ya no corresponden y registra `schedule_cancelled` en el historial.

Publicar y archivar

Al publicar se establece `published_at` y la noticia puede aparecer en el portal si su categoría también cumple las reglas de visibilidad. Archivar cambia el estado a Archivada y retira la noticia del portal, pero conserva el registro y sus revisiones anteriores.

Urgente y Destacada

El editor puede marcar una noticia publicada o programada como Urgente o Destacada. En el código esas marcas se guardan como `is_breaking` e `is_featured`. La portada admite como máximo 7 noticias destacadas activas al mismo tiempo.

TRAMA también limita a 12 las categorías que pueden participar públicamente al mismo tiempo. Si una publicación hiciera aparecer una sección número 13, la operación se rechaza hasta que otra categoría deje de participar del portal.

Código fuente real: del Scheduler a la publicación automática

`routes/console.php` registra qué comando debe revisar Laravel y con qué frecuencia. Esta configuración no inicia por sí sola el Scheduler; solamente define la tarea:

```
Schedule::command('articles:publish-scheduled')
    ->everyMinute()
    ->withoutOverlapping();
```

`everyMinute()` indica la frecuencia. `withoutOverlapping()` evita iniciar una segunda ejecución del mismo comando mientras la anterior siga activa. Cuando el Scheduler lo ejecuta, `PublishScheduledArticles` busca las noticias programadas cuya fecha ya llegó. El primer fragmento muestra la consulta y `chunkByld(100, ...)`, que las procesa en lotes de hasta 100 registros:

```
Article::query()
    ->where('status', 'scheduled')
    ->whereNotNull('scheduled_at')
    ->where('scheduled_at', '<=', $editorialNow)
    ->orderBy('id')
    ->select('id')
    ->chunkByld(100, function ($articles) use ($revisions, $categoryOrder, $editorialNow, &$published):
void {
    foreach ($articles as $articleReference) {
```

Dentro de cada lote, el `foreach` procesa una noticia por vez. La publicación se ejecuta dentro de una transacción: antes de cambiar el estado, el comando vuelve a leer la fila con `lockForUpdate()` y comprueba nuevamente que siga programada y que su fecha ya haya llegado:

```
$wasPublished = DB::transaction(function () use ($articleReference, $revisions, $categoryOrder,
$editorialNow): bool {
    $article = Article::query()
        ->with('tags:id,name,slug')
        ->whereKey($articleReference->id)
        ->lockForUpdate()
        ->first();

    if (
        ! $article
        || $article->status !== 'scheduled'
        || $article->scheduled_at === null
        || $article->scheduled_at->gt($editorialNow)
    ) {
        return false;
    }
}
```

El bloqueo evita que dos ejecuciones decidan al mismo tiempo sobre la misma noticia. Fuera de estos fragmentos, el método valida el cupo público de categorías, guarda el snapshot anterior, publica la noticia y registra `published_automatically` en el historial. Si una regla lanza `ValidationException`, esa noticia permanece programada y el resto del lote continúa.

18. Historial de la noticia, revisiones y restauración

Cada vez que se realiza un guardado importante, TRAMA puede conservar una copia de cómo quedó la noticia. Así es posible saber qué tenía antes, qué cambió, quién realizó el cambio y qué estado tenía en ese momento. Esas copias se guardan en `article_revisions`.

Ejemplo de una revisión

Versión anterior
 Título: Nuevas rutas comerciales reordenan la logística regional

 Versión posterior
 Título: Nuevos corredores comerciales reordenan la logística regional

articles conserva el estado actual de la noticia. `article_revisions` permite consultar también las versiones anteriores. En el código, la copia de los campos de una noticia en un momento determinado se llama `snapshot`.

Campo de la revisión	Qué significa
<code>snapshot</code>	Copia del contenido y de los datos importantes de la noticia en ese momento.
<code>changed_fields</code>	Lista de campos que cambiaron respecto de la revisión anterior.
<code>captured_at</code>	Fecha y hora en que se generó esa copia.
<code>status_from</code> / <code>status_to</code>	Estado anterior y estado posterior de la acción.
<code>user_id</code>	Cuenta que realizó la acción registrada.

Qué acciones dejan una revisión

- La creación inicial.
- Un guardado manual que cambia información.
- Enviar o reenviar a revisión.
- Devolver una noticia para corrección.
- Programar, publicar o cancelar una programación.
- La publicación automática.
- Cambiar las marcas de portada.
- Archivar.
- Restaurar una versión anterior.

El autoguardado es distinto: protege el trabajo en curso, pero no agrega una revisión cada 15 segundos. De esta forma Historial no se llena con cientos de cambios mínimos.

Observaciones del editor

Cuando Mateo devuelve una nota a Valentina y escribe “Verificar la cifra del segundo párrafo”, la observación se guarda en `article_review_feedback` junto con quién la envió. Cuando Valentina corrige la noticia y la vuelve a enviar, esa devolución puede quedar marcada como resuelta.

Restaurar una versión

Sólo el editor puede restaurar desde Historial y no puede hacerlo sobre un borrador ajeno. Restaurar no borra lo que ocurrió después. TRAMA toma la revisión elegida y crea un nuevo Borrador basado en ella, limpia las fechas públicas, recupera categoría, etiquetas y portada, vuelve a preparar `search_text` y registra una nueva revisión `restored_as_draft`.

Código fuente real: cómo se guarda una revisión

`app/Services/Editorial/ArticleRevisionService.php` **gestiona la creación de revisiones**. La noticia actual se convierte en un `snapshot` y se compara con el `snapshot` anterior para identificar qué campos cambiaron:

```

$article->load('tags:id,name,slug');
$snapshot = $article->revisionSnapshot();

return $article->revisions()->create([
    'user_id' => $user->id,
    'action' => $action,
    'status_from' => $statusFrom,
    'status_to' => $article->status,
    'snapshot' => $snapshot,
    'changed_fields' => $this->changedFields($previousSnapshot, $snapshot),
    'restored_from_revision_id' => $restoredFrom?->id,
]);

```

snapshot conserva el estado que queda después de la acción. previousSnapshot representa el estado anterior y changedFields() determina qué campos cambiaron entre ambos estados. restored_from_revision_id permite conservar qué revisión histórica fue el origen de una restauración.

Código fuente real: restaurar no reemplaza directamente una publicación

ArticleWorkflowService::restore() toma sólo los campos editoriales permitidos del snapshot elegido y fuerza el resultado a Borrador. Después restaura etiquetas, reconstruye search_text y crea una nueva revisión:

```

$restorable = Arr::only($snapshot, [
    'title', 'subtitle', 'excerpt', 'body', 'category_id',
    'cover_image', 'cover_alt', 'is_breaking', 'is_featured',
]);

$locked->update([
    ...$restorable,
    'status' => 'draft',
    'published_at' => null,
    'scheduled_at' => null,
]);

$locked->tags()->sync(array_map('intval', $snapshot['tag_ids'] ?? []));
$this->searchIndexer->index($locked);
$this->revisions->record(
    $locked,
    $user,
    'restored_as_draft',
    $previousStatus,
    $previousSnapshot,
    $lockedRevision,
);

```

La versión histórica no vuelve a publicarse de forma silenciosa. Se crea un nuevo estado editable y el propio acto de restaurar queda registrado, de modo que el historial conserva tanto lo ocurrido originalmente como la restauración posterior.

Paso a paso: Mateo restaura una revisión sin borrar el historial anterior

La restauración no convierte mágicamente una publicación vieja en el estado actual: crea un nuevo punto de trabajo controlado.

Paso	Qué hace la persona y qué ocurre internamente
1	Mateo abre la pestaña Historial de una noticia y selecciona una revisión anterior.
2	La interfaz muestra qué campos tenía esa versión y qué valores cambiaron respecto de otras revisiones.
3	Al elegir Restaurar, Laravel vuelve a comprobar permisos y el estado actual de la noticia.
4	ArticleWorkflowService toma los datos recuperables de la revisión y los aplica dentro de una transacción protegida.

Paso	Qué hace la persona y qué ocurre internamente
5	La noticia restaurada vuelve como un nuevo borrador editable en lugar de sobrescribir silenciosamente una publicación ya visible.
6	El propio acto de restaurar genera una nueva revisión, de modo que el historial conserva tanto la versión antigua como la decisión de recuperarla.
7	Mateo o el autor pueden seguir trabajando desde ese borrador y recorrer nuevamente el flujo editorial normal.

19. Categorías y etiquetas

Categorías y etiquetas sirven para organizar noticias, pero cumplen funciones diferentes. Una categoría es una sección principal del diario, por ejemplo Política o Economía. Una etiqueta representa un tema que puede aparecer en noticias de distintas secciones, por ejemplo Inteligencia artificial.

Categorías

Ejemplo práctico: Marina crea una categoría

CONTENIDO ✕

Nueva categoría

Creá una sección completa para utilizarla en nuevas noticias.

Imagen de portada

JPG, JPEG, PNG o WEBP · hasta 5 MB · mínimo 1600 × 900 px; TRAMA valida, recorta y guarda automáticamente como WebP.

 **Seleccionar archivo** sociedaduno..jpg

Nombre

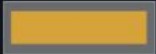
Sociedad

Descripción

Educación, salud, ambiente, comunidad y vida cotidiana.

Un único párrafo, sin saltos de línea. 55/120

Color

 La categoría se crea ACTIVA para uso editorial. Al publicar su primera noticia pasa a PUBLICADA y recibe orden editorial. 6 de 12 categorías publicadas.

Cancelar **Crear categoría**

Formulario Nueva categoría del panel a dministrador.

La creación de categorías también usa useForm en Admin/Categories/Index.vue. createForm conserva nombre, descripción, color e imagen de portada; form.processing bloquea el botón durante el envío y form.errors muestra los problemas devueltos por Laravel. Las categorías nuevas se crean activas desde el backend, por lo que el estado no forma parte de createForm.

Nombre

tecnología

Ya existe una categoría con ese nombre.

El nombre duplicado se muestra junto al campo que debe corregirse.

Cada noticia pertenece a una categoría. El administrador puede crear y editar su nombre, descripción, color, portada y estado. Una categoría participa del portal sólo cuando está activa, tiene completos sus datos públicos y posee al menos una noticia publicada disponible.

CategoryEditorialOrderService mantiene el orden público 1, 2, 3... Si una categoría deja de mostrarse, las demás se vuelven a numerar para que no queden posiciones vacías. TRAMA admite como máximo 12 categorías públicas al mismo tiempo.

Qué es un Observer y por qué TRAMA lo usa con Article

Un Observer es una clase que reacciona automáticamente ante eventos de un Model. En lugar de obligar a cada Controller o Service a recordar una tarea secundaria, Laravel puede avisar al Observer cuando un registro se guarda, elimina, restaura o se borra de forma definitiva.

TRAMA registra ArticleObserver desde AppServiceProvider. El Observer presta atención a cambios capaces de afectar la disponibilidad pública de una categoría —por ejemplo estado, fecha de publicación o category_id— y, una vez confirmado el cambio en la base, llama a CategoryEditorialOrderService para volver a sincronizar el orden público.

La expresión “después del commit” significa que el Observer espera a que la transacción se confirme antes de actuar. Así no reorganiza categorías a partir de un cambio que finalmente podría haberse revertido.

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina guarda una noticia y el cambio queda confirmado en MySQL.
2	Laravel detecta el evento del Model Article.
3	ArticleObserver comprueba si cambió un dato relevante para la visibilidad pública.
4	Si corresponde, llama a CategoryEditorialOrderService.
5	El servicio vuelve a calcular la secuencia pública de categorías para evitar posiciones vacías o incoherentes.

Una categoría no puede eliminarse mientras tenga alguna noticia asociada. En ese caso puede desactivarse, pero no borrarse de la base.

Página pública de una categoría

Cuando un lector abre /seccion/{slug}, Public\CategoryController::show() busca una categoría que cumpla las reglas de visibilidad pública. La sección debe estar activa y tener contenido publicado disponible. Después carga únicamente noticias públicamente visibles de esa categoría, ordenadas desde la más reciente, y las muestra de a 9 por página en Public/CategoryShow.vue. CategoryResource prepara los datos de la sección y ArticleCardResource prepara cada noticia.

Ejemplo: Candela abre la sección Economía. Si Economía está activa, TRAMA muestra su encabezado y las primeras 9 noticias públicas; las siguientes quedan en las páginas posteriores. Si la categoría se desactiva, /seccion/economia deja de ser una página pública válida y Laravel responde con 404 en lugar de mostrar una sección que el administrador retiró del portal.

Etiquetas y relación muchos-a-muchos

Una noticia puede tener varias etiquetas y una misma etiqueta puede utilizarse en muchas noticias. Por ejemplo, una noticia de Economía puede usar IA, Empresas y Estados Unidos; al mismo tiempo, IA puede aparecer también en noticias de Tecnología o Política.

En una base de datos este tipo de relación se llama muchos-a-muchos. TRAMA la representa mediante `article_tag`, una tabla que guarda cada combinación entre una noticia y una etiqueta.

Fusionar etiquetas duplicadas

Ejemplo práctico: Marina fusiona dos etiquetas

`Admin/Tags/Index.vue` mantiene un `mergeForm` independiente con `target_tag_id`. Esto evita mezclar los errores y el estado de procesamiento de la fusión con los formularios de alta y edición. Laravel realiza después la operación transaccional que mueve relaciones y conserva la trazabilidad de la etiqueta fusionada.

IA	Inteligencia artificial, automatización y nuevos modelos de uso.	12	ACTIVA				
Inteligencia Artificial	Inteligencia artificial, automatización, modelos generativos y nuevas tecnologías.	0	ACTIVA				

Resultado de una fusión de etiquetas dentro de la grilla.

Supongamos que existen las etiquetas IA e Inteligencia Artificial y el administrador decide conservar IA como principal. `TagMergeService` mueve hacia IA las relaciones de las noticias que usaban Inteligencia Artificial, evita crear combinaciones duplicadas y deja la etiqueta anterior inactiva con `merged_into_id` apuntando a la principal.

```

ANTES
Noticia 10 → Inteligencia Artificial
Noticia 11 → IA

DESPUÉS
Noticia 10 → IA
Noticia 11 → IA

Inteligencia Artificial.merged_into_id → IA

```

Una etiqueta tampoco puede eliminarse si todavía tiene noticias asociadas, si fue fusionada con otra o si otras etiquetas fueron fusionadas dentro de ella.

Código fuente real: cómo se fusionan etiquetas sin duplicar relaciones

El fragmento siguiente muestra el núcleo del movimiento de relaciones. `insertOrIgnore()` evita crear una combinación duplicada cuando una noticia ya utiliza la etiqueta principal; después se eliminan los vínculos con la etiqueta secundaria.

```

$articleIds = DB::table('article_tag')
    ->where('tag_id', $source->id)
    ->pluck('article_id');

foreach ($articleIds as $articleId) {
    DB::table('article_tag')->insertOrIgnore([
        'article_id' => $articleId,
        'tag_id' => $target->id,
    ]);
}

DB::table('article_tag')
    ->where('tag_id', $source->id)
    ->delete();

```

`insertOrIgnore()` evita violar la clave única de `article_tag` cuando una noticia ya utiliza ambas etiquetas. Después se eliminan los vínculos con la etiqueta secundaria. La desactivación de la etiqueta origen y su `merged_into_id` ya fueron explicados en el ejemplo anterior, por lo que no se repiten en este fragmento.

Paso a paso: Marina crea una categoría y resuelve después una etiqueta duplicada

Categorías y etiquetas cumplen funciones distintas, pero ambas muestran cómo la administración modifica datos que luego reutilizan noticias y búsquedas.

Paso	Qué hace la persona y qué ocurre internamente
1	Marina abre Categorías y completa nombre, descripción, color e imagen. <code>useForm</code> mantiene esos valores y bloquea el botón mientras se procesa la petición.
2	Laravel valida que el nombre sea válido y no esté duplicado. Si falla, <code>Inertia</code> devuelve el error al campo correspondiente.
3	Si la categoría se crea correctamente, <code>CategoryEditorialOrderService</code> la incorpora al orden editorial y MySQL guarda la nueva fila.
4	Las noticias pueden usar esa categoría mediante <code>category_id</code> y el portal podrá mostrarla cuando esté activa y tenga contenido público.
5	Más adelante Marina detecta dos etiquetas equivalentes, por ejemplo IA e Inteligencia Artificial, y abre la acción Fusionar.
6	El servicio de fusión mueve las relaciones <code>article_tag</code> hacia la etiqueta principal sin crear combinaciones duplicadas.
7	La etiqueta secundaria queda resuelta según las reglas del módulo y las noticias conservan su contenido asociado.
8	<code>ArticleSearchIndexer</code> puede reconstruir <code>search_text</code> para que la búsqueda del CMS utilice los nombres definitivos de categoría y etiquetas.

20. Imágenes: validación, procesamiento y recuperación

Las imágenes que el equipo carga en TRAMA pasan por un recorrido antes de quedar disponibles en el sitio. Laravel valida el archivo recibido, solicita la clasificación local de seguridad y, si el contenido es aceptado, `Node.js` y `Sharp` generan la versión final con el formato y las dimensiones que necesita cada módulo. Recién entonces se conserva el archivo procesado y se registra en MySQL la ruta que utilizará la aplicación.

Qué es WebP

WebP es un formato de imagen que permite mantener buena calidad con un peso de archivo reducido. TRAMA convierte a WebP las portadas, las fotos de empleados, las portadas de categorías y las publicidades procesadas.

Proceso común

Paso	Qué ocurre
1	Laravel comprueba que el archivo tenga un formato admitido, que no supere el tamaño máximo y que posea dimensiones suficientes.
2	<code>SafedImageProcessor</code> solicita una clasificación local del contenido antes de transformar la imagen.

Paso	Qué ocurre
3	ArticleImageSafetyPolicy compara las puntuaciones obtenidas con los límites configurados por TRAMA y decide si el archivo puede continuar.
4	Si la imagen es válida, Node.js ejecuta scripts/images/process-image.mjs. Sharp corrige la orientación, recorta cuando corresponde, redimensiona y convierte el resultado a WebP.
5	El archivo final se guarda en la carpeta configurada y Laravel conserva en MySQL la ruta pública que utilizará la interfaz.

En una portada de noticia, el recorrido puede leerse así: archivo original → validación de Laravel → clasificación de seguridad → procesamiento con Sharp → WebP 1600 × 900 → guardado del archivo → ruta registrada en la noticia.

EXIF y orientación de una foto

Algunos teléfonos no giran físicamente los píxeles de una foto vertical. Guardan dentro del archivo un dato que indica cómo debe mostrarse. Ese dato forma parte de los metadatos EXIF. Antes de recortar, Sharp aplica esa orientación para evitar que una fotografía termine de costado.

Clasificación de seguridad

El modelo local devuelve tres valores. SFW representa contenido considerado seguro; NSFW indica contenido sexual o adulto; NSFL se utiliza para contenido gráfico extremo. TRAMA bloquea una imagen cuando la probabilidad de NSFW o NSFL alcanza el 80 % configurado para esa categoría.

Uso	Archivo de entrada	Resultado
Portada de noticia	JPG/JPEG/PNG/WebP · máx. 5 MB · resolución suficiente	WebP 1600 × 900 · calidad 80
Foto de empleado	JPG/JPEG/PNG/WebP · máx. 4 MB · mínimo 600 × 600	WebP 600 × 600 · calidad 82
Portada de categoría	Imagen válida con resolución suficiente	WebP 1600 × 900
Publicidad	Imagen válida y tamaño exacto según su ubicación	WebP en el tamaño requerido

Qué es Sharp y qué función cumple

Sharp es una librería de Node.js especializada en procesamiento de imágenes. En TRAMA no decide si una imagen es segura ni ejecuta reglas editoriales. Se utiliza para leer y transformar píxeles: corregir orientación EXIF, redimensionar, recortar, eliminar canal alfa cuando el modelo lo necesita y generar archivos WebP.

Sharp no es el clasificador

Sharp realiza transformaciones deterministas sobre la imagen. ONNX Runtime es quien ejecuta el modelo de aprendizaje automático. Laravel aplica después la política que decide si la imagen puede continuar.

Cómo se ejecuta la clasificación de seguridad desde PHP

ArticleImageSafetyClassifier.php es la pieza que conecta Laravel con scripts/images/classify-image-safety.mjs. El servicio obtiene la ruta de Node desde config('trama.node_binary'), establece la raíz del proyecto como directorio de trabajo y ejecuta el script pasando la ruta absoluta de la imagen como argumento independiente.

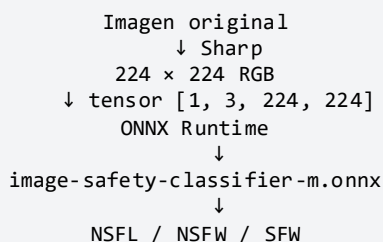
```
$result = Process::path(base_path())
->timeout(self::PROCESS_TIMEOUT_SECONDS)
```

```
->env([
    'SystemRoot' => getenv('SystemRoot'),
    'PATH' => getenv('PATH'),
])
->run([
    (string) config('trama.node_binary'),
    $scriptPath,
    $absoluteImagePath,
]);
```

El comando equivale conceptualmente a `node scripts/images/classify-image-safety.mjs "ruta/de/la/imagen"`. El array de argumentos evita construir manualmente una cadena de shell. `SystemRoot` y `PATH` se transmiten expresamente porque el entorno original de Windows los necesita para que Node y sus dependencias se inicialicen correctamente.

Qué hace `classify-image-safety.mjs`

El script recibe una ruta física, prepara la imagen con Sharp y ejecuta el modelo local mediante `onnxruntime-node`. La imagen se rota según EXIF, se lleva a 224 × 224 píxeles, se convierte a RGB sin alfa y se reorganizan sus valores en un tensor [1, 3, 224, 224], que representa una imagen, tres canales, altura y ancho.



El script devuelve por stdout un JSON con la categoría de mayor probabilidad y las probabilidades NSFL, NSFW y SFW. `ArticleImageSafetyClassifier` convierte ese JSON en un array PHP y valida su estructura. La decisión de aceptar o bloquear no se toma en Node: `ArticleImageSafetyPolicy` compara después las probabilidades con los límites definidos por TRAMA.

```
$classification = json_decode(
    trim($result->output()),
    true,
    512,
    JSON_THROW_ON_ERROR
);
```

Cómo se ejecuta el procesamiento final de la imagen

Cuando la imagen ya fue aceptada, `SafelImageProcessor.php` inicia `scripts/images/process-image.mjs`. En este caso los argumentos incluyen origen, destino, ancho, alto y calidad WebP.

```
$result = Process::path(base_path())
->timeout(self::PROCESS_TIMEOUT_SECONDS)
->env([
    'SystemRoot' => getenv('SystemRoot'),
    'PATH' => getenv('PATH'),
])
->run([
    (string) config('trama.node_binary'),
    $scriptPath,
    $absoluteSourcePath,
    $workingDestination,
    (string) $width,
    (string) $height,
    (string) $quality,
]);
```

`process-image.mjs` comprueba que el origen exista, valida dimensiones y parámetros, corrige la orientación, aplica `resize` con `fit=cover` sin ampliar imágenes pequeñas y escribe el WebP final. Después devuelve por `stdout` un JSON con la ruta, ancho, alto y formato. Laravel comprueba esa respuesta antes de continuar.

Resumen: clasificación y procesamiento son dos etapas distintas

Etapas	Script	Tecnologías	Resultado
Seguridad	<code>classify-image-safety.mjs</code>	Sharp + ONNX Runtime	Probabilidades NSFL / NSFW / SFW en JSON.
Decisión	<code>ArticleImageSafetyPolicy</code> (PHP)	Reglas de Laravel	Acepta o bloquea según los umbrales.
Archivo final	<code>process-image.mjs</code>	Sharp	WebP con dimensiones y calidad requeridas.

Paso a paso: Valentina carga una portada aceptada

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina selecciona una imagen en <code>Admin/Articles/Form.vue</code> y guarda la noticia. El archivo viaja en el formulario <code>Inertia</code> como <code>cover_image_file</code> junto con el resto de los campos editoriales.
2	<code>app/Http/Requests/Admin/ArticleRequest.php</code> valida la petición en Laravel. Para la portada acepta JPG, JPEG, PNG o WebP y limita el archivo a 5 MB; las demás reglas del artículo dependen del estado que Valentina intenta guardar.
3	<code>ArticleController::store()</code> recibe el <code>ArticleRequest</code> ya validado. Si <code>hasFile("cover_image_file")</code> es verdadero, llama a <code>processCover()</code> , que transforma cualquier <code>RuntimeException</code> del procesamiento en un error de validación asociado al propio campo <code>cover_image_file</code> .
4	<code>ArticleCoverProcessor</code> prepara el destino de la portada: 1600 × 900 píxeles, calidad WebP 80 y un nombre disponible basado en el slug estable de la noticia. Después delega el trabajo compartido en <code>SafedImageProcessor</code> .
5	<code>SafedImageProcessor</code> comprueba que el archivo exista y que sus dimensiones visuales, incluida la orientación EXIF, alcancen al menos 1600 × 900. Luego <code>ArticleImageSafetyClassifier</code> ejecuta <code>scripts/images/classify-image-safety.mjs</code> : Sharp prepara un tensor 224 × 224 y ONNX Runtime devuelve probabilidades NSFL, NSFW y SFW.

Paso	Qué hace la persona y qué ocurre internamente
6	ArticleImageSafetyPolicy compara las probabilidades con los umbrales configurados. Si la imagen es aceptada, SafeImageProcessor ejecuta scripts/images/process-image.mjs. Sharp corrige la orientación, recorta con fit=cover sin ampliar una imagen pequeña y genera el WebP final de 1600 × 900 con calidad 80.
7	Laravel comprueba el JSON devuelto por Node.js, las dimensiones, el formato y la existencia del archivo generado. ArticleController añade la URL y el texto alternativo a los datos de la noticia y ArticleWorkflowService::create() guarda la noticia, sincroniza etiquetas y registra el recurso visual correspondiente.
8	Si todo termina correctamente, la portada queda referenciada desde MySQL y puede mostrarse en el panel y el portal. Si el archivo llegó a generarse pero después falla el guardado de la noticia, ArticleController elimina esa portada nueva para no dejar un archivo huérfano. Si el procesamiento falla antes, el error vuelve debajo del campo de portada.

```
Archivo seleccionado: portada-original.jpg
Tamaño: 2400 × 1350
```

```
TRAMA comprueba:
```

```
✓ formato admitido
✓ menos de 5 MB
✓ dimensiones suficientes
✓ contenido permitido
```

```
Resultado:
1600 × 900
WebP
calidad 80
```

Qué pasa si alguien borra una imagen a mano

MySQL puede seguir guardando una ruta aunque el archivo físico ya no exista. Por ejemplo, cover_image puede seguir indicando /images/articles/rutas-comerciales.webp aunque alguien haya borrado public/images/articles/rutas-comerciales.webp.

En el portal, Article::publicCoverImageUrl() utiliza noticia-default.webp para no mostrar una imagen rota. En el panel, TRAMA detecta que la ruta existe pero el archivo falta y obliga a volver a cargar la portada antes de completar una acción que la necesita. Con la foto de un empleado ocurre algo parecido: User::avatarUrl() utiliza avatar-default.webp y el formulario de edición puede exigir una nueva foto.

MySQL y los archivos no se deshacen juntos

Supongamos que TRAMA ya generó una portada nueva en el disco y luego falla el guardado de la noticia en MySQL. MySQL puede revertir sus propios cambios, pero no puede borrar por sí mismo ese archivo. Por eso ArticleController elimina la portada nueva cuando el guardado posterior falla.

El comportamiento de “todo o nada” que MySQL garantiza dentro de una transacción se denomina atomicidad. El sistema de archivos está fuera de esa transacción, por lo que TRAMA agrega código específico para limpiar archivos nuevos o restaurar una versión anterior cuando corresponde.

21. Comentarios y participación de usuarios registrados

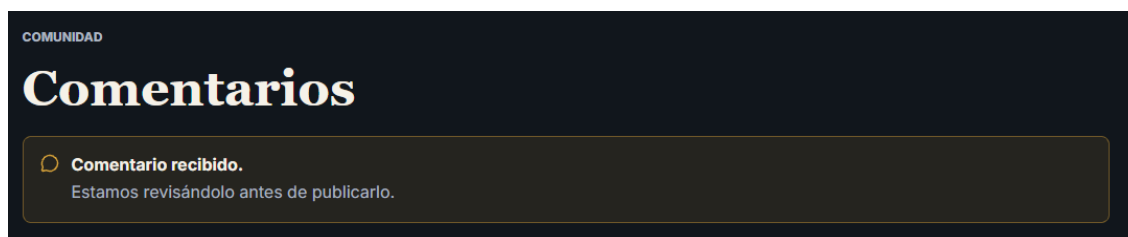
Un usuario registrado puede escribir un comentario principal en una noticia y responder comentarios de otros usuarios. Antes de aparecer públicamente, sus comentarios y respuestas pasan por una revisión automática.

Ejemplo: enviar un comentario

Candela escribe “Muy buena nota, explica claramente el problema” y pulsa Enviar comentario. Laravel comprueba primero la longitud, los permisos y las reglas de participación. Si todo es válido, crea una fila en comments con status=processing. Eso significa que el comentario ya existe en MySQL, pero todavía no se muestra públicamente.

Antes de guardar, CommentController comprueba que la noticia siga publicada, normaliza el texto y valida que tenga entre 8 y 1200 caracteres. También verifica que la cuenta esté activa, que tenga permiso para participar y que no exista otro comentario principal activo de esa misma cuenta en la noticia. Si se trata de una respuesta, comprueba además que el comentario padre sea un comentario principal aprobado y que la respuesta cumpla las reglas del hilo.

Para un lector normal, la fila se crea dentro de una transacción con status=processing, moderation_revision=1, moderation_source=automatic y moderation_reason=automation_queued. Después Laravel despacha ModerateComment a la cola moderation y devuelve al navegador el mensaje de revisión junto con el ID del comentario que la interfaz debe seguir. El análisis automático ocurre después; la petición web no espera a que terminen los modelos.



Mensaje que ve el usuario mientras el comentario está siendo revisado.

Estado	Qué significa
processing / En proceso	El comentario está guardado, pero el análisis todavía no terminó.
approved / Aprobado	El comentario aparece públicamente.
pending / Pendiente	Sólo lo ve su autor y pasa al editor para revisión manual.
rejected / Rechazado	No aparece en la conversación pública.

Reglas de la conversación

Candela puede escribir un único comentario principal activo por noticia. Pablo puede responderle una vez. Candela no puede responder su propio comentario principal. Una respuesta no puede recibir otra respuesta: TRAMA mantiene un solo nivel de profundidad. Si Pablo elimina su respuesta, puede volver a responder ese comentario.

Los estados processing, pending y approved cuentan como participación activa. Si el comentario principal queda rejected, el usuario puede intentar escribir otro.

Editar un comentario pendiente

Sólo puede editarse un comentario que quedó Pendiente de moderación. Si todavía está processing, hay que esperar. Si ya fue aprobado o rechazado, no se edita desde este flujo. Al modificar un comentario pendiente, vuelve a processing, aumenta `moderation_revision` y se analiza otra vez desde cero.

Límites de uso

Acción	Límite
Crear comentario o respuesta	5 intentos cada 10 minutos.
Editar o eliminar una participación propia	20 acciones por minuto.
Dar o quitar Me gusta	10 acciones cada 5 minutos.
Reportar comentarios	10 intentos cada 10 minutos.

Me gusta y reportes

Un usuario no puede dar Me gusta a su propio comentario y MySQL impide guardar dos votos iguales de la misma cuenta sobre el mismo comentario. Reportar no elimina automáticamente el contenido: crea un caso para el editor. La misma cuenta no puede reportar dos veces el mismo comentario ni reportar su propio contenido.

Si un usuario había dado Me gusta y después reporta ese mismo comentario, TRAMA elimina el Me gusta antes de guardar el reporte para no dejar las dos acciones activas al mismo tiempo.

“Mis comentarios”: historial personal de participación

La ruta `/mis-comentarios` abre una pantalla privada para la cuenta autenticada. `MyCommentController` consulta únicamente los comentarios de ese usuario, los ordena desde el más reciente y los muestra en páginas de 10. La grilla muestra fragmento, noticia, estado, cantidad de Me gusta y cantidad de respuestas.

Estado	Cómo aparece en “Mis comentarios”
processing	Se muestra para confirmar que TRAMA recibió la participación mientras la moderación automática sigue ejecutándose.
pending	Se muestra como pendiente y permite al usuario reconocer qué participación está esperando revisión manual.
approved	Se muestra con enlace a la noticia; el enlace incluye <code>?comentario={id}</code> para que Vue pueda localizar y centrar esa participación.
rejected	Sólo se conservan en esta grilla los rechazos cuyo <code>moderation_source</code> es editorial. No se genera enlace público y el detalle se abre dentro de la propia pantalla.

El enlace a la noticia tampoco se genera si la noticia dejó de estar publicada, todavía no alcanzó su fecha pública o su categoría está inactiva. Si el comentario apuntado quedó fuera de los primeros 10 comentarios que normalmente carga `ArticleController`, el controlador lo incorpora de forma adicional para que el salto desde “Mis comentarios” funcione.

Ejemplo

Candela abre “Mis comentarios” y selecciona una participación aprobada de una noticia con 40 comentarios. La URL se construye como `/noticias/{slug}?comentario=123`. `ArticleController` busca el comentario 123, comprueba que Candela pueda verlo y, aunque no estuviera dentro de la primera tanda, lo agrega a los datos enviados a `ArticleShow.vue`. La interfaz puede entonces llevar al usuario directamente a su participación.

Paso a paso: Candela envía un comentario y TRAMA prepara la moderación

La participación pública parece una acción simple, pero antes de mostrarse atraviesa varias reglas de cuenta, noticia y estado.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela inicia sesión, abre una noticia publicada y escribe un comentario principal.
2	Vue envía la petición a la ruta de comentarios. Laravel comprueba autenticación, límites de uso y que Candela no tenga otra participación principal activa en esa noticia.
3	Si los datos son válidos, CommentController guarda la fila en comments con status=processing, moderation_revision=1 y origen automático.
4	La petición web no espera a que terminen todos los modelos: Laravel despacha ModerateComment a la cola moderation.
5	Candela vuelve a la noticia y ve que su participación está siendo revisada en lugar de quedar publicada inmediatamente.
6	Un worker toma el Job y ejecuta el pipeline automático explicado en el capítulo siguiente.
7	Cuando el estado cambia, la pantalla puede actualizar la información correspondiente y mostrar la participación como aprobada, pendiente o rechazada según la decisión.

22. Moderación automática de comentarios

Cuando un usuario envía un comentario, TRAMA no obliga al navegador a esperar mientras se ejecutan todos los controles. Laravel guarda primero la participación y deja el análisis como una tarea pendiente. Esa tarea entra en una cola y un proceso separado, llamado worker, la toma para ejecutar la moderación en segundo plano.

Cómo se pone en marcha el worker de moderación

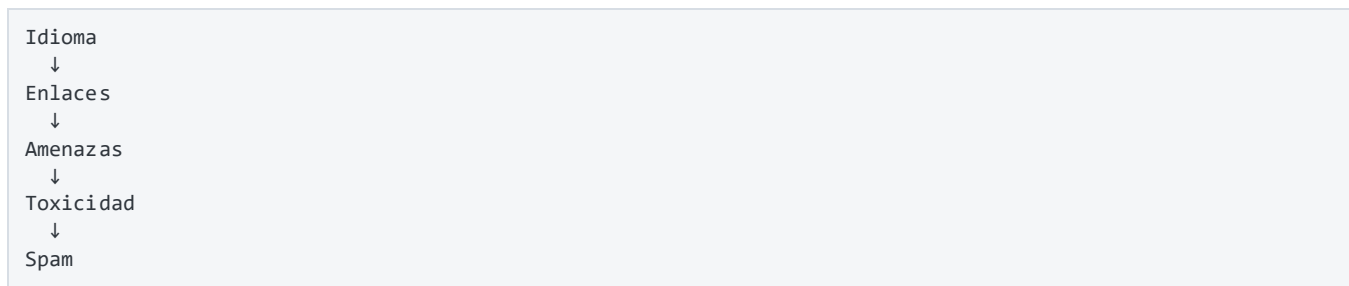
Despachar ModerateComment sólo deja el trabajo pendiente en la cola. Para que ese Job se ejecute durante el desarrollo debe existir un worker consumiendo las colas moderation y default.

```
php artisan queue:work --queue=moderation,default --tries=3 --timeout=70
```

Si el worker no está activo, el comentario permanece guardado con status=processing y el Job espera en la cola hasta que un worker vuelva a procesarlo. Job, cola y worker son piezas distintas: ModerateComment es el trabajo; moderation es la cola; y queue:work es el proceso que consume esa cola.

Paso	Qué ocurre
1	CommentController guarda el comentario como processing.
2	Laravel despacha ModerateComment como un Job y lo deja pendiente en la cola moderation.
3	El navegador ya puede mostrar “Comentario recibido”.
4	El worker toma el trabajo.
5	Node.js ejecuta scripts/comments/moderate-comment.mjs.
6	Laravel recibe el resultado y actualiza el mismo comentario.

Secuencia de análisis



Scripts que controlan la moderación

Script	Qué hace
moderate-comment.mjs	Es el único script que Laravel ejecuta directamente. Recibe el ID del comentario, moderation_revision y el texto; valida la entrada, coordina el pipeline y devuelve una sola decisión JSON: approved, pending o rejected.
detect-comment-language.mjs	Analiza si el texto puede tratarse como español mediante reglas lingüísticas y franc como señal estadística secundaria. Un idioma extranjero o una mezcla confirmada se deriva a pending. Si el idioma queda unknown por ser corto o insuficiente, los controles de seguridad pueden continuar.
detect-comment-links.mjs	Es determinista y no usa IA. Busca URLs normales y formas camufladas, como dominios sin protocolo, puntos ocultos o protocolos modificados. Si encuentra un enlace, el coordinador rechaza el comentario y corta el pipeline.
detect-comment-threats.mjs	Combina reglas de contexto y estructura con un modelo ONNX local como segunda opinión selectiva. Una amenaza HIGH produce rechazo; MEDIUM o review_recommended deja una razón provisional de revisión manual y el análisis puede continuar.
classify-comment-toxicity.mjs	Usa un modelo DistilBERT multilingüe en ONNX junto con reglas de contexto, normalización y detección de ataques personales. review_recommended deriva a revisión; una toxicidad de 0,70 o más, sin esa recomendación, puede producir rechazo automático.
classify-comment-spam.mjs	Usa un modelo ONNX de spam en español y reglas para ofertas, captación, promociones, phishing y otras señales. review_recommended deriva a revisión; un score de 0,78 o más, sin esa recomendación, puede producir rechazo automático.

Laravel no inicia cinco procesos Node.js. ModerateComment hace una sola llamada mediante Process::run() al coordinador moderate-comment.mjs. A partir de ahí, ese coordinador llama a los demás módulos dentro de la misma ejecución y devuelve un único resultado a PHP.

Idioma y enlaces se cargan al comenzar porque son las primeras comprobaciones. Amenazas, toxicidad y spam se importan sólo si el comentario llega hasta esas etapas. De esta forma TRAMA evita cargar modelos más pesados cuando una comprobación anterior ya resolvió el caso. Cuando los archivos están instalados, los clasificadores buscan

sus modelos y tokenizadores en `resources/models` y los ejecutan localmente mediante `onnxruntime-node`; este flujo no necesita un servidor HTTP de moderación.

No siempre se ejecutan todos los pasos. Si se detecta un enlace prohibido, por ejemplo, el comentario puede rechazarse en esa etapa sin cargar los analizadores posteriores. Una señal dudosa puede dejar el comentario Pendiente para que lo revise un editor.

Código fuente real: cómo Laravel inicia Node.js

`ModerateComment.php` es el punto donde PHP inicia realmente el coordinador de moderación. El Job no ejecuta cinco procesos separados: llama una sola vez a `moderate-comment.mjs` y ese módulo importa los detectores que necesita durante la misma ejecución.

```
$process = Process::path(base_path())
->timeout(self::PROCESS_TIMEOUT_SECONDS)
->env([
    'SystemRoot' => getenv('SystemRoot'),
    'PATH' => getenv('PATH'),
])
->run([
    $nodeBinary,
    $scriptPath,
    (string) $comment->id,
    (string) $this->moderationRevision,
    (string) $comment->body,
]);
```

`Process::path(base_path())` establece la raíz de TRAMA como directorio de trabajo. `timeout()` limita la duración del proceso. `env()` transmite las variables necesarias al subprocesso y `run([...])` entrega cada argumento por separado. Conceptualmente, la ejecución equivale a `node scripts/comments/moderate-comment.mjs 85 3 "texto..."`.

El coordinador escribe una única respuesta JSON por `stdout`. Laravel usa `$process->output()`, la decodifica y valida que el comentario, `moderation_revision` y decisión correspondan al trabajo que se está procesando. Un código de salida distinto de cero se trata como fallo técnico y nunca como aprobación automática.

```
$result = json_decode(
    trim($process->output()),
    true,
    512,
    JSON_THROW_ON_ERROR,
);
```

Paso a paso: Candela envía un comentario normal y termina aprobado

Supongamos que Candela escribe “Muy buena nota, explica claramente el problema”. El recorrido esperado de ese comentario, si no aparece ninguna señal de riesgo, es el siguiente:

Paso	Qué ocurre
1	<code>CommentController</code> valida la noticia, la cuenta, el texto y las reglas de participación. Guarda la misma fila en <code>comments</code> como <code>processing</code> , con <code>moderation_revision=1</code> .
2	Laravel despacha <code>ModerateComment</code> a la cola <code>moderation</code> y responde al navegador sin esperar a los analizadores.
3	El worker toma el Job. <code>ModerateComment</code> usa <code>Process::run()</code> y el ejecutable configurado en <code>TRAMA_NODE_BINARY</code> para iniciar <code>moderate-comment.mjs</code> .

Paso	Qué ocurre
4	detect-comment-language.mjs reconoce el texto como apto para continuar y detect-comment-links.mjs no encuentra enlaces.
5	El coordinador carga después los detectores de amenazas, toxicidad y spam. Ninguno produce una causa de rechazo ni recomienda revisión manual.
6	moderate-comment.mjs construye un JSON con decision=approved y reason=automatic_clean y lo escribe por stdout.
7	ModerateComment valida elJSON, comprueba que moderation_revision siga siendo la misma y cambia la fila de processing a approved.
8	ArticleShow.vue, que consulta el estado mientras el comentario está processing, detecta el cambio y el comentario puede aparecer sin recargar toda la página.

Paso a paso: Candela envía un comentario con una amenaza clara

Supongamos que Candela escribe “Si seguís hablando de mí, te voy a golpear” y pulsa Enviar comentario. El objetivo de este ejemplo es seguir el recorrido interno de una amenaza física directa hasta que TRAMA decide que el contenido no debe publicarse.

1. Laravel recibe el comentario. CommentController comprueba las reglas normales de participación y, si son válidas, guarda la fila en comments con status=processing. El comentario ya existe en MySQL, pero todavía no es público. moderation_revision identifica qué versión del texto se está analizando.
2. La moderación pasa a la cola. Laravel despacha el Job ModerateComment a la cola moderation. La petición web puede terminar y el usuario ve que su comentario fue recibido y está siendo revisado.
3. El worker toma el trabajo. El proceso de colas ejecuta ModerateComment y desde allí se inicia scripts/comments/moderate-comment.mjs, que coordina los distintos controles de Node.js.
4. El análisis avanza por etapas. Primero se comprueba el idioma y después se buscan enlaces. Como el texto del ejemplo puede analizarse normalmente y no contiene enlaces, la cadena continúa hasta la comprobación de amenazas.
5. El detector de amenazas combina reglas de contexto con un modelo local. En este ejemplo la señal es suficientemente clara para rechazar el comentario. Como ya existe una decisión terminal, no es necesario cargar obligatoriamente los analizadores posteriores de toxicidad y spam.
6. El resultado vuelve a Laravel. Node.js entrega una respuesta estructurada y ModerateComment comprueba que moderation_revision siga correspondiendo al texto que fue analizado. Si el usuario hubiera generado una revisión más nueva, el resultado anterior se descartaría.
7. Laravel actualiza la misma fila de comments de processing a rejected. No se crea un segundo comentario: cambia el estado de la participación que ya estaba guardada.
8. La interfaz detecta el nuevo estado y deja de mostrar el aviso de revisión. El usuario recibe el mensaje general de rechazo; TRAMA no expone públicamente qué detector intervino ni la puntuación interna que produjo la decisión.

Ejemplos de resultados

Comentario	Resultado interno	Qué ve el usuario
“Muy buena nota, gracias por la información.”	approved / automatic_clean	El aviso de revisión desaparece y el comentario aparece.

Comentario	Resultado interno	Qué ve el usuario
“Mirá esto: https://ejemplo.com”	rejected / link_detected	“Tu comentario no pudo publicarse porque incumple las normas de participación.”
Texto completamente en un idioma no admitido	pending / unsupported_language	“Tu comentario está pendiente de moderación.”
Amenaza, toxicidad o spam claros	rejected	Se utiliza el mismo mensaje público de rechazo sin mostrar el detector ni su puntuación.

Puntuaciones de los modelos

Los modelos devuelven un número entre 0 y 1 que indica qué tan fuerte fue la señal detectada. En el código ese valor se llama score. En la configuración actual, una toxicidad clara puede producir rechazo automático desde 0,70 y el spam utiliza 0,78, siempre que el propio detector no haya recomendado revisión manual.

Qué es JSON, stdout y stderr

Cuando Node termina necesita comunicarle el resultado a Laravel. Devuelve un pequeño bloque de datos en JSON, un formato de texto utilizado para intercambiar información entre programas. La respuesta normal se escribe en stdout, que es la salida normal del proceso. Si ocurre un error, el detalle técnico se escribe en stderr. Laravel utiliza esa diferencia para saber si recibió una decisión válida o si debe reintentar el trabajo.

Cómo se evita aplicar un resultado viejo

Cada comentario tiene moderation_revision. Supongamos que Candela envía “Buen artículo” y comienza el análisis con revisión 1. Antes de que termine, edita el comentario pendiente y escribe “Buen artículo, muy completo”. TRAMA aumenta moderation_revision a 2 y crea un trabajo nuevo.

```
revisión 1 → Job empieza
Candela edita → moderation_revision = 2
Job de revisión 1 termina → resultado descartado
Job de revisión 2 → único resultado que puede aplicarse
```

Qué pasa si la automatización falla

ModerateComment puede intentarlo hasta 3 veces, con esperas de 5, 15 y 30 segundos. Si la cola ni siquiera puede recibir el trabajo o si el Job agota sus intentos, el comentario queda Pendiente para revisión manual. Un error técnico nunca se interpreta como una aprobación.

Actualización sin recargar la página

ArticleShow.vue consulta el estado cada 4 segundos mientras exista una participación propia en processing. Cuando el análisis termina, deja de consultar y actualiza la parte necesaria de la noticia. Por eso un comentario aprobado puede aparecer sin que el usuario recargue manualmente toda la página.

23. Revisión manual de comentarios y cuentas

La revisión manual empieza cuando la revisión automática no tiene una señal suficientemente clara para aprobar o rechazar un comentario. En ese caso TRAMA lo deja como Pendiente: el contenido no se muestra a los demás usuarios y pasa al módulo Comentarios del editor para que una persona tome la decisión.

Ejemplo completo: un comentario Pendiente llega al editor

Qué ve Mateo cuando la automatización deja el caso Pendiente

Detalle de un comentario Pendiente dentro del módulo de Moderación.

La captura ayuda a separar dos momentos: la automatización ya terminó y dejó una decisión pendiente; a partir de aquí el editor actúa sobre información que Laravel recupera de MySQL. Aprobar, rechazar, eliminar o derivar la cuenta son rutas distintas y vuelven a comprobar permisos en el servidor.

Formulario con el que Mateo deriva la cuenta al administrador junto con el comentario como evidencia.

Supongamos ahora que Candela escribe “Seguí hablando de mí y te voy a encontrar para hacerte pagar”. El detector encuentra una señal de intimidación preocupante, pero no una amenaza física directa suficientemente clara para rechazarla de forma automática. TRAMA cambia el estado de processing a pending y termina el trabajo automático para que el caso pase a una revisión manual.

Mateo abre el módulo Comentarios y entra al detalle. Allí puede ver que el estado es Pendiente, que el origen fue automático, quién escribió el comentario, en qué noticia aparece, el motivo de moderación y el texto completo que necesita evaluar.

En este punto la automatización ya no decide. El editor revisa el contexto y elige qué hacer con ese comentario.

Qué puede hacer el editor

Desde el detalle, Mateo puede Aprobar, Rechazar, Eliminar o Solicitar revisión de la cuenta. Son acciones distintas y cada una tiene un efecto concreto sobre el comentario o sobre la cuenta que lo publicó.

Acción	Qué ocurre
Aprobar	El comentario pasa a Aprobado y puede mostrarse públicamente.
Rechazar	Queda como Rechazado, deja de mostrarse y conserva el antecedente de moderación.
Eliminar	El comentario se elimina de forma definitiva desde el módulo de moderación.
Solicitar revisión	Envía el caso al administrador con el comentario como evidencia. Esta acción no bloquea la cuenta por sí sola.

Qué ocurre cuando un comentario publicado es reportado

Un reporte parte de una situación diferente. El comentario ya estaba Aprobado y visible cuando otro usuario lo reportó. Reportarlo no lo retira automáticamente del portal: crea un caso para que el editor revise el contenido y el motivo de la denuncia.

Mateo abre el detalle y puede consultar cuántos reportes existen y qué motivo eligió cada persona. Si decide Mantener, el comentario continúa publicado. Si decide Retirar, deja de mostrarse en el portal.

Rechazar no es eliminar

Rechazar cambia el estado del comentario y deja el caso disponible como antecedente dentro de Moderación. Eliminar es una acción definitiva. Por eso TRAMA presenta ambas opciones por separado.

Respuestas del equipo

Un periodista puede responder comentarios dentro de una noticia de su autoría y la respuesta aparece identificada como Autor de la nota. El editor puede responder como Equipo TRAMA. Estas respuestas internas se crean directamente como Aprobadas y no pasan por los filtros automáticos destinados a los usuarios registrados. El administrador no participa en la conversación pública.

Del comentario a la revisión de una cuenta

Solicitar revisión no bloquea automáticamente la cuenta. Si Mateo considera que, además de resolver el comentario, la conducta del usuario merece una evaluación administrativa, puede enviar el caso a Marina. TRAMA guarda el motivo, la observación, el comentario utilizado como evidencia, el título de la noticia y quién solicitó la revisión. Después es la administradora quien decide si cierra el caso sin aplicar una medida o si bloquea la cuenta.

users mantiene un resumen del estado administrativo actual de la cuenta y user_moderation_reviews guarda las revisiones realizadas mientras esa cuenta existe. Si el usuario registrado se elimina, esas filas se eliminan junto con la cuenta por la regla definida en MySQL.

Ejemplo

Mateo rechaza un comentario, considera que la conducta de la cuenta necesita revisión y envía el caso a Marina. Marina puede bloquear la cuenta o cerrar la revisión sin aplicar una medida.

De dónde sale el código: del modal de Mateo al Controller

El bloque PHP que aparece después no se ejecuta de forma aislada. Primero Mateo selecciona una fila en Admin/Comments/Index.vue. El modal resources/js/Components/Admin/CommentModerationModal.vue carga el detalle bajo demanda con una petición Axios a admin.comments.show:

```
const response = await window.axios.get(
  route('admin.comments.show', props.commentId),
);

comment.value = response.data?.comment || null;
```

La ruta GET /admin/comments/{comment} llega a CommentController::show(). El controlador vuelve a comprobar canModerateComments(), impide abrir por URL directa intervenciones internas del equipo, carga noticia, autor, contexto del padre y reportes abiertos, y transforma el resultado con AdminCommentDetailResource. Recién entonces el modal tiene el contenido completo que Mateo necesita para decidir.

Cuando Mateo pulsa Aprobar, Rechazar/Retirar o Mantener, el modal no modifica el objeto local como si el cambio ya fuera válido. Envía la acción a Laravel mediante Inertia router.patch():

```
function moderate(action) {
  router.patch(
    route('admin.comments.update', comment.value.id),
    { action },
    {
      preserveState: true,
      preserveScroll: true,
      onSuccess: () => closeModalAfterSuccess(),
    },
  );
}
```

routes/web.php conecta esa petición con CommentController::update(). Antes de entrar en la lógica de negocio, CommentModerationRequest normaliza action y sólo admite approve, reject o maintain; una petición manipulada con otro valor se rechaza antes de aplicar cambios.

```
Route::get('comments/{comment}', [AdminCommentController::class, 'show'])
    ->name('comments.show');

Route::patch('comments/{comment}', [AdminCommentController::class, 'update'])
    ->name('comments.update');

// CommentModerationRequest
'action' => [
    'required',
    'string',
    Rule::in(['approve', 'reject', 'maintain']),
],
```

A partir de ahí sí entra el código del Controller mostrado a continuación. DB::transaction() y lockForUpdate() garantizan que la decisión se tome sobre el estado real de la fila y que dos editores no puedan confirmar acciones incompatibles al mismo tiempo.

Código fuente real: bloqueo antes de una decisión editorial

CommentController::update() procesa la decisión dentro de una transacción y vuelve a leer el comentario con lockForUpdate() antes de modificarlo. El fragmento siguiente muestra ese punto de concurrencia: dos editores no pueden confirmar decisiones incompatibles sobre la misma fila al mismo tiempo.

```
DB::transaction(
    function () use (
        $comment,
        $action,
        $editorId,
    ): void {
        $lockedComment = Comment::query()
            ->whereKey($comment->id)
            ->lockForUpdate()
            ->firstOrFail();
```

Una vez bloqueada la fila, el mismo método aplica approve, reject o maintain según el estado actual. La tabla siguiente resume el efecto de cada acción, incluido el retiro de respuestas y la resolución de reportes cuando corresponde.

Qué cambia internamente según la decisión

Acción visible	Petición / método	Resultado interno principal
Aprobar	PATCH admin.comments.update · action='approve'	Exige status=pending. Guarda status=approved, moderation_source=editorial y moderation_reason=editorial_approved.
Rechazar / Retirar	PATCH admin.comments.update · action='reject'	Guarda status=rejected. Si era un comentario principal, retira respuestas activas y resuelve reportes abiertos.
Mantener	PATCH admin.comments.update · action='maintain'	Sólo para un comentario approved con reportes abiertos. Conserva la publicación y marca esos reportes como revisados/descartados.
Eliminar	DELETE admin.comments.destroy	Abre una confirmación aparte. El Controller bloquea la fila y la elimina definitivamente; si el padre posee respuestas, la relación de base aplica la consecuencia definida para ellas.
Solicitar revisión de cuenta	PATCH admin.comments.refer-to-admin	Guarda una revisión administrativa con motivo, observación y comentario como evidencia. No cambia is_active y por sí sola no bloquea al lector.

Eliminar y derivar la cuenta usan rutas distintas porque no son estados de moderación equivalentes a approve/reject/maintain. El modal llama router.delete() para destruir el comentario y otra petición PATCH para refer-to-admin con reason y note.

Paso a paso: Mateo recibe un comentario pendiente y decide qué hacer

Pendiente significa que la automatización terminó y dejó la decisión en manos del editor.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela envía un comentario y el pipeline automático termina con status=pending. La fila ya existe en comments, pero el contenido no es público.
2	Mateo abre Admin/Comments/Index.vue y filtra Pendientes. Laravel ya había construido la grilla paginada; esa fila contiene sólo el resumen necesario para el listado.

Paso	Qué hace la persona y qué ocurre internamente
3	Mateo abre el caso. CommentModerationModal.vue ejecuta loadCommentDetail() y hace GET a admin.comments.show con el id del comentario.
4	CommentController::show() comprueba permisos, carga noticia, autor, comentario padre, cantidad de respuestas y reportes abiertos y devuelve AdminCommentDetailResource como JSON.
5	El modal muestra el texto completo y el contexto. Mateo decide Aprobar y moderate("approve") envía PATCH a admin.comments.update con { action: "approve" }.
6	CommentModerationRequest normaliza la acción y comprueba que pertenezca a la lista cerrada approve/reject/maintain. Si no es válida, el Controller no aplica la decisión.
7	CommentController::update() vuelve a comprobar canModerateComments(), abre DB::transaction() y relee la fila con lockForUpdate(). Así decide sobre el estado actual y no sobre una copia vieja.
8	Como el comentario sigue pending, Laravel lo actualiza a approved y registra moderation_source=editorial y moderation_reason=editorial_approved. Para otras acciones ejecuta sus reglas específicas.
9	El Controller devuelve back()->with("status", ...). Inertia actualiza la grilla y los conteos manteniendo filtros y scroll; el modal se cierra después de la operación exitosa.
10	Si Mateo considera que también debe evaluarse la conducta de Candela, abre Solicitar revisión. Esa segunda ruta registra un caso para Marina con el comentario como evidencia, pero la cuenta sigue activa hasta una decisión administrativa.

24. Bloqueo y eliminación de usuarios y empleados

Bloquear no es eliminar

Ejemplo práctico: Marina bloquea una cuenta con antecedentes

SEGURIDAD

Bloquear usuario

Brenda Escobar · brendaescobar@nube.test

La cuenta dejará de poder participar.
No podrá iniciar sesión, comentar, responder, dar Me gusta ni reportar mientras permanezca bloqueada.
El contenido histórico aprobado no será eliminado ni mostrará públicamente que la cuenta está bloqueada.

Contexto de moderación
Mateo Ledesma solicitó revisar esta cuenta el 19 jul 2026, 2:02 p. m..
Motivo: **Insultos o acoso**
Observación: Moderación solicita revisar el contexto de la cuenta y sus antecedentes antes de decidir una medida.

Comentario usado como evidencia
Entiendo tu punto sobre la personalización de cargas de entrenamiento. Yo sumaría la parte de la combinación de video y datos diarios, porque en esta nota las dos cosas aparecen bastante conectadas.
Noticia: Preparadores físicos ajustan cargas con datos en tiempo real

Motivo

Cancelar **Bloquear cuenta**

Ventana de bloqueo: la administradora ve contexto de moderación antes de decidir.

El formulario de bloqueo utiliza un useForm separado llamado blockForm. Contiene motivo y observación administrativa; blockForm.processing controla el envío y blockForm.errors mantiene los errores de esa decisión sin mezclarlos con el formulario de edición de usuarios.

Del formulario de bloqueo al backend

Para un lector activo, Admin/Users/Index.vue abre una ventana específica y utiliza blockForm, creado con useForm(), para mantener blocked_reason y blocked_note. El botón no cambia is_active localmente: blockReader() envía una petición PATCH a la ruta administrativa real.

```
function blockReader() {
  blockForm.patch(
    route('admin.users.toggle-active', blockTarget.value.id),
    {
      preserveScroll: true,
      onSuccess: () => closeBlockAfterSuccess(),
    },
  );
}
```

routes/web.php conecta PATCH /admin/users/{user}/toggle-active con UserController::toggleActive(). Antes, ToggleUserStatusRequest autoriza sólo administradores. Cuando el objetivo es un reader activo exige un motivo admitido y una observación interna de 10 a 200 caracteres; para empleados esos campos no forman parte del estado administrativo.

```
Route::patch(
    'users/{user}/toggle-active',
    [AdminController::class, 'toggleActive']
)->name('users.toggle-active');

// ToggleUserStatusRequest
'blocked_reason' => [
    Rule::requiredIf($blockingReader),
    'nullable',
    Rule::in(['moderation', 'abuse', 'spam', 'security', 'terms', 'other']),
],
'blocked_note' => [
    Rule::requiredIf($blockingReader),
    'nullable', 'string', 'min:10', 'max:200',
],
```

Código fuente real: qué cambia UserController::toggleActive()

El Controller protege además reglas que la interfaz no puede garantizar por sí sola: Marina no puede desactivar su propia cuenta y TRAMA no permite bloquear al último administrador activo. Dentro de una transacción vuelve a buscar el usuario con lockForUpdate() y calcula el estado final.

```
$locked = User::query()
    ->whereKey($user->id)
    ->lockForUpdate()
    ->firstOrFail();

$willBeActive = ! $locked->is_active;

$statusData = [
    'is_active' => $willBeActive,
    'disabled_at' => $willBeActive ? null : TramaClock::now(),
    'remember_token' => $willBeActive ? $locked->remember_token : null,
    'updated_at' => TramaClock::now(),
];

if ($locked->role === 'reader') {
    $statusData['blocked_reason'] = $willBeActive
        ? null : $request->validated('blocked_reason');
    $statusData['blocked_note'] = $willBeActive
        ? null : $request->validated('blocked_note');
}

$locked->forceFill($statusData)->save();
```

Cuando el bloqueo responde a una revisión administrativa pendiente, TRAMA cierra también esa revisión para que el caso y el estado de la cuenta no queden contradictorios. Reactivar invierte is_active y limpia disabled_at; en lectores también limpia blocked_reason y blocked_note del estado actual, sin borrar el historial guardado en user_moderation_reviews.

Qué ocurre con una sesión que ya estaba abierta

```

Marina guarda el bloqueo
↓
users.is_active = false
↓
Candela conserva una pestaña abierta
↓
Candela realiza la siguiente petición protegida
↓
EnsureActiveAccount vuelve a consultar el estado
↓
detecta is_active = false
↓
invalida esa sesión concreta
↓
Candela vuelve al login con el mensaje de cuenta bloqueada

```

Por diseño, `toggleActive()` no necesita borrar inmediatamente todas las filas de `sessions`. `is_active` hace efectivo el bloqueo en el servidor desde el momento de la transacción, y `EnsureActiveAccount` expulsa cada sesión abierta cuando vuelve a usar la aplicación. Esto también cubre sesiones abiertas en otros dispositivos.

Bloquear una cuenta no borra sus datos. La fila continúa en `users`, pero `is_active` pasa a `false` y la cuenta deja de poder iniciar sesión. Si ya tenía una sesión abierta, `EnsureActiveAccount` la invalida en la siguiente petición.

En un usuario registrado, el administrador también guarda un motivo y una observación interna. Reactivar la cuenta vuelve a permitir el acceso, pero no borra las decisiones administrativas que ya quedaron registradas.

Empleados con historial

Un periodista, editor o administrador puede haber participado en información que TRAMA necesita conservar, como noticias, revisiones, comentarios internos o devoluciones editoriales. En esos casos la cuenta interna no se elimina: se desactiva para mantener correctamente las referencias históricas.

Alta, edición y recuperación de acceso de empleados

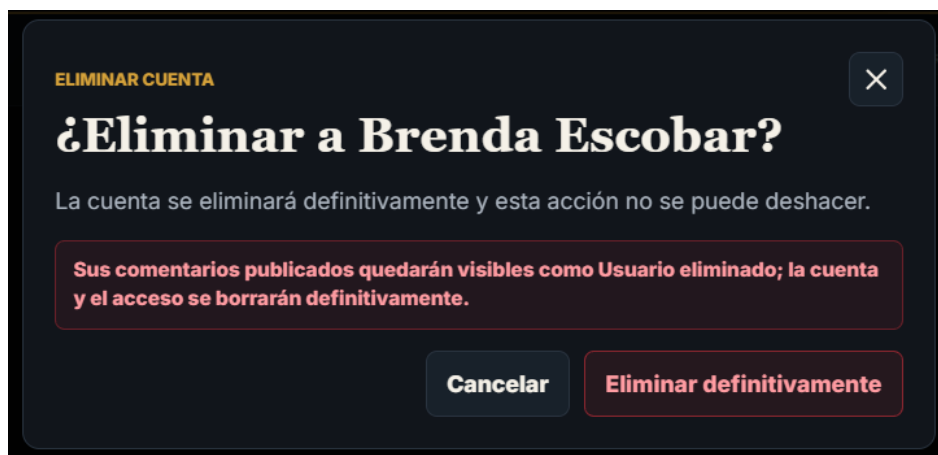
Desde Empleados, el administrador puede crear cuentas internas de administrador, editor o periodista. `StoreEmployeeRequest` valida nombre, identificador de correo, rol, cargo, biografía cuando corresponde, contraseña inicial y foto opcional. El correo se construye con el dominio interno `@trama.test` y, al crear la cuenta, `UserController` registra `email_verified_at` con la hora editorial de `TramaClock`. Si se carga una foto, se procesa como WebP de 600 × 600 antes de guardar su ruta.

La edición permite actualizar identidad, rol, cargo, biografía y avatar, pero mantiene protecciones administrativas: un administrador no puede quitarse a sí mismo su propio rol y TRAMA no permite dejar el sistema sin otro administrador activo. Para una cuenta activa también existe Restablecer acceso: `UserController::resetAccess()` crea un token de recuperación, marca `password_reset_required_at`, elimina sus sesiones abiertas y recién después envía el enlace por correo.

Ejemplo: alta y restablecimiento de un periodista

Marina crea a Sofía como periodista, define su identificador de correo, cargo, biografía, contraseña inicial y foto. Laravel crea la cuenta interna y la deja lista para iniciar sesión. Más adelante, si Sofía necesita recuperar el acceso, Marina utiliza Restablecer acceso. TRAMA invalida las sesiones anteriores y envía un enlace; hasta completar ese flujo, la marca `password_reset_required_at` permite exigir el cambio de contraseña antes de volver a usar normalmente el CMS.

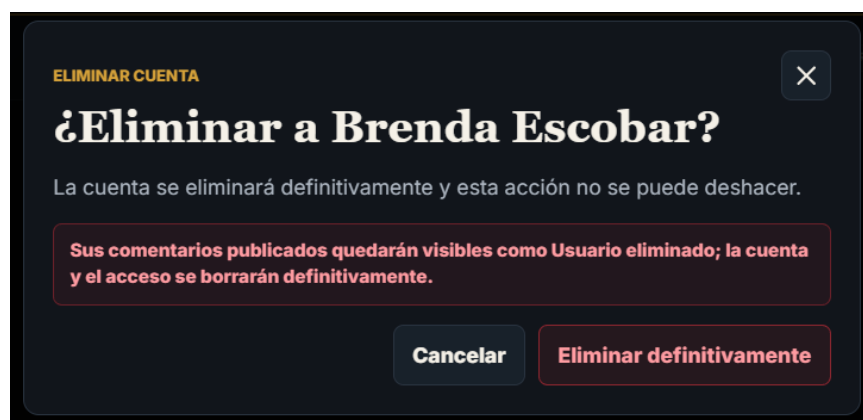
Ejemplo: eliminar un usuario registrado con comentarios



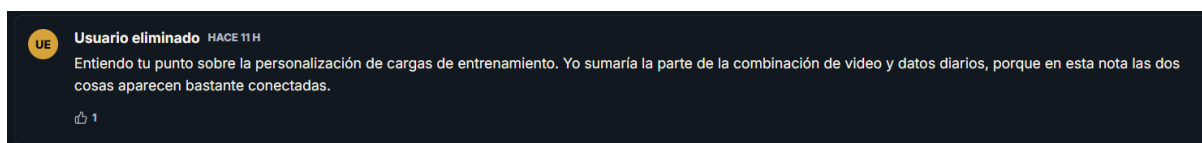
Confirmación de eliminación definitiva de una cuenta registrada.

El resultado visible corresponde a la política explicada en esta sección: la cuenta desaparece, pero los comentarios públicos que deben conservar la conversación permanecen y se anonimizan como “Usuario eliminado”.

Los comentarios públicos no impiden por sí solos eliminar una cuenta registrada. Antes de borrar users, UserController desvincula esos comentarios y elimina los datos identificatorios que pertenecían a la cuenta.



Confirmación que explica qué ocurrirá antes de eliminar la cuenta.



Después de eliminar la cuenta, el comentario se conserva con el autor “Usuario eliminado”.

Antes	Después
comments.user_id = 125	comments.user_id = NULL
comments.author_name = "Brenda Escobar"	comments.author_name = "Usuario eliminado"
comments.author_email = "..."	comments.author_email = NULL
Sesiones de la cuenta	Se eliminan.
Me gusta de esa cuenta	Se eliminan.
Reportes hechos por esa cuenta	Se eliminan.

Antes	Después
user_moderation_reviews de esa cuenta	Se eliminan.

Una cuenta registrada no puede eliminarse si figura como autora de noticias, responsable de revisiones editoriales o responsable de devoluciones. Esas relaciones forman parte del historial editorial y la base impide romperlas.

Tampoco se permite que un administrador elimine su propia cuenta desde esta acción. Si se intenta eliminar un administrador activo, el sistema comprueba que exista al menos otro administrador activo.

Código fuente real: qué se comprueba antes de eliminar una cuenta

La ruta DELETE /admin/users/{user} llega a `UserController::destroy()`. Eliminar es más restrictivo que bloquear: antes de borrar la fila vuelve a bloquearla con `lockForUpdate()` y comprueba si existe contenido histórico que deba conservar referencias hacia esa persona.

```
if ($request->user()->is($user)) {
    throw ValidationException::withMessages([
        'user' => 'No podés eliminar tu propia cuenta.',
    ]);
}

$locked = User::query()
    ->whereKey($user->id)
    ->lockForUpdate()
    ->firstOrFail();

if ($this->deletionBlockedByHistoricalContent($locked)) {
    throw ValidationException::withMessages([
        'user' => 'La cuenta conserva contenido histórico y debe desactivarse en lugar de eliminarse.',
    ]);
}
```

Si la cuenta puede eliminarse, `destroy()` borra sus sesiones, anonimiza primero los comentarios de un reader y finalmente elimina users. En empleados con autoría, revisiones o devoluciones, la regla anterior obliga a utilizar bloqueo/desactivación para conservar el historial editorial.

Código fuente real: anonimizar comentarios antes de eliminar un lector

Cuando la cuenta es un usuario registrado y no existe historial editorial que impida eliminarla, `UserController` borra sus sesiones y llama a `detachReaderCommentsBeforeDelete()` antes de eliminar users. Los comentarios permanecen, pero dejan de identificar a la persona:

```

DB::table('sessions')
    ->where('user_id', $locked->id)
    ->delete();

$this->detachReaderCommentsBeforeDelete($locked);
$locked->delete();

private function detachReaderCommentsBeforeDelete(User $user): void
{
    if ($user->role !== 'reader') {
        return;
    }

    Comment::query()
        ->where('user_id', $user->id)
        ->update([
            'user_id' => null,
            'author_name' => 'Usuario eliminado',
            'author_email' => null,
            'updated_at' => TramaClock::now(),
        ]);
}

```

Esta operación explica el “antes/después” mostrado anteriormente: el comentario sigue formando parte de la conversación, pero `user_id` y `author_email` dejan de identificar a la cuenta eliminada y `author_name` pasa a “Usuario eliminado”. Las cuentas internas con relaciones históricas se desactivan en lugar de seguir este borrado.

Paso a paso: Marina bloquea una cuenta y el bloqueo se aplica en la siguiente petición

Bloquear conserva la cuenta y su historial; eliminarla es una decisión distinta y definitiva.

Paso	Qué hace la persona y qué ocurre internamente
1	Marina abre Usuarios registrados en Admin/Users/Index.vue. La fila ya incluye estado, antecedentes y, si existe, la revisión derivada desde Moderación.
2	Marina pulsa Bloquear. Como Candela es reader y está activa, Vue abre la ventana blockForm en lugar de ejecutar el cambio inmediatamente.
3	Marina selecciona el motivo y escribe la observación. <code>blockReader()</code> envía PATCH a <code>admin.users.toggle-active</code> con <code>blocked_reason</code> y <code>blocked_note</code> .
4	<code>ToggleUserStatusRequest</code> comprueba que quien actúa sea administrador, normaliza la nota y exige los campos administrativos porque se está bloqueando a un lector activo.
5	<code>UserController::toggleActive()</code> impide el autobloqueo, abre una transacción, relee users con <code>lockForUpdate()</code> y comprueba protecciones como la del último administrador activo cuando corresponde.
6	Laravel guarda <code>is_active=false</code> , <code>disabled_at=TramaClock::now()</code> , <code>remember_token=null</code> y, para el lector, <code>blocked_reason</code> y <code>blocked_note</code> . Si había revisión pendiente, la resuelve como blocked.
7	La respuesta vuelve a Admin/Users/Index.vue y el panel muestra la cuenta como bloqueada. La fila users y su historial continúan existiendo.
8	Si Candela tenía una sesión abierta, su siguiente petición pasa por <code>EnsureActiveAccount</code> . El middleware detecta <code>is_active=false</code> , invalida esa sesión y evita que continúe.

Paso	Qué hace la persona y qué ocurre internamente
9	Si Marina reactiva la cuenta, el mismo endpoint invierte el estado y vuelve a permitir el acceso. Si decide eliminarla, se usa DELETE admin.users.destroy y se aplican las reglas históricas y de anonimización explicadas aparte.

25. Contacto y páginas informativas

TRAMA incluye las páginas públicas Quiénes somos, Contacto, Privacidad y Términos. InstitutionalController entrega estas pantallas mediante TramaBridge. Contacto agrega un flujo real de escritura en MySQL y envío de correo, mientras que las demás páginas presentan información general y legal del portal.

Ruta	Función
/quienes-somos	Presenta el proyecto y su enfoque editorial.
/contacto	Muestra el formulario y recibe consultas públicas. Si hay sesión iniciada, nombre y correo se completan como valores iniciales.
/privacidad	Presenta el tratamiento de datos del portal.
/terminos	Presenta las condiciones generales de uso.

Validación del formulario de Contacto

Campo	Regla	Tratamiento
Motivo	account, editorial, advertising u other.	Debe coincidir con una opción admitida por PublicContactRequest.
Nombre	2 a 40 caracteres.	Se recortan espacios al inicio y al final.
Correo	Formato válido, máximo 50 y regla StrictEmail.	Puede pertenecer a un invitado o a una cuenta autenticada.
Mensaje	15 a 1200 caracteres.	Normaliza saltos de línea y reduce tres o más saltos consecutivos a un máximo de una línea vacía entre párrafos.

Por qué TRAMA guarda primero y envía el correo después

InstitutionalController::storeContact() crea primero contact_messages. Esa fila es la fuente de verdad de la consulta. Sólo después Laravel intenta enviar ContactMessageReceivedMail a config('mail.contact_address'), valor alimentado por la configuración de correo y TRAMA_CONTACT_EMAIL. El mensaje interno utiliza Reply-To con el nombre y correo de quien escribió para que el equipo pueda responder desde su cliente de correo.

Situación	Resultado
MySQL guarda y SMTP envía	La consulta queda registrada y el equipo recibe la notificación por correo.

Situación	Resultado
MySQL guarda pero SMTP falla	La consulta permanece en contact_messages; el fallo de correo se registra en TramaLog y el usuario no necesita reenviar.
Falla el guardado en MySQL	OperationFailureHandler devuelve un mensaje seguro y la consulta no se considera recibida.

Ejemplo completo

Qué ocurre en Contact.vue antes de llegar a InstitutionalController

Formulario público de Contacto.

Contact.vue usa useForm para reason, name, email y message. Cuando existe una sesión, Laravel puede enviar defaultName y defaultEmail como props. computed() calcula si el formulario está bloqueado, la longitud útil del mensaje y si el botón puede habilitarse. Después del envío, usePage() detecta el mensaje flash del servidor. Un mensaje flash es un dato temporal guardado para la siguiente navegación, útil para mostrar confirmaciones como “Consulta enviada correctamente”. Luego deja todo el formulario deshabilitado para evitar una segunda consulta accidental.

```
const page = usePage();
const form = useForm({
  reason: 'account',
  name: props.defaultName,
  email: props.defaultEmail,
  message: '',
});

const submitted = computed(() => Boolean(page.props.flash?.status));
const formLocked = computed(() => form.processing || submitted.value);
```

Candela selecciona “Problemas con mi cuenta”, escribe su nombre, correo y consulta y pulsa Enviar. PublicContactRequest normaliza y valida los campos. Laravel crea la fila en contact_messages. A continuación intenta mandar el correo interno con asunto relacionado con el motivo y Reply-To a Candela. Si el SMTP no responde, la fila ya guardada permite que el equipo recupere la consulta desde la base y el error técnico quede diagnosticado en los logs.

Este flujo también es un ejemplo concreto del manejo de errores de la sección 29: si falla la escritura en MySQL, InstitutionalController utiliza OperationFailureHandler y la consulta no se considera recibida; si MySQL ya guardó el mensaje pero falla SMTP, TramaLog registra el problema y la consulta permanece disponible para el equipo.

Paso a paso: Candela envía una consulta desde Contacto

El ejemplo permite ver cómo una operación puede guardar datos en MySQL y después intentar una acción externa como el correo.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela abre Contacto, elige un motivo y completa nombre, email y mensaje. Si ya inició sesión, parte de esos datos puede venir precargada.
2	Contact.vue mantiene el formulario y muestra validaciones inmediatas de interfaz cuando corresponde.
3	Al enviar, Laravel vuelve a validar todos los campos con las reglas del servidor.
4	InstitutionalController guarda primero la consulta en la base de datos para que el mensaje no se pierda aunque el envío de correo falle después.
5	Una vez persistida la consulta, TRAMA intenta notificar por el mailer configurado.
6	Si el correo se envía, la pantalla muestra la confirmación normal. Si falla el servicio de correo, el sistema puede registrar el problema sin deshacer necesariamente la consulta ya guardada.
7	La respuesta vuelve a Vue y Candela recibe un mensaje útil sin ver excepciones, rutas internas ni detalles del servidor.

26. Publicidad: banners, impresiones, clics y reporte

El módulo Publicidades permite crear campañas y medir cuántas veces se mostraron y cuántos clics recibieron. Cada campaña guarda nombre, marca, ubicación, imagen, dirección de destino, fecha de inicio, fecha de finalización y estado.

Ubicación visible	Nombre interno	Tamaño exigido
Banner horizontal superior	header	1456 × 180 px
Lateral superior de una noticia	article_sidebar_top	600 × 500 px
Lateral inferior de una noticia	article_sidebar_bottom	600 × 1200 px

Esas dimensiones corresponden al archivo que TRAMA valida y almacena. Como referencia de presentación, el CSS utiliza una proporción de 728 × 90 para el banner del encabezado, 300 × 250 para el lateral superior y 300 × 600 para el lateral inferior; el ancho puede adaptarse de forma responsiva según la pantalla. El archivo se conserva con el doble de esas dimensiones para mantener nitidez.

Las imágenes de publicidad deben tener exactamente el tamaño requerido por su ubicación. Pasan por el mismo control de seguridad general de imágenes y se convierten a WebP.

Dónde aparece cada publicidad en el portal

placement no es sólo un nombre guardado en la base: determina en qué componente del portal puede aparecer la campaña. El header pertenece al layout público compartido por las páginas de noticias. Los dos espacios laterales se usan dentro del detalle de una noticia, en una columna aside separada del cuerpo principal.

```

PublicNewsLayout.vue
└─ <AdvertisementBanner
    :advertisement="visibleHeaderAdvertisement"
    variant="header"
  />

Public/ArticleShow.vue
└─ <aside class="article-ad-rail">
    <AdvertisementBanner
      :advertisement="sidebarTopAdvertisement"
      variant="sidebarTop"
    />
    <AdvertisementBanner
      :advertisement="sidebarBottomAdvertisement"
      variant="sidebarBottom"
    />
  </aside>

```

PublicNewsLayout.vue recibe la colección header y rota la campaña visible cada 6500 ms cuando existe más de una. ArticleShow.vue recibe sidebar_top y sidebar_bottom para la noticia cargada. En ambos casos el componente común AdvertisementBanner.vue aplica la variante visual correspondiente y se ocupa de impresión y clic.

Qué significa una impresión

Una impresión no se cuenta sólo porque Laravel haya enviado los datos del banner al navegador. La imagen debe cargar correctamente y al menos el 50 % del banner debe entrar en la parte visible de la página.

Para detectar esa visibilidad, Vue utiliza una función disponible en navegadores modernos llamada IntersectionObserver. Si el navegador no dispone de esa función, TRAMA utiliza una alternativa. Si la imagen está rota, la impresión no se registra.

Qué significa un clic

Cuando el visitante selecciona un banner, el navegador comienza a ir al sitio de destino. Al mismo tiempo TRAMA intenta registrar el clic. JavaScript utiliza keepalive para dar a esa petición la posibilidad de terminar aunque la navegación ya haya comenzado.

Paso a paso: Candela hace clic en un banner

El clic publicitario está diseñado para que la medición nunca se convierta en una barrera entre Candela y el anunciante. El enlace conserva como href el target_url real y el tracking se envía en paralelo.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela ve un AdvertisementBanner cuya imagen ya cargó. El enlace del componente apunta directamente a advertisement.target_url.
2	Candela hace clic. El evento @click ejecuta registerClick(), pero el <a> conserva su navegación normal y abre el destino en una pestaña nueva.
3	registerClick() toma advertisement.click_url y ejecuta window.fetch() con method=GET, cache=no-store y keepalive=true.
4	La ruta /ads/{advertisement}/click llega a AdvertisementClickController. No redirige al anunciante porque esa navegación ya la está realizando el navegador.
5	El Controller usa TramaClock::today(), busca o crea la fila de advertisement_daily_metrics para publicidad + fecha editorial y aumenta clicks_count en una unidad.

Paso	Qué hace la persona y qué ocurre internamente
6	Laravel responde 204 No Content. Aunque el tracking falle, el catch de JavaScript no reemplaza ni cancela la apertura de target_url.
7	Cuando Marina vuelve al reporte administrativo, esa fila diaria participa de los totales, el CTR y la serie de clics del gráfico.

CTR

CTR significa Click Through Rate y representa qué porcentaje de las impresiones terminó en un clic.

```
Impresiones: 1.000
Clics: 10

CTR = 10 / 1.000 × 100
CTR = 1 %
```

advertisement_daily_metrics guarda impresiones y clics agrupados por publicidad y fecha. El panel utiliza esos datos para mostrar totales, CTR, campañas activas y una serie diaria con Chart.js.

Cómo se construye el reporte y qué gráfico utiliza TRAMA

El reporte no dibuja puntos a partir de las filas visibles de la grilla. AdvertisementController aplica primero los mismos filtros y período, obtiene los ids de las campañas incluidas y suma advertisement_daily_metrics directamente en MySQL. Para la serie temporal agrupa por date y calcula las impresiones y clics de cada día.

```
$dailyMetrics = AdvertisementDailyMetric::query()
    ->whereIn('advertisement_id', $advertisementIds)
    ->selectRaw('date')
    ->selectRaw('SUM(impressions_count) as impressions_total')
    ->selectRaw('SUM(clicks_count) as clicks_total')
    ->groupBy('date')
    ->orderBy('date')
    ->get();

$chart = [
    'labels' => $dailyMetrics
        ->map(fn ($metric) => $metric->date->format('d/m')),
    'impressions' => $dailyMetrics->pluck('impressions_total'),
    'clicks' => $dailyMetrics->pluck('clicks_total'),
];
```

Ese array viaja como prop chart a Admin/Advertisements/Index.vue. TRAMA no utiliza un plugin de Laravel para dibujarlo: usa la librería JavaScript Chart.js como motor y vue-chartjs como integración con Vue. El componente concreto importado es Line, por lo que el reporte es un gráfico de líneas.

```
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  LineElement,
  PointElement,
  Tooltip,
  Legend,
} from 'chart.js';
import { Line } from 'vue-chartjs';

ChartJS.register(
  CategoryScale,
  LinearScale,
  LineElement,
  PointElement,
  Tooltip,
  Legend,
);
```

buildChartData() crea dos datasets sobre las mismas fechas. Impresiones utiliza el eje yImpressions y Clicks el eje yClicks. Las escalas son independientes porque las impresiones suelen ser mucho mayores que los clics; con una sola escala, una serie pequeña podría quedar visualmente aplastada cerca de cero.

```
function buildChartData(chart) {
  return {
    labels: chart?.labels || [],
    datasets: [
      {
        label: 'Impresiones',
        data: chart?.impressions || [],
        yAxisID: 'yImpressions',
      },
      {
        label: 'Clicks',
        data: chart?.clicks || [],
        yAxisID: 'yClicks',
      },
    ],
  };
}

<Line :data="chartData" :options="chartOptions" />
```

chartOptions coloca yImpressions a la izquierda y yClicks a la derecha. El eje horizontal representa los días devueltos por labels. El mismo panel muestra además impresiones totales, clics totales, CTR y cantidad de campañas activas, todos calculados para el período y filtros seleccionados.

Paso a paso: Marina abre Publicidades y llega al gráfico

Paso	Qué hace la persona y qué ocurre internamente
1	Marina abre Publicidades y elige, por ejemplo, Últimos 30 días. AdvertisementIndexRequest valida los filtros de la URL.
2	AdvertisementController construye filteredAdvertisementQuery() y obtiene los ids de las campañas que cumplen búsqueda, ubicación, marca, estado y demás filtros.
3	Sobre advertisement_daily_metrics suma impresiones y clics del período para las tarjetas generales y calcula $CTR = clics / impresiones \times 100$.

Paso	Qué hace la persona y qué ocurre internamente
4	Para el gráfico vuelve a consultar las métricas filtradas, agrupa por date y suma impressions_count y clicks_count de todas las campañas incluidas.
5	Laravel prepara chart.labels, chart.impressions y chart.clicks y los envía a Admin/Advertisements/Index.vue mediante Inertia.
6	buildChartData() transforma esos tres arrays en dos datasets de Chart.js y los asocia a yImpressions y yClicks.
7	vue-chartjs renderiza <Line>. Marina ve la evolución diaria y puede comparar tendencia de impresiones y clics sin confundir sus escalas.

Cómo decide TRAMA qué publicidad mostrar

AdvertisementSelectorService filtra primero por ubicación, is_active y vigencia exacta frente a TramaClock: starts_at debe ser menor o igual al momento editorial y ends_at mayor o igual. Después comprueba que el WebP registrado en image_path exista realmente dentro de la carpeta configurada. Una campaña activa en MySQL cuyo archivo fue borrado a mano deja de participar del portal hasta que vuelva a tener una imagen válida.

El header funciona distinto de las barras laterales. Para header Laravel comparte todas las campañas vigentes y PublicNewsLayout.vue conserva la actual y rota entre ellas cada 6500 ms. Para article_sidebar_top y article_sidebar_bottom se selecciona una campaña válida de forma aleatoria durante la carga de la noticia. Seleccionar o enviar una campaña a Vue todavía no cuenta una impresión.

Ejemplo de una campaña de punta a punta

Paso a paso: Marina edita una campaña publicitaria

PUBLICIDAD

Editar publicidad

Actualizá la campaña sin perder sus métricas históricas.

Marca
HORIZONTE

Nombre de campaña
HORIZONTE - Header

Ubicación
Header - 1456 x 180 px

Destino del anuncio
/demo/anunciantes/universidad-horizonte

Header
horizonte-horizonte-header-1456x180.webp
1456 x 180 px

Cambiar imagen
Opcional - 1456 x 180 px exactos - máximo 4 MB
Seleccionar archivo Ningún archivo seleccionado

Inicio
01/07/2026 00:00

Finalización
31/07/2026 23:59

Campaña habilitada
Desactivarla la deja en estado PAUSADA sin borrar el historial.

Cancelar Guardar cambios

Ventana real de edición de una publicidad.

Admin/Advertisements/Index.vue mantiene createForm y editForm por separado. De esta forma cada modal conserva sus propios valores, errores y estado processing. La interfaz puede anticipar errores de URL o dimensiones, pero Laravel vuelve a validar la campaña y la imagen según la ubicación elegida.

The screenshot shows a dark-themed form with the title "Destino del anuncio". Below the title is a text input field containing the value "1111". Below the input field, there is a red error message that reads "Ingresá una dirección web válida.".

Ejemplo visual de una dirección inválida antes de completar la edición.

Paso	Qué ocurre
1	Marina crea un banner header con imagen válida, URL de destino, starts_at=19/07/2026 15:00, ends_at=19/07/2026 20:00 e is_active=true.
2	A las 15:30 de TramaClock, AdvertisementSelectorService lo considera vigente si el archivo físico todavía existe.
3	Laravel lo entrega al layout. Si hay más de un header vigente, Vue rota la pieza cada 6,5 segundos.
4	La imagen carga y al menos el 50 % entra en la zona visible del navegador (viewport). AdvertisementBanner.vue llama a la ruta de impresión una sola vez para esa aparición.
5	El visitante hace clic. El navegador abre directamente target_url y, en paralelo, keepalive intenta registrar el clic sin interponer una redirección de TRAMA.
6	AdvertisementDailyMetric busca o crea la fila de esa publicidad y del día editorial y aumenta impressions_count o clicks_count según la acción.
7	A las 20:01 la campaña ya no cumple ends_at >= TramaClock::now() y deja de ser seleccionada.

IntersectionObserver evita contar banners que fueron enviados al navegador pero nunca llegaron a verse. Si el navegador no soporta esa API, TRAMA utiliza una alternativa y registra la impresión después de que la imagen cargó. Un error de tracking nunca debe impedir que el visitante lea la noticia ni abra el sitio del anunciante.

Páginas de pruebas de los anunciantes

Los banners del conjunto de datos de prueba pueden apuntar a rutas locales como /demo/anunciantes/{advertiser}. AdvertiserDemoController reconoce únicamente las marcas ficticias definidas por el proyecto y responde con 404 para un slug desconocido. Estas páginas se renderizan con una vista Blade independiente: no pasan por Inertia, Trama.vue ni el layout público de TRAMA, porque representan el sitio externo ficticio al que llega el visitante después del clic.

Ejemplo: un banner de Banco NEXO puede tener como destino /demo/anunciantes/banco-nexo. El clic se registra mediante el mecanismo publicitario ya explicado y el navegador abre directamente esa landing ficticia del anunciante. Allí Laravel muestra la marca, el mensaje, los beneficios y la llamada a la acción preparados para ese anunciante, sin reconstruir antes una pantalla del portal de noticias.

Código fuente real: cuándo se registra una impresión

resources/js/Components/News/AdvertisementBanner.vue comprueba primero que la imagen haya cargado y que el banner no esté roto. Después utiliza IntersectionObserver con un umbral del 50 %:

```

impressionObserver = new IntersectionObserver(
  (entries) => {
    const isVisible = entries.some(
      (entry) => entry.isIntersecting && entry.intersectionRatio >= 0.5
    );

    if (isVisible) {
      registerImpression();
    }
  },
  {
    threshold: [0.5],
  }
);

impressionObserver.observe(element);

```

Cuando `registerImpression()` se ejecuta por primera vez, marca localmente la impresión para no repetirla y envía la petición de medición. Si el navegador no dispone de `IntersectionObserver`, TRAMA utiliza como alternativa el registro después de comprobar que la imagen cargó correctamente.

```

void window.fetch(impressionUrl, {
  method: 'GET',
  credentials: 'same-origin',
  cache: 'no-store',
  keepalive: true,
  headers: {
    Accept: 'application/json',
    'X-Requested-With': 'XMLHttpRequest',
  },
}).catch(() => {
  // La medición nunca debe afectar la lectura ni ocultar el banner.
});

```

`keepalive` permite que el navegador intente completar el registro aunque la página cambie de estado. El mismo principio se utiliza al registrar un clic: la navegación hacia el anunciante no depende de que la ruta de métricas (endpoint) termine correctamente.

Código fuente real: del clic al contador diario

`AdvertisementBanner.vue` conserva el destino real en `href` y registra la métrica en el evento del enlace. La petición interna usa `click_url`; el Controller público incrementa únicamente `clicks_count` y devuelve una respuesta vacía.

```

<a
    :href="advertisement.target_url"
    target="_blank"
    @click="registerClick"
    @auxclick.middle="registerClick"
>

void window.fetch(clickUrl, {
    method: 'GET',
    credentials: 'same-origin',
    cache: 'no-store',
    keepalive: true,
});

// AdvertisementClickController
$dailyMetric = AdvertisementDailyMetric::query()->firstOrCreate(
    [
        'advertisement_id' => $advertisement->id,
        'date' => TramaClock::today()->toDateString(),
    ],
    [
        'impressions_count' => 0,
        'clicks_count' => 0,
    ],
);

$dailyMetric->increment('clicks_count');
return response()->noContent();

```

La impresión utiliza el mismo modelo diario pero aumenta `impressions_count`. Por eso una sola fila puede resumir, por ejemplo, 1.000 impresiones y 10 clics de una campaña durante un día sin almacenar una fila individual por cada visitante.

27. Vistas, perfiles y búsquedas

Vistas y ranking de noticias

Cuando se abre una noticia pública, `ArticleController` aumenta `articles.views` y también crea una fila en `article_views`. Si Candela abre una noticia y luego la recarga, ambas cargas pueden sumar vistas. Por eso la métrica no representa personas únicas.

`articles.views` guarda rápidamente el total que necesita la portada y los listados. `article_views` conserva el detalle de cada carga registrada.

Paso a paso: Candela abre una noticia y TRAMA registra la vista

La métrica de vistas se entiende mejor siguiendo una carga real de `ArticleShow` desde el clic hasta MySQL.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela selecciona una tarjeta y el navegador solicita <code>/noticias/{slug}</code> .
2	<code>ArticleController</code> pide a <code>ArticleQueryService</code> la noticia publicada correspondiente. Si el slug no pertenece a contenido públicamente visible, Laravel responde 404.
3	Dentro de una transacción, <code>ArticleController</code> incrementa <code>articles.views</code> para mantener un contador rápido en la propia noticia.

Paso	Qué hace la persona y qué ocurre internamente
4	En la misma transacción crea una fila en <code>article_views</code> con <code>article_id</code> , la fecha de <code>TramaClock</code> y hashes SHA-256 de IP y <code>user-agent</code> , sin guardar esos valores en texto claro.
5	El controlador vuelve a cargar la noticia con autor, categoría y etiquetas y prepara comentarios, relacionadas y espacios publicitarios.
6	<code>TramaBridge</code> envía las props a <code>Public/ArticleShow.vue</code> y <code>Candela</code> ve la página con el contador ya actualizado.
7	Si <code>Candela</code> recarga la página, el recorrido se repite y puede sumar otra vista. Por eso <code>views</code> cuenta cargas registradas, no personas únicas.

Código fuente real: ArticleController registra una vista

El recorrido anterior corresponde directamente a `app/Http/Controllers/Public/ArticleController.php`. `findPublishedBySlug()` obtiene una noticia pública; después una sola transacción mantiene sincronizados el contador rápido y el registro detallado:

```
$article = $articles->findPublishedBySlug($slug);

DB::transaction(function () use ($article, $request): void {
    $article->increment('views');

    ArticleView::query()->create([
        'article_id' => $article->id,
        'ip_hash' => hash('sha256', (string) $request->ip()),
        'user_agent_hash' => hash('sha256', (string) $request->userAgent()),
        'viewed_on' => TramaClock::today()->toDateString(),
    ]);
});

$article = $article->fresh(['author', 'category', 'tags']);
```

`increment("views")` ejecuta el aumento del contador de artículos. `ArticleView::create()` agrega la fila histórica del día con las huellas SHA-256. `fresh()` vuelve a leer el artículo para que `Vue` reciba el contador ya actualizado. Si cualquiera de las dos escrituras de la transacción falla, no debería quedar una mitad aplicada y la otra no.

Perfiles de periodistas

Cada periodista tiene una página pública con sus datos visibles y sus noticias publicadas. La relación `articles.author_id` → `users.id` conserva quién escribió cada nota. Si la foto física del periodista ya no existe, `User` puede devolver el avatar predeterminado para que la interfaz no muestre una imagen rota.

Qué muestra el perfil público de un periodista

`JournalistController::show()` acepta únicamente cuentas internas cuyo rol pertenece a `User::EDITORIAL_ROLES` y trabaja sobre sus publicaciones públicamente visibles. El perfil muestra nombre, cargo, biografía, avatar, cantidad total de noticias publicadas y fecha de la publicación más reciente. También calcula hasta cuatro secciones principales, ordenadas por cantidad de notas, y permite utilizar esas secciones como filtro dentro del mismo perfil. El correo del empleado no se envía a la pantalla pública.

El archivo de noticias del periodista se ordena por `published_at` y muestra 9 notas por página. Si el lector selecciona una de las secciones ofrecidas, la URL conserva el perfil y agrega el filtro `seccion`; `Laravel` sólo aplica ese filtro cuando corresponde a una de las secciones disponibles para ese periodista.

Paso a paso: Candela filtra el archivo público de Valentina

Valentina tiene publicaciones en varias secciones y Candela quiere ver solamente las de Economía. El recorrido siguiente muestra la acción visible y lo que sucede internamente hasta que Vue recibe la primera página filtrada.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela abre el perfil público de Valentina. <code>JournalistController</code> acepta únicamente cuentas internas cuyo rol pertenece a <code>User::EDITORIAL_ROLES</code> .
2	Laravel cuenta las noticias públicas del perfil, calcula la última fecha publicada y obtiene hasta cuatro secciones principales ordenadas por cantidad de notas.
3	<code>JournalistShow.vue</code> recibe esos datos y muestra los chips de sección. Cada chip contiene una URL construida por Laravel para el mismo perfil con el parámetro <code>seccion</code> .
4	Candela pulsa Economía. <code>Inertia</code> solicita el perfil con <code>?seccion=economia</code> conservando el estado y la posición de scroll.
5	<code>JournalistController</code> sólo acepta el filtro si ese slug pertenece realmente a una de las secciones disponibles para Valentina y entonces limita la consulta por categoría.
6	Las noticias se ordenan por <code>published_at</code> , se pagan de a 9 y <code>withQueryString()</code> conserva el filtro en los enlaces siguientes.
7	La respuesta vuelve a <code>JournalistShow.vue</code> , que marca Economía como activa y reemplaza la grilla sin exponer el email ni otros datos privados del perfil.

Código fuente real: cómo `JournalistController` arma el perfil

El perfil tampoco se arma en Vue a partir de todos los datos de users. `JournalistController` selecciona explícitamente qué información es pública y prepara una consulta paginada de noticias visibles.

```
abort_unless(
    in_array($user->role, User::EDITORIAL_ROLES, true),
    404,
);

$publishedArticlesQuery = $this->publishedArticlesQuery($user);
$publishedArticlesCount = (clone $publishedArticlesQuery)->count();
$latestPublishedAt = (clone $publishedArticlesQuery)->max('published_at');
$sections = $this->sectionsWrittenByJournalist($publishedArticlesQuery);

$articles = $filteredArticlesQuery
    ->with(['author', 'category', 'tags'])
    ->latest('published_at')
    ->paginate(9)
    ->withQueryString();
```

Además de la consulta paginada mostrada, la respuesta pública que `TramaBridge` entrega a Vue incluye una prop `journalist` con `id`, `name`, `slug`, `job_title`, `bio`, `avatar`, cantidades, secciones y URL pública; no incluye email. El filtro `seccion` sólo se acepta cuando su slug pertenece a las secciones calculadas para ese periodista.

Búsqueda pública

Si un visitante busca “fútbol”, TRAMA compara ese texto con el título, la bajada, el resumen, la categoría y las etiquetas de las noticias publicadas. Puede limitar los resultados a una categoría. No busca dentro del cuerpo completo para evitar que una palabra aislada en una nota larga genere resultados poco útiles.

Paso a paso: Candela busca “fútbol” en el portal público

La búsqueda pública prioriza campos que ayudan a encontrar una nota por tema sin revisar todo el cuerpo completo.

Paso	Qué hace la persona y qué ocurre internamente
1	Candela escribe fútbol en Public/Search.vue. v-model mantiene form.q sincronizado y computed canSearch habilita el botón cuando existe al menos un carácter real.
2	Al enviar, search() recorta espacios y usa router.get(route("search")) con q y la categoría seleccionada.
3	SearchController recibe los query parameters, normaliza el término y llama a ArticleQueryService::searchPublishedArticles().
4	El servicio parte de Article::publiclyVisible() y busca con LIKE en title, subtitle y excerpt, además de category.name y tags.name.
5	Si Candela eligió una categoría, Laravel agrega ese filtro por slug. El cuerpo completo no participa en esta búsqueda pública.
6	Los resultados se ordenan por published_at, se paginan de a 9 y mantienen q y category en los enlaces siguientes.
7	TramaBridge devuelve filters, categories y results. Search.vue conserva lo escrito y PaginatedArticleGrid muestra las noticias encontradas.

Código fuente real: de Search.vue a la consulta pública

Public/Search.vue genera una navegación GET con q y category. routes/web.php resuelve GET /buscar hacia SearchController; el Controller normaliza q y delega la búsqueda real en ArticleQueryService::searchPublishedArticles().

```
// Search.vue
router.get(route('search'), {
  q: form.q.trim(),
  category: form.category || undefined,
});

// SearchController
$term = trim((string) $request->query('q'));
$category = $request->query('category');
$results = $articles->searchPublishedArticles($term, $category);
```

Dentro del Service se parte siempre de publiclyVisible(). applySearchTerm() escapa los comodines % y _ y construye el LIKE sólo sobre título, bajada, resumen, categoría y etiquetas. El cuerpo queda deliberadamente fuera de este buscador público.

```

$query = Article::query()->publiclyVisible();

$like = '%'.str_replace(
    ['%', '_'],
    ['\%', '\_'],
    $term,
).'%';

$query->where(function (Builder $inner) use ($like): void {
    $inner
        ->where('title', 'like', $like)
        ->orWhere('subtitle', 'like', $like)
        ->orWhere('excerpt', 'like', $like)
        ->orWhereHas('category', fn (Builder $category) =>
            $category->where('name', 'like', $like)
        )
        ->orWhereHas('tags', fn (Builder $tag) =>
            $tag->where('name', 'like', $like)
        );
});

```

Después, el cierre de `searchPublishedArticles()` aplica el filtro opcional de categoría, carga `author/category/tags`, ordena por `published_at` y pagina de a 9. El segundo fragmento muestra esas operaciones y `withQueryString()`, que conserva `q` y `category` al pasar de página:

```

return $query
    ->when($categorySlug, function ($query) use ($categorySlug): void {
        $query->whereHas(
            'category',
            fn ($category) => $category
                ->publiclyVisible()
                ->where('slug', $categorySlug)
        );
    })
    ->with(['author', 'category', 'tags'])
    ->latest('published_at')
    ->paginate(9)
    ->withQueryString();

```

Búsqueda dentro del panel

El panel editorial necesita una búsqueda más amplia. Por eso cada noticia tiene un campo interno llamado `search_text` que reúne título, bajada, resumen, cuerpo sin HTML, categoría y etiquetas en un texto preparado para buscar.

MySQL crea sobre ese campo un índice llamado `FULLTEXT`. Un índice es una estructura que permite encontrar datos más rápido sin recorrer de la misma forma todas las filas completas. `FULLTEXT` está pensado especialmente para localizar palabras dentro de textos.

```

título + bajada + resumen + cuerpo + categoría + etiquetas
      ↓
articles.search_text
      ↓
índice FULLTEXT

```

Paso a paso: Mateo busca una noticia desde la grilla del CMS

La búsqueda interna es más amplia que la pública porque necesita localizar trabajo editorial incluso por palabras presentes en el cuerpo de la nota.

Paso	Qué hace la persona y qué ocurre internamente
1	Mateo escribe logística en el buscador de Admin/Articles/Index.vue. El componente normaliza el texto y conserva los demás filtros activos.
2	applySearch() llama a visitList(), que usa router.get() hacia admin.articles.index con preserveState, preserveScroll y replace.
3	Laravel valida y limpia los filtros del índice antes de construir la consulta de noticias accesibles para el rol del usuario.
4	ArticleIndexQueryService extrae términos válidos de tres caracteres o más y, en MySQL, transforma cada uno en una búsqueda FULLTEXT booleana por prefijo, por ejemplo +logística*.
5	whereFullText() consulta articles.search_text. Ese campo ya contiene título, bajada, resumen, cuerpo en texto plano, categoría y etiquetas.
6	La consulta combina la búsqueda con estado, autor, categoría, fecha u otros filtros que Mateo ya hubiera aplicado.
7	La respuesta vuelve a la misma grilla. Vue reemplaza las filas y restaura la posición de scroll que el componente guardó antes de la navegación.

Cómo se mantiene search_text

ArticleSearchIndexer construye search_text reuniendo título, bajada, resumen, cuerpo convertido a texto plano, nombre de categoría y nombres de etiquetas. HtmlSanitizer elimina HTML y normaliza el contenido antes de concatenarlo. La actualización se realiza directamente sobre search_text para no cambiar updated_at ni crear una revisión editorial sólo porque se reconstruyó un dato auxiliar de búsqueda.

```
título + bajada + resumen + cuerpo sin HTML + categoría + etiquetas
↓
ArticleSearchIndexer::build()
↓
articles.search_text
↓
índice FULLTEXT de MySQL
```

Cuando se guarda, actualiza o restaura una noticia, el flujo editorial vuelve a preparar este texto. Si por una migración, importación o mantenimiento se necesita reconstruirlo para todas las noticias, puede ejecutarse php artisan articles:rebuild-search-index. El comando procesa por bloques de 500 de forma predeterminada y admite --chunk para cambiar ese tamaño sin cargar todas las noticias en memoria al mismo tiempo.

Búsqueda	Campos considerados	Comportamiento
Pública (/buscar)	Título, bajada, resumen, categoría y etiquetas.	Sólo noticias públicamente visibles; puede filtrar por categoría y muestra 9 resultados por página. No busca el cuerpo completo.
Editorial (CMS)	Título, bajada, resumen, cuerpo sin HTML, categoría y etiquetas reunidos en search_text.	Utiliza FULLTEXT para una búsqueda interna más amplia y rápida.

Paso a paso: Valentina cambia una etiqueta y search_text se actualiza sin fingir una edición nueva

search_text es un dato auxiliar para buscar; reconstruirlo no debería modificar updated_at ni llenar el historial con revisiones artificiales.

Paso	Qué hace la persona y qué ocurre internamente
1	Valentina guarda cambios editoriales que modifican texto, categoría o etiquetas de una noticia.
2	Después de sincronizar las relaciones definitivas, ArticleSearchIndexer carga category y tags para trabajar con los nombres actuales.
3	build() reúne title, subtitle, excerpt, body, category.name y cada tag.name.
4	HtmlSanitizer::toPlainText() elimina HTML, normaliza el texto y descarta fragmentos vacíos antes de concatenarlos.
5	index() actualiza únicamente la columna articles.search_text mediante DB::table(...)->update().
6	Como no vuelve a guardar todo el modelo, esa reconstrucción auxiliar no cambia updated_at ni dispara una revisión editorial por sí sola.
7	La siguiente búsqueda FULLTEXT del CMS utiliza inmediatamente el texto reconstruido con los nombres y contenidos actuales.

Ejemplo: la misma palabra en dos buscadores

Si “litio” aparece únicamente en el sexto párrafo del cuerpo de una noticia, la búsqueda pública puede no devolverla porque prioriza campos visibles de identificación. En el panel editorial, “litio” sí forma parte de search_text y puede localizar esa nota. Esta diferencia es deliberada: el lector recibe resultados más relevantes y el equipo interno dispone de una búsqueda documental más amplia.

Noticias relacionadas

Al abrir una noticia, ArticleQueryService::related() busca otras publicaciones visibles que compartan la misma categoría o alguna etiqueta, excluye la noticia actual, ordena por fecha de publicación y ArticleController solicita tres para el bloque “Seguir leyendo”. Así la relación no se guarda manualmente en una tabla: se calcula a partir de la taxonomía que ya posee cada noticia.

Paso a paso: Candela termina una noticia y TRAMA elige qué mostrar en “Seguir leyendo”

Las relacionadas se resuelven en el servidor para que el criterio editorial sea el mismo independientemente de cómo Vue pinte las tarjetas.

Paso	Qué hace la persona y qué ocurre internamente
1	ArticleController ya tiene la noticia actual y sus etiquetas cargadas.
2	ArticleQueryService::related() parte de noticias públicamente visibles y excluye la noticia que Candela está leyendo.
3	La consulta acepta candidatas de la misma categoría o que compartan alguna de las etiquetas de la nota actual.
4	Las candidatas se ordenan por published_at para priorizar contenido reciente dentro de ese conjunto relacionado.

Paso	Qué hace la persona y qué ocurre internamente
5	ArticleController pide tres resultados para completar la fila de Seguir leyendo y los convierte mediante ArticleCardResource.
6	TramaBridge envía related junto con la noticia principal y ArticleShow.vue reutiliza las tarjetas públicas para mostrarlas al final.

Qué guarda una vista

ArticleController incrementa `articles.views` y crea una fila en `article_views` dentro de la misma transacción. La fila guarda `article_id`, `viewed_on` y hashes SHA-256 de la IP y del User-Agent en lugar de conservar esos valores en texto claro. El hash permite mantener una referencia técnica sin convertir `article_views` en un registro directo de direcciones IP o cadenas completas del navegador. Aun así, cada recarga válida puede sumar otra vista: la métrica no representa usuarios únicos.

El User-Agent es una cadena técnica que el navegador envía para identificarse ante el servidor. Un hash es una huella derivada de un dato: permite comparar o reconocer una referencia sin guardar directamente el valor original. TRAMA conserva esas huellas en `article_views` en lugar de almacenar la IP y el User-Agent completos en texto claro.

Código fuente real: la búsqueda FULLTEXT del CMS

ArticleIndexQueryService convierte cada término en una búsqueda obligatoria por prefijo y utiliza `whereFullText()` cuando la conexión es MySQL. Por ejemplo, `ciudad` se transforma en `+ciudad*` para aceptar palabras que comienzan de esa forma:

```
$booleanSearch = implode(
    ', ',
    array_map(
        static fn (string $term): string => '+' . $term . '*',
        $searchTerms,
    ),
);

$query->whereFullText(
    'articles.search_text',
    $booleanSearch,
    [
        'mode' => 'boolean',
    ],
);
```

La columna `search_text` posee un índice FULLTEXT creado por una migración cuando el driver es MySQL. Durante PHPUnit se utiliza SQLite en memoria, que no admite esta búsqueda; por eso ArticleIndexQueryService aplica allí una alternativa con LIKE para poder comprobar el comportamiento general sin depender de una función exclusiva de MySQL.

28. Colas, tareas programadas, transacciones y concurrencia

TRAMA utiliza colas, workers, tareas programadas, transacciones y bloqueos porque no todos los trabajos deben ejecutarse en el mismo momento ni bajo las mismas garantías. Esta sección explica desde cero cómo se ejecutan esos mecanismos y después los conecta con dos casos reales: la moderación automática de comentarios y la publicación programada de noticias.

Por qué TRAMA necesita trabajo fuera de la petición HTTP

Una petición web tiene una persona esperando del otro lado. Cuando Candela envía un comentario, Laravel necesita validar y guardar lo imprescindible, pero no conviene mantener el navegador esperando mientras Node.js carga detectores y modelos. En otros casos ni siquiera existe una persona haciendo clic: una noticia programada debe publicarse cuando llega su hora. Queue y Scheduler resuelven problemas distintos de ejecución.

Job, cola y worker: tres piezas distintas

Pieza	Qué representa	Ejemplo real en TRAMA
Job	Un objeto que describe un trabajo que Laravel podrá ejecutar después.	ModerateComment contiene el id del comentario y moderation_revision y sabe ejecutar la moderación automática.
Queue / cola	El lugar lógico donde un Job queda pendiente hasta que pueda ser tomado.	ModerateComment usa la cola moderation.
Worker	Un proceso de Laravel que permanece ejecutándose, busca Jobs disponibles y llama a su método handle().	<pre>php artisan queue:work -- queue=moderation,default --tries=3 -- timeout=70</pre>

No son sinónimos. ModerateComment es el trabajo; moderation es el nombre de la cola; queue:work es el proceso que consume trabajos. Despachar un Job no significa que ya se esté ejecutando.

QUEUE_CONNECTION=database: dónde queda esperando el Job

En el entorno documentado, .env define QUEUE_CONNECTION=database. config/queue.php convierte esa opción en la conexión de colas por base de datos, cuya tabla predeterminada es jobs. La migración de Laravel crea allí columnas como queue, payload, attempts, reserved_at y available_at.

```
Schema::create('jobs', function (Blueprint $table) {
    $table->id();
    $table->string('queue')->index();
    $table->longText('payload');
    $table->unsignedTinyInteger('attempts');
    $table->unsignedInteger('reserved_at')->nullable();
    $table->unsignedInteger('available_at');
    $table->unsignedInteger('created_at');
});
```

La fila de jobs no reemplaza al comentario. comments conserva el dato editorial con status=processing; jobs conserva la descripción serializada del trabajo que falta ejecutar. Cuando un worker reserva el Job, incrementa su seguimiento de intentos y lo ejecuta. Si termina correctamente, el trabajo pendiente deja de estar en esa cola; si falla, Laravel puede reintentarlo según su configuración.

Código fuente real: cómo está definido ModerateComment

app/Jobs/ModerateComment.php declara tanto la política de reintentos como la cola que utiliza. afterCommit() es importante: si el Job se despacha dentro de una transacción, Laravel no debe dejarlo disponible antes de que la fila del comentario quede confirmada en MySQL.

```

public int $tries = 3;
public int $timeout = 70;
public bool $failOnTimeout = true;

public function __construct(
    public readonly int $commentId,
    public readonly int $moderationRevision,
) {
    $this->onQueue('moderation');
    $this->afterCommit();
}

public function backoff(): array
{
    return [5, 15, 30];
}

public function handle(): void
{
    $this->processAutomaticModeration();
}

```

tries=3 significa un primer intento y hasta dos reintentos. timeout=70 limita la duración total de una ejecución del Job; el proceso Node tiene a su vez un timeout menor. backoff() indica pausas de 5, 15 y 30 segundos entre intentos fallidos. handle() no lo llama CommentController manualmente: es el punto que invoca Laravel cuando el worker toma el Job.

Cómo llega ModerateComment a la cola

Después de guardar un comentario de lector como processing, CommentController llama a un método auxiliar que intenta despachar la versión exacta que debe analizarse. Si la infraestructura ni siquiera puede recibir el Job, el catch no deja la fila eternamente en processing: la deriva a pending con una razón técnica.

```

try {
    ModerateComment::dispatch(
        $comment->id,
        (int) $comment->moderation_revision,
    );
} catch (Throwable $exception) {
    report($exception);

    Comment::query()
        ->whereKey($comment->id)
        ->where('status', 'processing')
        ->where('moderation_revision', (int) $comment->moderation_revision)
        ->update([
            'status' => 'pending',
            'moderation_source' => 'automatic',
            'moderation_reason' => 'automation_enqueue_error',
        ]);
}

```

Cómo se pone en marcha el worker

```
php artisan queue:work --queue=moderation,default --tries=3 --timeout=70
```

Este comando debe permanecer activo durante el desarrollo si se quiere ejecutar trabajo asíncrono. --queue=moderation,default indica qué colas debe consumir; --tries y --timeout fijan límites del worker, mientras el Job también declara sus propias propiedades. El worker consulta la conexión database, toma un Job disponible y Laravel ejecuta ModerateComment::handle().

Si el worker está apagado, el comentario ya guardado no desaparece: continúa processing y el Job permanece pendiente hasta que vuelva a existir un worker. Si ModerateComment llega a agotar sus intentos, failed() registra el error

técnico y, si esa revisión sigue vigente, cambia el comentario a pending con `moderation_reason=automation_error`. La regla es deliberada: un fallo técnico nunca se interpreta como aprobación.

Paso a paso: Candela envía un comentario y el worker lo procesa

Paso	Qué hace la persona y qué ocurre internamente
1	Candela pulsa Enviar comentario. <code>CommentController</code> valida la participación y crea <code>comments</code> con <code>status=processing</code> y <code>moderation_revision=1</code> .
2	Dentro del flujo de guardado se despacha <code>ModerateComment</code> con el id y la revisión. <code>afterCommit()</code> evita que el Job quede disponible antes de confirmar la transacción.
3	Con <code>QUEUE_CONNECTION=database</code> , Laravel serializa el trabajo en la tabla <code>jobs</code> y lo asigna a <code>queue=moderation</code> . La petición web puede terminar sin esperar a <code>Node.js</code> .
4	El proceso <code>queue:work</code> detecta el Job, lo reserva y Laravel instancia la clase. No es <code>Vue</code> ni <code>CommentController</code> quien llama <code>handle()</code> .
5	<code>handle()</code> delega en <code>processAutomaticModeration()</code> , que vuelve a buscar <code>comments</code> y comprueba que <code>status</code> y <code>moderation_revision</code> sigan coincidiendo con el trabajo.
6	<code>ModerateComment</code> inicia <code>node scripts/comments/moderate-comment.mjs</code> mediante <code>Process::run()</code> . Node coordina idioma, enlaces, amenazas, toxicidad y spam y devuelve JSON por <code>stdout</code> .
7	Laravel valida el resultado y actualiza la misma fila de <code>comments</code> a <code>approved</code> , <code>pending</code> o <code>rejected</code> . Si la revisión cambió mientras el Job trabajaba, el resultado viejo no se aplica.
8	Cuando el Job termina correctamente sale de la cola. <code>ArticleShow.vue</code> puede detectar el nuevo estado mediante el polling documentado en la sección 22.

Qué es Laravel Scheduler y por qué una fecha guardada no alcanza

Scheduler resuelve otro problema. Guardar `scheduled_at=16:40` sólo conserva una fecha en MySQL; esa columna no ejecuta código cuando el reloj llega a las 16:40. El proyecto necesita un proceso que revise periódicamente qué tareas corresponden. Laravel Scheduler gestiona esa programación y ejecuta las tareas cuando corresponde.

`routes/console.php` registra la tarea de TRAMA. Esta declaración define qué comando debe ejecutarse y con qué frecuencia, pero no inicia por sí sola ningún proceso:

```
Schedule::command('articles:publish-scheduled')
    ->everyMinute()
    ->withoutOverlapping();
```

`everyMinute()` registra la frecuencia. `withoutOverlapping()` evita iniciar una segunda ejecución del mismo comando mientras la anterior siga activa. Cuando llega un tick del Scheduler, Laravel evalúa sus tareas y ejecuta `articles:publish-scheduled` si corresponde.

schedule:work en desarrollo y schedule:run en producción

Entorno / comando	Qué ocurre
Desarrollo · <code>php artisan schedule:work</code>	Mantiene un proceso activo en la terminal que revisa continuamente la agenda de Laravel. Es cómodo para trabajar localmente.

Entorno / comando	Qué ocurre
Producción · <code>php artisan schedule:run</code>	Realiza una revisión puntual de las tareas que vencen en ese momento y termina. El sistema operativo o la plataforma debe invocarlo cada minuto.

TRAMA no debe confundir este proceso con TramaClock. TramaClock define qué hora editorial considera actual el proyecto; Scheduler decide cuándo lanzar tareas y puede consultar esa hora a través del comando de publicación programada.

Paso a paso: Mateo programa una noticia para las 16:40

Paso	Qué hace la persona y qué ocurre internamente
1	Mateo elige 16:40 en el formulario editorial. ArticleRequest valida la fecha y ArticleWorkflowService guarda <code>status=scheduled</code> y <code>scheduled_at=16:40</code> .
2	La noticia queda almacenada, pero guardar <code>scheduled_at</code> no inicia ningún temporizador dentro de esa fila ni ejecuta el comando automáticamente.
3	En desarrollo Carlos deja <code>php artisan schedule:work</code> activo. En producción la plataforma invoca <code>php artisan schedule:run</code> cada minuto.
4	Laravel lee <code>routes/console.php</code> y encuentra <code>articles:publish-scheduled</code> con <code>everyMinute()</code> . A las 16:39 puede ejecutar el comando, pero éste comprueba que la hora todavía no llegó y no publica esa noticia.
5	Cuando TramaClock alcanza 16:40, una revisión posterior del Scheduler vuelve a lanzar el comando. PublishScheduledArticles busca <code>status=scheduled</code> cuya <code>scheduled_at</code> ya venció.
6	El comando procesa en lotes y, para cada candidata, abre una transacción, relee la noticia con <code>lockForUpdate()</code> y vuelve a verificar <code>status</code> y <code>scheduled_at</code> sobre el estado real.
7	Si sigue correspondiendo, cambia <code>status</code> a <code>published</code> , guarda <code>published_at</code> , limpia <code>scheduled_at</code> y registra <code>published_automatically</code> en el historial.
8	La noticia puede entrar al portal según sus demás reglas públicas. <code>withoutOverlapping()</code> y <code>lockForUpdate()</code> reducen el riesgo de dos ejecuciones actuando sobre el mismo trabajo.

Queue y Scheduler no son lo mismo

Pregunta	Queue / worker	Scheduler
¿Qué lo dispara?	Ya existe un trabajo que fue despachado y está esperando ejecución.	Llegó una frecuencia o momento en el que debe revisarse una tarea.
Ejemplo TRAMA	ModerateComment después de que Candela envía un comentario.	<code>articles:publish-scheduled</code> cada minuto.

Pregunta	Queue / worker	Scheduler
Dónde se define	Job PHP + conexión de queue + nombre de cola.	routes/console.php mediante <code>Schedule::command(...)</code> .
Proceso de desarrollo	<code>php artisan queue:work</code>	<code>php artisan schedule:work</code>
Si el proceso está apagado	Los Jobs permanecen pendientes y no avanzan hasta que un worker los consuma.	Las tareas periódicas no se disparan hasta que el Scheduler vuelva a ser ejecutado.

Queue y Scheduler pueden combinarse en otras arquitecturas, pero en estos dos flujos de TRAMA cumplen responsabilidades separadas. A partir de aquí aparece otro problema distinto: aunque el trabajo se ejecute en el momento correcto, varias operaciones podrían intentar modificar los mismos datos. Eso es lo que resuelven transacciones, atomicidad y bloqueos.

Transacciones y atomicidad

Una transacción agrupa varios cambios de MySQL como una sola operación. Si todos terminan correctamente se confirman; si uno falla, MySQL revierte el conjunto. Esta propiedad de “todo o nada” se denomina atomicidad.

Operaciones simultáneas

Dos peticiones pueden intentar modificar el mismo registro casi al mismo tiempo. Esa situación se denomina concurrencia. TRAMA utiliza `lockForUpdate()` cuando una operación necesita leer un estado y actualizarlo sin que otra petición se adelante durante ese intervalo.

Paso a paso: Valentina guarda mientras `lockForUpdate()` ordena las dos escrituras

Valentina termina una corrección justo cuando vence el temporizador del autoguardado y, casi al mismo tiempo, pulsa Guardar cambios. Las dos peticiones quieren escribir sobre la misma noticia. La primera que llega abre una transacción y vuelve a consultar la fila con `lockForUpdate()`. MySQL mantiene esa fila bloqueada para la otra escritura hasta que la transacción termina. La segunda petición espera; cuando obtiene el bloqueo vuelve a leer la noticia en su estado actual y aplica sus reglas sobre ese dato ya actualizado.

Paso	Qué hace Valentina y qué ocurre internamente
A	Valentina termina una corrección justo cuando se cumple el tiempo del autoguardado. Vuelve ya puede iniciar la petición silenciosa sobre la noticia.
B	Casi al mismo tiempo, Valentina pulsa Guardar cambios. La segunda petición intenta trabajar sobre la misma fila y debe respetar el bloqueo de la operación que llegó primero.
C	La primera operación abre una transacción, vuelve a leer la noticia con <code>lockForUpdate()</code> , aplica sus validaciones, escribe y confirma. Al terminar libera el bloqueo.
D	La segunda petición continúa después, vuelve a leer la noticia ya actualizada y aplica sus reglas sobre el estado real. Así las dos escrituras se ordenan en lugar de decidir sobre una fila que cambió a mitad de la operación.

El mismo principio aparece en varias partes del proyecto: `ArticleWorkflowService`, publicación programada, orden de categorías, fusión de etiquetas, moderación y administración de cuentas. `lockForUpdate()` no reemplaza a una transacción: el bloqueo tiene sentido porque se mantiene mientras la transacción protege el conjunto de lectura, comprobación y escritura.

Código fuente real: transacción y bloqueo en una operación cotidiana

ArticleWorkflowService::autosave() ofrece un ejemplo pequeño del patrón que TRAMA repite en operaciones sensibles. Primero abre una transacción, después vuelve a leer la noticia con lockForUpdate() y recién entonces aplica la actualización:

```
return DB::transaction(function () use ($article, $attributes, $user): Article {
    $locked = Article::query()
        ->whereKey($article->id)
        ->lockForUpdate()
        ->firstOrFail();

    if (! in_array($locked->status, Article::JOURNALIST_EDITABLE_STATUSES, true)) {
        throw ValidationException::withMessages([
            'article' => 'La noticia ya no admite guardado automático.',
        ]);
    }

    $locked->update($attributes);

    return $locked->refresh();
}, attempts: 3);
```

La transacción define el período durante el cual el bloqueo tiene sentido. Si otra petición intenta bloquear la misma noticia, espera hasta que esta operación termine. Cuando obtiene el bloqueo vuelve a leer el estado real y puede descubrir que la noticia ya dejó de ser editable. Así lockForUpdate() evita tomar una decisión sobre un dato que cambió mientras la petición estaba en curso.

29. Manejo de errores, logs y páginas de contingencia

TRAMA distingue entre los errores esperables del funcionamiento normal y los fallos técnicos inesperados. Si una persona deja un campo obligatorio vacío, intenta entrar en una pantalla sin permiso o solicita una dirección que no existe, Laravel ya dispone de una respuesta concreta para esa situación. En cambio, si durante una operación aparece una excepción inesperada, el sistema necesita guardar suficiente información técnica para poder investigarla sin mostrar esos detalles internos al usuario.

Validación, autorización y excepción no son lo mismo

Una validación responde a una regla conocida. Por ejemplo, una noticia sin categoría no puede enviarse a revisión: ArticleRequest detecta el problema y devuelve un mensaje asociado al formulario. Una autorización responde a otra pregunta conocida: si la cuenta actual tiene permiso para realizar una acción. Una excepción técnica es diferente: representa un problema producido mientras el código intentaba completar una operación que, en condiciones normales, debía poder terminar.

PHP representa esos problemas técnicos mediante objetos de excepción. En TRAMA suelen recibirse en una variable llamada \$exception. Ese objeto permite consultar, entre otros datos, la clase del error, su mensaje técnico, el archivo y la línea donde se produjo. Esa información puede ser útil para diagnosticar el problema, pero no debe enviarse tal cual al navegador.

Qué es un log y qué significa logging

Un log es un registro técnico de lo que ocurrió durante la ejecución de una aplicación. Puede guardar la fecha y hora, el nivel del problema, un mensaje y datos de contexto que ayuden a reconstruir el fallo. Logging es el proceso de crear y escribir esos registros. En TRAMA los errores inesperados importantes se separan del log general de Laravel y terminan en archivos diarios propios, por defecto dentro de storage/logs.

El objetivo del log no es mostrar un mensaje al visitante. Su destinatario es quien mantiene el proyecto. Por eso puede contener información técnica que nunca debería aparecer en la interfaz, mientras que el usuario recibe únicamente un texto breve y seguro que le permita volver a intentar o continuar.

Ejemplo 1: un fallo capturado al guardar una consulta de Contacto

La sección 25 explicó que Contacto guarda primero la consulta en MySQL. El método `InstitutionalController::storeContact()`, ubicado en `app/Http/Controllers/Public/InstitutionalController.php`, rodea esa escritura con `try` y `catch`. `try` delimita el código que puede fallar. Si la inserción termina correctamente, `catch` no se ejecuta. Si se produce un `Throwable`, PHP interrumpe el bloque y entrega el objeto del problema en `$exception`.

```
try {
    $contactMessage = ContactMessage::query()->create($request->validated());
} catch (Throwable $exception) {
    $this->operationFailures->fail(
        $exception,
        'public_contact',
        'store_message',
        'No pudimos enviar tu consulta en este momento. Intentá nuevamente.',
        [
            'email' => $request->validated('email'),
            'reason' => $request->validated('reason'),
        ],
    );
}
```

`Throwable` es el tipo general que permite capturar errores y excepciones de PHP. Si `ContactMessage::create()` funciona, la ejecución continúa normalmente. Si falla, `catch` no intenta construir el log por su cuenta: entrega la excepción a una clase auxiliar llamada `OperationFailureHandler`.

Qué es `OperationFailureHandler` y por qué existe

`OperationFailureHandler` es una clase auxiliar ubicada en `app/Support/OperationFailureHandler.php`. Su función es evitar que cada controlador repita el mismo procedimiento cuando una operación falla de manera inesperada. El controlador le entrega cinco cosas: la excepción recibida, el área del sistema, el nombre de la operación, el mensaje seguro que puede ver el usuario y, cuando hace falta, datos de contexto adicionales.

El método `fail()` primero comprueba si el problema pertenece a una situación que Laravel ya sabe manejar de forma específica. Las validaciones, la autenticación, los permisos, los errores HTTP que Laravel ya sabe tratar —por ejemplo un 404 por un recurso no encontrado— y una caída completa de MySQL no se convierten en un mensaje general: la excepción se vuelve a lanzar para conservar su tratamiento correspondiente. Sólo cuando se trata de un fallo inesperado registra el detalle técnico y después devuelve a la interfaz el mensaje seguro indicado por el controlador.

```

if (
    $exception instanceof ValidationException
    || $exception instanceof AuthenticationException
    || $exception instanceof AuthorizationException
    || $exception instanceof ModelNotFoundException
    || $exception instanceof HttpExceptionInterface
    || $this->isDatabaseUnavailable($exception)
) {
    throw $exception;
}

TramaLog::error(
    'Falló una operación.',
    [
        ...$context,
        'area' => $area,
        'operation' => $operation,
        'exception_class' => $exception::class,
        'exception_message' => $exception->getMessage(),
        'exception_file' => $exception->getFile(),
        'exception_line' => $exception->getLine(),
    ]
);

throw ValidationException::withMessages([
    'operation' => $userMessage,
]);

```

Las dos partes del método tienen destinatarios diferentes. `TramaLog::error()` conserva la información necesaria para diagnosticar el fallo. La `ValidationException` final contiene únicamente `$userMessage`; Inertia puede mostrar ese texto en un formulario y una petición Axios lo recibe como respuesta 422. El navegador no recibe `exception_message`, `exception_file` ni `exception_line`.

Qué es TramaLog y dónde está

TramaLog es otra clase auxiliar propia del proyecto, ubicada en `app/Support/TramaLog.php`. Actúa como punto común para registrar los errores técnicos de TRAMA. Gracias a esta clase, los controladores y servicios no necesitan conocer el nombre del archivo de log, la forma de escribirlo ni la configuración interna que utiliza Laravel: llaman a TramaLog y la infraestructura de logging se ocupa del resto.

La clase tiene tres piezas principales. `error()` registra un mensaje y un contexto opcional; `mailFailureMessage()` transforma ciertos errores extensos del sistema de correo en mensajes técnicos más cortos; y `formatContext()` elimina valores null y prepara el contexto como JSON legible antes de incorporarlo al registro. Los dos primeros métodos son públicos porque otras partes del proyecto los llaman. `formatContext()` es privado porque sólo se utiliza dentro de la propia clase.

```

TramaLog
├── error()
│   └── registra el mensaje y el contexto técnico
├── mailFailureMessage()
│   └── resume determinados errores del mailer
└── formatContext()
    └── limpia el contexto y lo convierte a JSON legible

```

Cómo el código registra un error con TramaLog

Registrar un error no significa abrir manualmente un archivo. El código llama al método `TramaLog::error()`. Ese método recibe el mensaje principal y un array de contexto y utiliza la fachada Log de Laravel para seleccionar el canal propio de TRAMA llamado `trama_unexpected`.

```
public static function error(string $message, array $context = []): void
{
    Log::channel('trama_unexpected')->error(
        $message.self::formatContext($context)
    );
}
```

Log es la fachada que Laravel ofrece para trabajar con su sistema de logging. `channel('trama_unexpected')` selecciona una configuración concreta y `error(...)` registra el mensaje con nivel ERROR. Antes de enviarlo, `formatContext()` agrega debajo del mensaje un bloque JSON cuando existen datos útiles.

```
private static function formatContext(array $context): string
{
    $context = array_filter(
        $context,
        static fn (mixed $value): bool => $value !== null
    );

    if ($context === []) {
        return '';
    }

    return "\n".json_encode(
        $context,
        JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES
    );
}
```

Si el contexto queda vacío, `formatContext()` devuelve una cadena vacía y el registro conserva sólo el mensaje. Si hay datos, `JSON_PRETTY_PRINT` deja cada campo en una línea distinta y las otras opciones mantienen caracteres y rutas de forma legible.

Qué es un canal de logging y cómo se configura el de TRAMA

Un canal de logging es una configuración que le dice a Laravel cómo debe tratar un grupo de registros: qué tipo de destino utilizar, dónde escribir, qué nivel mínimo aceptar, cuánto tiempo conservar los archivos y qué personalizaciones aplicar. TRAMA define su canal de errores inesperados en `config/logging.php` con el nombre `trama_unexpected`.

```
'trama_unexpected' => [
    'driver' => 'daily',
    'path' => $tramaLogPath,
    'level' => 'error',
    'days' => 30,
    'tap' => [App\Logging\TramaLogFormatter::class],
    'replace_placeholders' => true,
],
```

`driver=daily` indica que se utiliza un archivo diferente por día. `path` contiene el archivo base; si `TRAMA_LOG_PATH` no está definido, el proyecto utiliza `storage/logs/trama-backend.log` y Laravel agrega la fecha al nombre final, por ejemplo `trama-backend-2026-09-15.log`. `level=error` establece ERROR como nivel mínimo, por lo que también admite niveles de mayor gravedad; `days=30` conserva hasta treinta archivos diarios. `replace_placeholders=true` permite que Laravel sustituya marcadores como `{id}` o `{email}` cuando el mensaje los utiliza y esos valores existen en el contexto.

La opción `tap` es especialmente importante: conecta el canal con `App\Logging\TramaLogFormatter`. Ningún controlador llama manualmente a esa clase. Cuando Laravel prepara el canal `trama_unexpected`, ejecuta la clase indicada en `tap` para permitir que TRAMA personalice el logger antes de empezar a escribir registros.

Monolog: el motor de logging que Laravel utiliza por debajo

Laravel ofrece una interfaz cómoda mediante Log y los canales definidos en `config/logging.php`, pero el trabajo interno de logging se apoya en una biblioteca PHP llamada Monolog. Monolog no es un proceso que haya que iniciar en otra terminal ni una herramienta que los controladores de TRAMA llamen directamente. Forma parte de la infraestructura que Laravel utiliza cuando se registra un mensaje.

La relación puede leerse así: el código de TRAMA llama a Log; Laravel consulta la configuración del canal; y por debajo Monolog administra las piezas que terminan enviando el registro a su destino. TRAMA sólo utiliza clases de Monolog de forma directa en TramaLogFormatter, porque allí necesita personalizar cómo se presenta cada entrada.

Handler y formatter: destino y presentación del registro

Dentro de Monolog, un handler es la pieza encargada de enviar o escribir un registro en un destino. Ese destino podría ser un archivo, la consola o un servicio externo. En el canal diario de TRAMA, Laravel prepara el handler necesario para escribir en el archivo correspondiente al día actual.

Un formatter cumple otra función: decide cómo se representa el registro antes de que el handler lo escriba. No decide qué errores entran al canal ni dónde se guardan. Sólo controla la presentación: orden de la fecha, nivel, mensaje, saltos de línea y otros detalles de formato.

Qué es TramaLogFormatter y cómo entra en el flujo

TramaLogFormatter es una clase propia de TRAMA ubicada en `app/Logging/TramaLogFormatter.php`. Su responsabilidad es personalizar la presentación de los registros del canal `trama_unexpected` para que el archivo sea corto y legible. La clase no captura excepciones, no decide qué debe registrarse y tampoco elige la ruta del archivo: esas decisiones ya fueron tomadas antes por el código y por `config/logging.php`.

Laravel ejecuta TramaLogFormatter automáticamente porque el canal la declaró en `tap`. Para hacerlo llama a su método especial `__invoke()`. PHP permite que un objeto que define `__invoke()` se ejecute como si fuera una función; Laravel aprovecha ese mecanismo para entregar a la clase el logger que acaba de preparar.

```
public function __invoke(LaravelLogger $logger): void
{
    $monolog = $logger->getLogger();

    if (! $monolog instanceof MonologLogger) {
        return;
    }

    $formatter = new LineFormatter(
        "[%datetime%] %level_name%: %message%\n",
        "Y-m-d H:i:s",
        true,
        true
    );

    foreach ($monolog->getHandlers() as $handler) {
        if (! $handler instanceof FormattableHandlerInterface) {
            continue;
        }

        $handler->setFormatter($formatter);
    }
}
```

`$logger->getLogger()` permite acceder al logger interno que Laravel preparó. La comprobación `instanceof MonologLogger` evita utilizar métodos específicos de Monolog si en algún momento el logger interno fuera diferente. Después se crea un `LineFormatter`, es decir, un formatter que construye cada entrada como texto siguiendo una plantilla.

En esa plantilla, `%datetime%` se reemplaza por la fecha y hora, `%level_name%` por el nivel —por ejemplo `ERROR`— y `%message%` por el mensaje que ya preparó TramaLog, incluido el JSON de contexto cuando existe. El `foreach` recorre los handlers de Monolog. Sólo a los que implementan `FormattableHandlerInterface` se les puede asignar un formatter mediante `setFormatter()`; los demás se dejan sin cambios.

Por eso TramaLog, TramaLogFormatter y Monolog no hacen lo mismo: TramaLog ofrece a TRAMA una forma única de registrar errores; Laravel selecciona y configura el canal; Monolog es el motor de logging utilizado por Laravel; el handler escribe el registro en su destino; y TramaLogFormatter personaliza cómo se verá la entrada final.

Recorrido completo: desde la excepción hasta el archivo

```

Controller / Service
↓
try
↓
se produce una excepción
↓
catch (Throwable $exception)
↓
OperationFailureHandler::fail()
↓
TramaLog::error()
↓
Log de Laravel
↓
canal trama_unexpected
↓
config/logging.php
↓
tap ejecuta TramaLogFormatter
↓
TramaLogFormatter asigna LineFormatter
↓
Monolog + handler diario
↓
el handler formatea y escribe
↓
storage/logs/trama-backend-AAAA-MM-DD.log
↓
OperationFailureHandler genera el mensaje seguro
↓
interfaz

```

El esquema muestra el recorrido utilizado cuando un controlador captura un fallo que puede convertirse en una respuesta normal para la interfaz. TramaLogFormatter interviene en la preparación del logger y modifica la presentación que utilizará el handler; no recibe directamente la excepción del controlador.

Ejemplo 2: un registro real producido durante Artisan

```

[2026-09-15 10:03:04] ERROR: Unable to locate file in Vite manifest: resources/js/Pages/Trama.vue. (View: C:\Users\PC\Desktop\project\TRAMA\resources\views\app.blade.php)
{
  "controller": "Console",
  "method": "artisan",
  "status": 500
}

```

Ejemplo real de un error registrado durante una ejecución local de TRAMA.

La captura muestra un registro real del proyecto. La primera línea conserva la fecha y hora, el nivel ERROR y el mensaje técnico: Vite no pudo localizar resources/js/Pages/Trama.vue dentro del manifest. Debajo aparece contexto estructurado con controller=Console, method=artisan y status=500.

Este segundo ejemplo no corresponde al try/catch de Contacto mostrado al comienzo. El error se produjo durante una ejecución de Artisan y llegó al manejo global de Laravel. Sirve para mostrar que distintas partes del proyecto pueden utilizar la misma infraestructura de logging y terminar en el mismo canal diario.

Los errores no capturados también pueden llegar a TramaLog

OperationFailureHandler se utiliza cuando un controlador o servicio puede capturar un fallo y devolver un mensaje seguro. Sin embargo, algunas excepciones ocurren fuera de esos try/catch, por ejemplo durante Artisan o antes de llegar a un controlador. bootstrap/app.php registra también esos errores internos no esperados y agrega contexto general antes de llamar a TramaLog.

```

if (app()->runningInConsole()) {
    $controller = 'Console';
    $method = 'artisan';
}

TramaLog::error($logMessage, [
    'controller' => $controller,
    'method' => $method,
    'status' => $isDatabaseUnavailable ? 503 : 500,
    'user_id' => optional($request->user())->id,
    'email' => $request->input('email'),
    'slug' => $request->route('slug') ?? $request->input('slug'),
    'article_id' => $articleId ?? $request->input('article_id'),
]);

```

Ese código explica por qué la captura anterior contiene Console, artisan y 500 aunque el error no haya pasado por `OperationFailureHandler`. El punto de entrada cambia, pero TramaLog vuelve a seleccionar el mismo canal y el registro termina usando la misma configuración y el mismo formato.

Errores de correo: registrar no siempre significa deshacer

El manejo de un fallo depende de qué parte de la operación ya quedó confirmada. En Contacto, MySQL es la fuente de verdad: primero se guarda `contact_messages` y después se intenta enviar la notificación por SMTP. Si el correo falla, la consulta no se elimina porque ya fue recibida correctamente; el proyecto registra el problema de notificación y permite que el equipo recupere el mensaje desde la base.

```

try {
    Mail::to((string) config('mail.contact_address'))
        ->send(new ContactMessageReceivedMail($contactMessage));
} catch (Throwable $exception) {
    TramaLog::error(TramaLog::mailFailureMessage($exception), [
        'controller' => self::class,
        'method' => 'storeContact',
        'contact_message_id' => $contactMessage->id,
        'recipient' => config('mail.contact_address'),
        'sender_email' => $contactMessage->email,
    ]);
}

```

Aquí el controlador llama directamente a TramaLog porque no necesita convertir el fallo en un error para el formulario: la consulta ya quedó guardada. `mailFailureMessage()` reduce determinados mensajes extensos del mailer a textos técnicos más cortos, como SMTP Error: Could not authenticate. o SMTP Error: Could not connect to SMTP host.

En el registro público la política es distinta: si falla el correo de verificación, `RegisteredUserController` registra el problema y elimina la cuenta recién creada porque el alta no puede completarse. La infraestructura de logging es la misma, pero la decisión de recuperación depende del flujo.

Qué pasa si MySQL deja de responder

Una caída completa de MySQL necesita un tratamiento global porque puede ocurrir incluso antes de poder leer la sesión, obtener el usuario o preparar una página normal de Inertia. Por eso `OperationFailureHandler` no la transforma en un mensaje de formulario: vuelve a lanzar esa `QueryException` y `bootstrap/app.php` decide la respuesta final.

```

if ($request->header('X-Inertia')) {
    $location = $request->headers->get('Referer') ? url('/');
    return Inertia::location($location);
}

if ($request->expectsJson()) {
    return response()->json([
        'success' => false,
        'code' => 'DATABASE_UNAVAILABLE',
        'message' => 'La página no está disponible en este momento.',
    ], 503);
}

if (view()->exists('errors.database')) {
    return response()->view('errors.database', [], 503);
}

```

Una navegación Inertia fuerza una carga completa para salir del flujo interno de Vue. Una petición que espera JSON recibe HTTP 503 y una estructura segura. Una navegación normal recibe `resources/views/errors/database.blade.php`, una página Blade independiente preparada para mostrarse sin volver a consultar MySQL.



Página mostrada cuando TRAMA no puede acceder a MySQL.

Tipo de petición	Respuesta
Navegación normal	HTTP 503 y la página <code>errors/database.blade.php</code> .
Navegación realizada por Inertia	<code>Inertia::location()</code> fuerza una carga completa; si MySQL sigue caído, esa nueva carga termina en la página de contingencia.
Petición que espera JSON	HTTP 503 con <code>code=DATABASE_UNAVAILABLE</code> y un mensaje público sin detalles de MySQL.

Error.vue y otros errores HTTP

Cuando Laravel todavía puede construir una respuesta normal de Inertia, `resources/js/Pages/Error.vue` presenta errores como 403, 404, 419, 500 y algunos 503 con textos comprensibles. Un 403 indica falta de permisos; un 404, que el recurso no existe; un 419 puede indicar que la sesión venció; y un 500 representa un error interno que la interfaz no debe detallar técnicamente.

Regla general

El usuario debe recibir información suficiente para corregir el formulario, volver a intentar o continuar. Los detalles necesarios para diagnosticar una falla —clase de excepción, mensaje técnico, archivo, línea, controlador, operación y otros datos de contexto— quedan del lado del servidor. TRAMA separa así dos responsabilidades: la interfaz comunica un problema de forma segura y el sistema de logging conserva la información que necesita quien mantiene la aplicación.

Paso a paso: Candela encuentra un error y Carlos investiga lo que TRAMA registró

No todos los errores significan lo mismo. Este recorrido diferencia un dato inválido, una prohibición y una excepción inesperada.

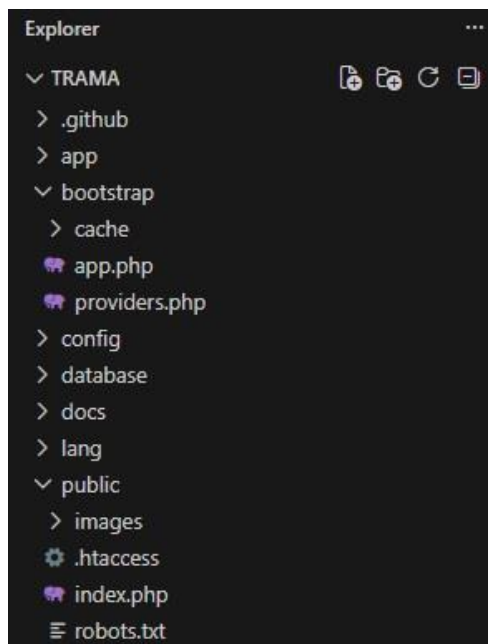
Paso	Qué hacen Candela, Valentina y Carlos y qué ocurre internamente
1	Candela envía un formulario con un dato inválido. Un Form Request detecta el problema antes de ejecutar la operación y devuelve errores de validación a la misma pantalla.
2	Valentina intenta una acción para la que no tiene permiso. Un middleware o una autorización detiene la petición antes de modificar datos.
3	En otro caso, una operación válida entra al controlador pero ocurre una excepción inesperada al acceder a una dependencia o proceso interno.
4	OperationFailureHandler y TramaLog permiten registrar contexto técnico útil para diagnosticar el fallo sin mostrarlo directamente al usuario.
5	Monolog envía el registro al canal configurado y TramaLogFormatter determina la presentación final de cada línea.
6	La interfaz recibe una respuesta o página de contingencia adecuada, por ejemplo un error HTTP controlado o una pantalla específica si la base de datos no está disponible.
7	Carlos revisa storage/logs y el contexto registrado para investigar la causa real, mientras el usuario sólo ve la información necesaria para continuar o volver a intentar.

30. Instalación local de TRAMA

Esta sección parte del escenario real de un desarrollador que clona o descarga TRAMA desde GitHub en una computadora nueva. El repositorio contiene el código fuente y los archivos que permiten reconstruir la instalación, pero no incluye las dependencias instaladas, el archivo .env ni los artefactos generados por Vite o SSR. El objetivo es terminar con TRAMA configurado, conectado a MySQL, con sus datos de prueba y con el bundle SSR preparado. La puesta en marcha diaria se explica por separado en la sección 31.

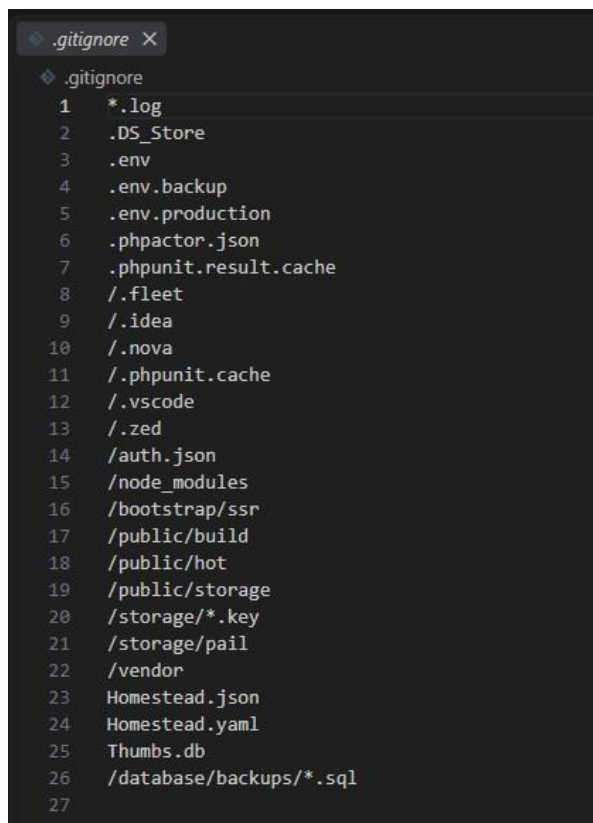
30.1 Qué llega desde GitHub y qué se reconstruye

No hay que crear un proyecto Laravel nuevo ni un proyecto Vue nuevo. app/, config/, database/, resources/, routes/, scripts/ y el resto del código ya forman parte del repositorio. Lo que se reconstruye en cada computadora son las dependencias instaladas y la configuración local.



TRAMA recién descargado desde GitHub, antes de instalar dependencias o generar archivos locales.

Elemento	¿Se espera en GitHub?	Qué ocurre en una instalación nueva
app/, config/, database/, resources/, routes/, scripts/	Sí	Contienen el código fuente y ya existen al clonar o descargar TRAMA.
composer.json + composer.lock	Sí	Describen y fijan el árbol de dependencias PHP que Composer utilizará para reconstruir vendor/.
package.json + package-lock.json	Sí	Describen y fijan el árbol de dependencias JavaScript que NPM utilizará para reconstruir node_modules/.
vendor/	No	Se crea localmente con composer install.
node_modules/	No	Se crea localmente con npm install.
.env.example	Sí	Es la plantilla de variables necesarias.
.env	No	Se crea localmente y puede contener claves, contraseñas y rutas propias del equipo.
bootstrap/ssr/	No	Contiene el bundle SSR generado; se crea al compilar SSR.
public/build/	No	Contiene el build de cliente de producción generado por Vite.



```

.gitignore
1 *.log
2 .DS_Store
3 .env
4 .env.backup
5 .env.production
6 .phpactor.json
7 .phpunit.result.cache
8 /.fleet
9 /.idea
10 /.nova
11 /.phpunit.cache
12 /.vscode
13 /.zed
14 /auth.json
15 /node_modules
16 /bootstrap/ssr
17 /public/build
18 /public/hot
19 /public/storage
20 /storage/*.key
21 /storage/pail
22 /vendor
23 Homestead.json
24 Homestead.yaml
25 Thumbs.db
26 /database/backups/*.sql
27

```

El archivo .gitignore confirma qué dependencias, secretos y artefactos generados no se versionan.

No subir vendor ni node_modules a GitHub

Composer y NPM pueden reconstruir esas carpetas a partir de composer.lock y package-lock.json. Tampoco debe versionarse .env, y los builds generados en bootstrap/ssr y public/build permanecen fuera del repositorio.

30.2 Programas y servicios necesarios

Componente	Uso en TRAMA
Visual Studio Code	Editor utilizado en esta guía para abrir el proyecto y trabajar con terminales integradas.
PHP 8.5.8 recomendado	composer.json declara ^8.3, pero el composer.lock actual contiene dependencias que requieren PHP 8.4.1 o superior. El entorno documentado fue verificado con PHP 8.5.8.
Composer	Instala Laravel y las dependencias PHP declaradas en composer.json y composer.lock.
Node.js + NPM	Instalan Vue, Vite, Sharp, ONNX Runtime y las demás dependencias JavaScript; Node también ejecuta SSR y scripts auxiliares.
MySQL	Servidor de base de datos utilizado por TRAMA.
MySQL Workbench	Cliente gráfico utilizado en esta guía para crear y comprobar la base.
resources/models	Ubicación prevista para los modelos y archivos auxiliares necesarios para la moderación automática y la clasificación local de imágenes.

Antes de instalar dependencias, comprobar qué PHP está utilizando realmente la terminal:

```
php -v
composer --version
node --version
npm --version
```

```
PC@DESKTOP-0JH6AUJ MINGW64 ~/Desktop/project/TRAMA
$ php -v
PHP 8.5.8 (cli) (built: Jul 1 2026 04:03:04) (NTS Visual C++ 2022 x64)
Copyright (c) The PHP Group
Built by The PHP Group
Zend Engine v4.5.8, Copyright (c) Zend Technologies
with Zend OPcache v8.5.8, Copyright (c), by Zend Technologies
```

Comprobación de PHP 8.5.8 en la terminal utilizada para instalar TRAMA.

No ejecutar composer update para resolver una versión de PHP incorrecta

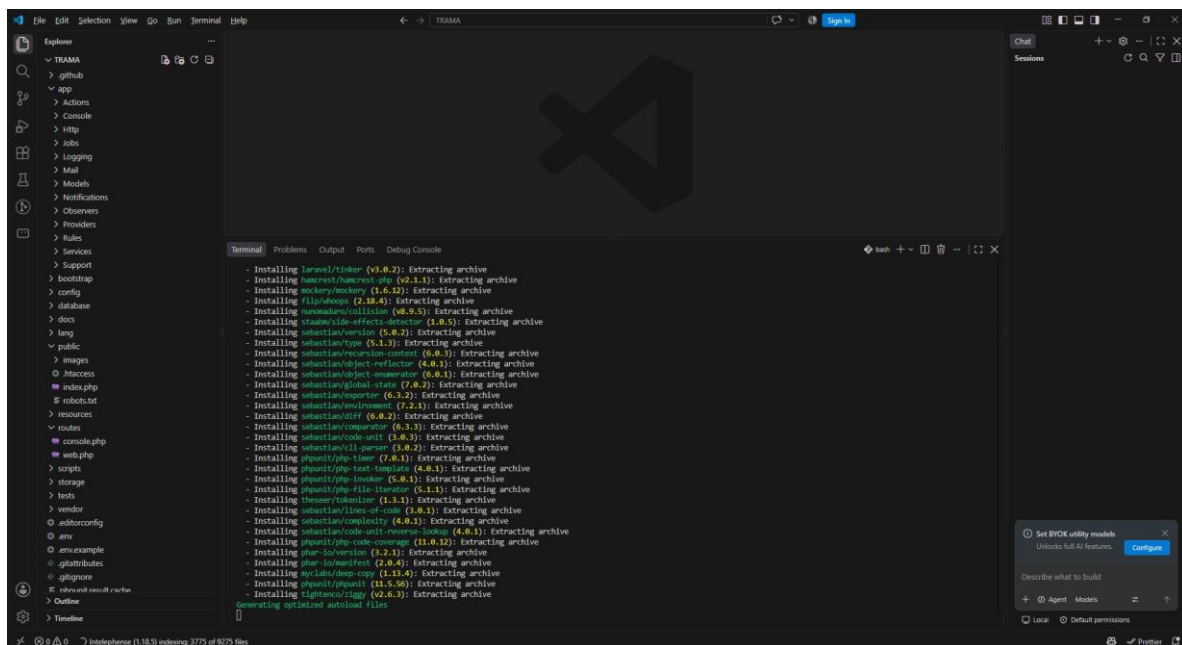
Si Composer informa que la plataforma no cumple los requisitos, primero debe corregirse la versión de PHP o el PATH de la terminal. `composer update` cambiaría versiones del árbol de dependencias y no es el arreglo para este caso.

30.3 Instalar las dependencias PHP

Abrir una terminal en la raíz de TRAMA, la carpeta que contiene artisan, composer.json y package.json, y ejecutar:

```
composer install
```

Composer lee `composer.json` y `composer.lock`, descarga las versiones fijadas y crea `vendor/`. Esa carpeta contiene Laravel y las librerías PHP instaladas; por eso no necesita estar en GitHub.



Composer instalando las dependencias PHP del proyecto; vendor/ se reconstruye localmente.

30.4 Instalar las dependencias JavaScript y de Node.js

Desde la misma raíz del proyecto ejecutar:

```
npm install
```

NPM lee package.json y package-lock.json y crea node_modules/. Allí quedan Vite, Vue, Sharp, ONNX Runtime y las demás dependencias que utilizan el frontend y los procesos de Node.js. npm install prepara la instalación; no es el comando que mantiene Vite ejecutándose durante el desarrollo.

```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ npm install
npm warn deprecated boolean@3.2.0: Package no longer supported.

added 247 packages, and audited 248 packages in 56s

68 packages are looking for funding
  run `npm fund` for details

4 vulnerabilities (2 moderate, 2 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

Resultado de npm install: las dependencias de Node.js quedaron instaladas en la copia local.

Advertencias de npm

NPM puede informar paquetes deprecated o vulnerabilidades conocidas. Ese informe no significa por sí solo que la instalación haya fallado: hay que distinguirlo de un error que interrumpe el comando. Cualquier actualización de dependencias debe evaluarse por separado para no cambiar el proyecto sin control.

30.5 Crear .env y generar APP_KEY

.env.example sí forma parte del repositorio y sirve como plantilla. Este archivo es el que se utiliza realmente en esa instalación y no se versiona. Puede crearse con el comando correspondiente a la terminal:

Terminal	Crear .env desde .env.example
Git Bash	cp .env.example .env
PowerShell	Copy-Item .env.example .env
CMD	copy .env.example .env

```

1 APP_NAME=TRAMA
2 APP_ENV=local
3 APP_KEY=
4 APP_DEBUG=true
5 APP_URL=http://127.0.0.1:8000
6 APP_TIMEZONE=America/Argentina/Buenos_Aires
7
8 # Fecha fija de referencia para que campañas, métricas y reportes no
9 TRAMA_REFERENCE_DATE=2026-07-19
10 # Hora inicial del reloj editorial demo. Solo se usa como punto de pa
11 # después TRAMA continúa desde el último instante guardado entre sesi
12 TRAMA_REFERENCE_TIME=15:30:00

```

.env.example contiene los nombres y valores de referencia; APP_KEY aparece vacío hasta generar la clave local.

Después de crear .env, generar la clave criptográfica de Laravel:

```
php artisan key:generate
```

key:generate crea APP_KEY y la escribe en .env. El archivo .env puede contener credenciales de MySQL, correo, rutas y otras configuraciones privadas; no debe subirse a GitHub ni mostrarse con secretos reales en capturas públicas.

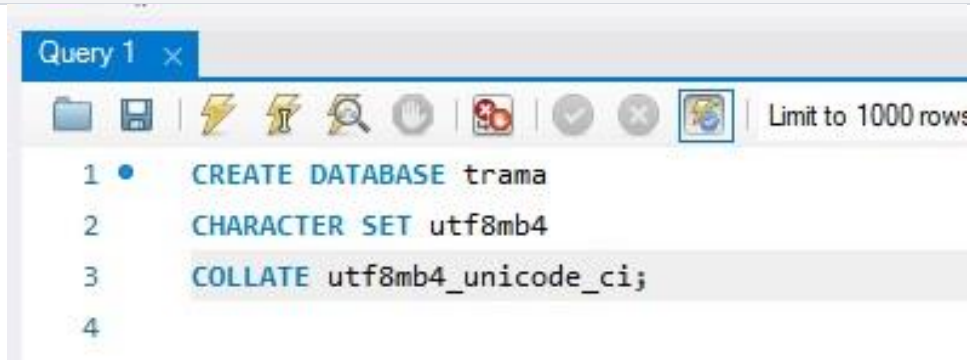
30.6 Crear la base trama y configurar MySQL

Iniciar el servidor MySQL que utilizará TRAMA y abrir MySQL Workbench. En una pestaña SQL crear una base vacía llamada trama:

```

CREATE DATABASE trama
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

```



Creación de la base de datos trama desde MySQL Workbench.

Después completar en .env los datos reales de esa conexión. DB_DATABASE debe apuntar a trama; el puerto, usuario y contraseña dependen del servidor MySQL de cada equipo.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=<puerto-mysql>
DB_DATABASE=trama
DB_USERNAME=<usuario-mysql>
DB_PASSWORD=<contraseña-mysql>
```

No copiar credenciales de otro equipo

DB_PORT, DB_USERNAME y DB_PASSWORD deben corresponder a la instalación local. La documentación utiliza marcadores deliberadamente y no publica contraseñas reales.

30.7 Crear las tablas con las migraciones

Con la base vacía creada y .env configurado, Laravel puede construir el esquema actual:

```
php artisan migrate
```

Las migraciones crean tablas, columnas, índices, claves foráneas y las tablas internas de Laravel. Al terminar puede actualizarse SCHEMAS > trama > Tables en Workbench.



Tablas de TRAMA creadas en MySQL después de ejecutar las migraciones.

30.8 Cargar los datos de prueba

Las migraciones crean la estructura, pero no cargan los datos de prueba. Para cargar empleados, usuarios registrados, noticias, comentarios, revisiones, publicidad y los demás datos preparados por TRAMA, ejecutar:

```
php artisan db:seed
```

DatabaseSeeder coordina los seeders del proyecto y TramaIntegritySeeder comprueba varias cantidades y reglas de coherencia del escenario final. En una carga limpia, algunos valores útiles para comprobar rápidamente el resultado son:

Elemento	Resultado esperado
Usuarios	2.506 (2.500 usuarios registrados + 6 empleados)
Noticias	90
Comentarios y respuestas	1.395
Publicidades	15
Vistas de noticias	64.212
Impresiones / clics de banners	35.298 / 145

Cuando se quiere destruir deliberadamente un entorno local y reconstruirlo completo en una sola operación puede utilizarse:

```
php artisan migrate:fresh --seed
```

Atención con migrate:fresh

migrate:fresh elimina todas las tablas de la base configurada antes de volver a ejecutar las migraciones y los seeders. No debe utilizarse sobre una base cuyos datos deban conservarse.

30.9 Preparar el bundle SSR inicial

El repositorio no incluye bootstrap/ssr porque esa carpeta está excluida por .gitignore. Sin embargo, php artisan inertia:start-ssr necesita que exista el bundle SSR compilado. En una copia limpia, después de npm install, generar ese bundle una vez con:

```
npm run build:ssr
```

Este comando compila resources/js/ssr.js y normalmente genera bootstrap/ssr/ssr.js. No inicia el servidor SSR. También puede generarse como parte de npm run build, que construye cliente y SSR; para la preparación local inicial basta con build:ssr si sólo falta el bundle del servidor.

El comando que activa SSR es otro

npm run build:ssr genera el archivo que Node ejecutará. El servidor SSR se pone en marcha durante el uso de TRAMA con php artisan inertia:start-ssr, explicado en la sección 31.

30.10 Comprobación de una instalación terminada

Elemento	Debe existir o responder
Dependencias PHP	vendor/
Dependencias Node	node_modules/
Configuración local	.env con APP_KEY y DB_* correctos
Base MySQL	trama con sus tablas
Datos de prueba	Seeders ejecutados sin errores
Bundle SSR	bootstrap/ssr/ssr.js

Al llegar a este punto TRAMA está instalado. `composer install`, `npm install`, las migraciones y los seeders no se repiten cada vez que se abre el proyecto. La sección siguiente explica únicamente qué procesos deben ponerse en marcha durante el trabajo diario.

Paso a paso: Carlos instala TRAMA desde una copia limpia

Esta secuencia reúne en un único recorrido lo que ocurre la primera vez que el proyecto se prepara en una PC nueva.

Paso	Qué hace la persona y qué ocurre internamente
1	Carlos descarga o clona el repositorio y comprueba que tiene PHP, Composer, Node.js/NPM y MySQL disponibles.
2	Ejecuta <code>composer install</code> . Composer reconstruye <code>vendor/</code> a partir de <code>composer.lock</code> .
3	Ejecuta <code>npm install</code> . NPM reconstruye <code>node_modules/</code> a partir de <code>package.json</code> y <code>package-lock.json</code> .
4	Copia <code>.env.example</code> como <code>.env</code> y genera <code>APP_KEY</code> con <code>php artisan key:generate</code> .
5	Crea manualmente la base MySQL <code>trama</code> y completa en <code>.env</code> <code>host</code> , <code>puerto</code> , <code>usuario</code> , <code>contraseña</code> y <code>DB_DATABASE</code> .
6	Ejecuta <code>php artisan migrate</code> . Laravel crea tablas, índices, claves foráneas y estructuras internas.
7	Ejecuta <code>php artisan db:seed</code> para cargar el escenario de datos de prueba y permitir que <code>TramaIntegritySeeder</code> verifique su coherencia.
8	Genera el bundle SSR inicial con <code>npm run build:ssr</code> para crear <code>bootstrap/ssr/ssr.js</code> , que no viene versionado en el repositorio.
9	Comprueba que <code>storage</code> y las carpetas administradas por la aplicación puedan escribirse y que las dependencias externas necesarias estén disponibles.
10	A partir de este punto la instalación inicial termina. En los días siguientes no repite estos pasos completos: sólo inicia los procesos de desarrollo del capítulo 31.

31. Cómo ejecutar TRAMA en desarrollo

Una vez completada la instalación local de la sección 30, TRAMA ya tiene sus dependencias, configuración, base de datos, datos de prueba y bundle SSR. A partir de aquí se distingue la preparación inicial de la puesta en marcha diaria: normalmente sólo hay que iniciar los procesos que mantienen la aplicación funcionando mientras se trabaja.

31.1 Qué procesos necesita TRAMA

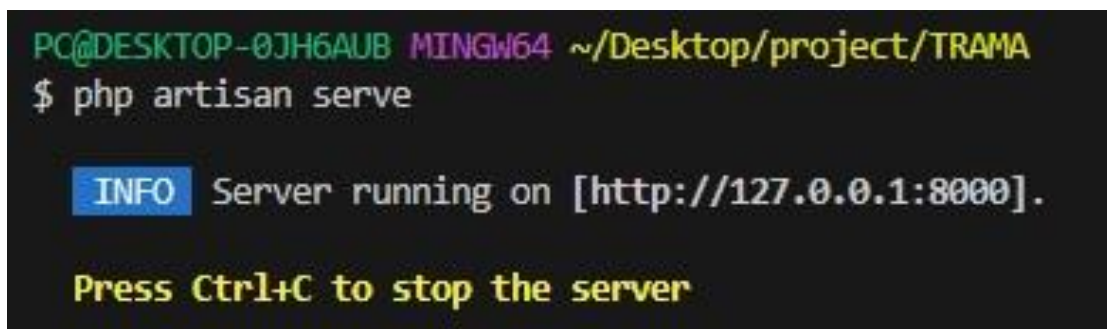
Proceso	Responsabilidad en desarrollo
Laravel	Atiende las peticiones HTTP mediante <code>php artisan serve</code> .
Vite	Sirve JavaScript y CSS de desarrollo mediante <code>npm run dev</code> .
SSR	Genera el HTML inicial de <code>Public/Home</code> mediante <code>php artisan inertia:start-ssr</code> .
Queue worker	Procesa Jobs de <code>moderation</code> y <code>default</code> .
Scheduler	Revisa las tareas programadas mediante <code>php artisan schedule:work</code> .

Estos procesos no son servidores duplicados. Cada uno cubre una responsabilidad diferente y puede estar funcionando mientras otro falla. Por eso, para aprender y diagnosticar el proyecto, esta guía los inicia en terminales separadas.

31.2 Terminal 1 - Servidor Laravel

```
php artisan serve
```

Laravel inicia su servidor de desarrollo y muestra la dirección HTTP, normalmente `http://127.0.0.1:8000`. Esta terminal debe permanecer abierta.



```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php artisan serve

INFO Server running on [http://127.0.0.1:8000].

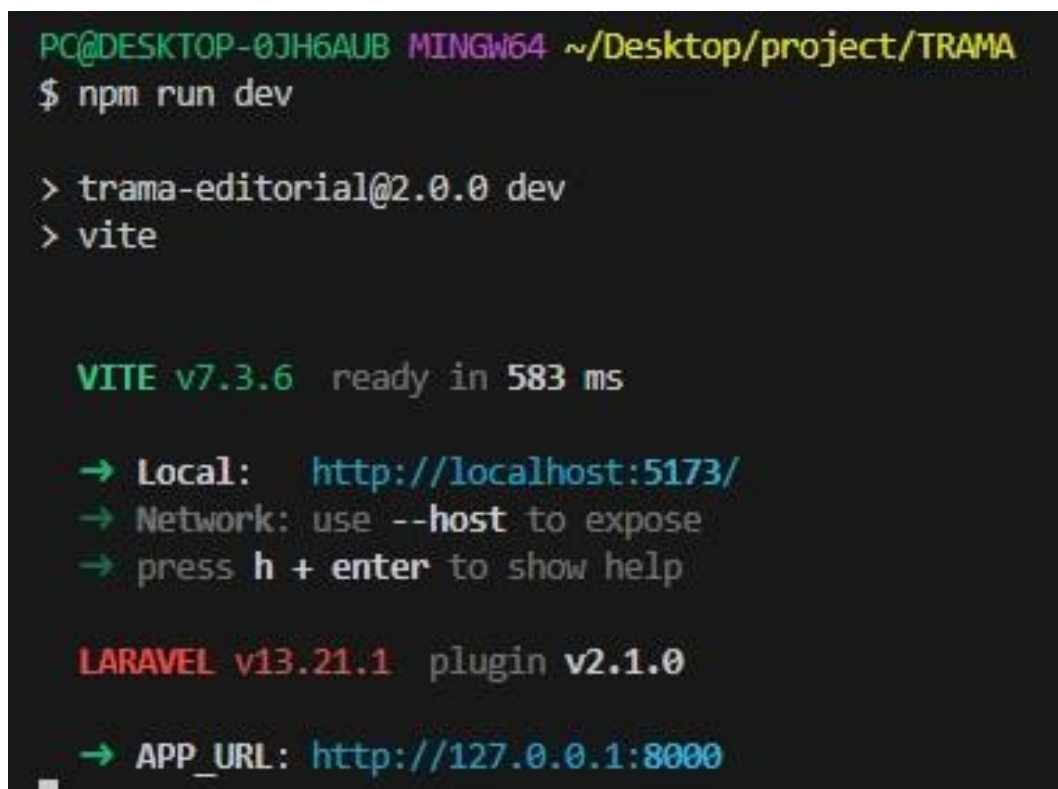
Press Ctrl+C to stop the server
```

Servidor de desarrollo de Laravel activo en `http://127.0.0.1:8000`.

31.3 Terminal 2 - Vite

```
npm run dev
```

Vite sirve los recursos del frontend y mantiene el modo de desarrollo de Vue y CSS. El puerto 5173 pertenece a Vite; el sitio de TRAMA se abre mediante la URL de Laravel, no navegando directamente a Vite.



```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ npm run dev

> trama-editorial@2.0.0 dev
> vite

VITE v7.3.6 ready in 583 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

LARAVEL v13.21.1 plugin v2.1.0

→ APP_URL: http://127.0.0.1:8000
```

Vite activo y conectado con la aplicación Laravel configurada en `APP_URL`.

31.4 Terminal 3 - Servidor SSR

```
php artisan inertia:start-ssr
```

Este es el comando que inicia realmente SSR. `INERTIA_SSR_ENABLED=true` sólo habilita la posibilidad de usarlo; no inicia Node por sí solo. El proceso utiliza `bootstrap/ssr/ssr.js` y debe permanecer abierto mientras se quiera conservar el pre-renderizado de `Public/Home`. Si se modifica código que participa en SSR, hay que ejecutar `npm run build:ssr` y reiniciar este proceso; `npm run dev` no recompila automáticamente el bundle SSR.



```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php artisan inertia:start-ssr
Starting SSR server on port 13714...
Inertia SSR server started.
```

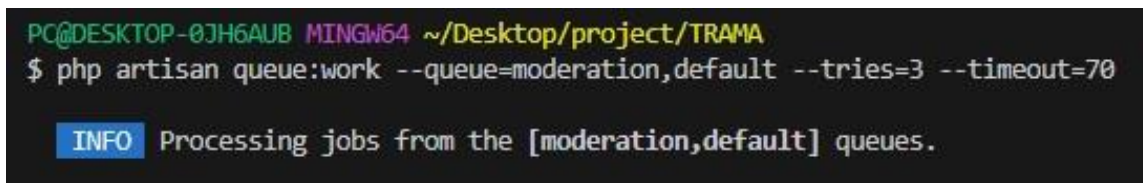
Servidor SSR de Inertia iniciado correctamente en el puerto 13714.

Los alias definidos en `composer.json` permiten ejecutar `composer ssr:start`, `composer ssr:check` y `composer ssr:stop`, pero `php artisan inertia:start-ssr` deja explícito qué proceso se está iniciando.

31.5 Terminal 4 - Worker de colas

```
php artisan queue:work --queue=moderation,default --tries=3 --timeout=70
```

El worker toma los Jobs pendientes. En particular, cuando un usuario envía un comentario, `ModerateComment` queda en la cola `moderation` y no ejecuta el pipeline de Node hasta que un worker lo consume.



```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php artisan queue:work --queue=moderation,default --tries=3 --timeout=70

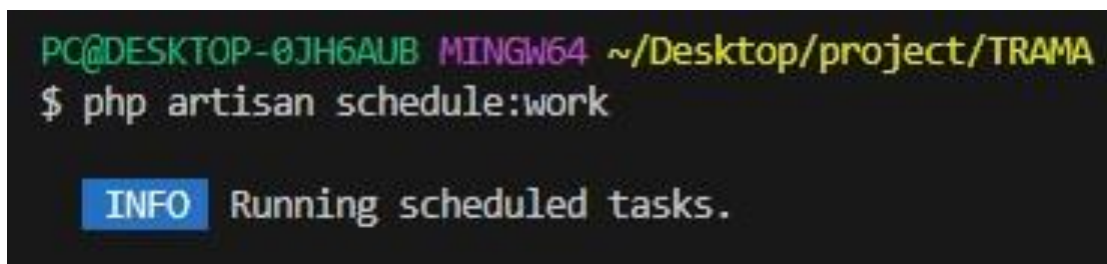
INFO Processing jobs from the [moderation,default] queues.
```

Worker de Laravel procesando las colas moderation y default.

31.6 Terminal 5 - Scheduler local

```
php artisan schedule:work
```

`schedule:work` mantiene activo el Scheduler durante el desarrollo. En TRAMA permite que Laravel revise cada minuto tasks como `articles:publish-scheduled`. Scheduler y `TramaClock` son mecanismos distintos: uno dispara tareas; el otro define la fecha editorial del escenario.



```
PC@DESKTOP-0JH6AUB MINGW64 ~/Desktop/project/TRAMA
$ php artisan schedule:work

INFO Running scheduled tasks.
```

Scheduler local activo y revisando las tareas programadas.

31.7 Abrir TRAMA en el navegador

Con Laravel y Vite activos ya puede abrirse la aplicación. Para probar además SSR, moderación automática y publicaciones programadas deben estar activos sus procesos correspondientes. La URL principal del entorno documentado es:

```
http://127.0.0.1:8000
```



Portada de TRAMA funcionando en el entorno local de desarrollo.

31.8 Paso a paso: Carlos pone TRAMA en marcha cada día

Paso	Qué hace Carlos
1	Carlos abre la raíz de TRAMA y comprueba que el servidor MySQL configurado esté activo.
2	Carlos abre la Terminal 1 y ejecuta <code>php artisan serve</code> .
3	Carlos abre la Terminal 2 y ejecuta <code>npm run dev</code> .
4	Carlos abre la Terminal 3 y ejecuta <code>php artisan inertia:start-ssr</code> .
5	Carlos abre la Terminal 4 y ejecuta <code>php artisan queue:work --queue=moderation,default --tries=3 --timeout=70</code> .
6	Carlos abre la Terminal 5 y ejecuta <code>php artisan schedule:work</code> .
7	Con los procesos necesarios activos, Carlos abre <code>http://127.0.0.1:8000</code> en el navegador.

31.9 Inicio integrado con Composer

TRAMA también incluye en el archivo `composer.json` un script de desarrollo que permite iniciar en una sola ejecución los procesos necesarios para trabajar con la aplicación, evitando tener que abrir y ejecutar manualmente las cinco terminales descritas en la subsección anterior.

```
composer run dev
```

Antes de iniciar los procesos, el script ejecuta:

```
npm run build:ssr
```

para generar el bundle utilizado por Inertia SSR. Después inicia simultáneamente:

Proceso	Comando
Laravel	php artisan serve
Cola trabajos	php artisan queue:listen -- queue=moderation,default -- tries=3 --timeout=70
Vite	npm run dev
Inertia SSR	php artisan inertia:start-ssr
Scheduler	php artisan schedule:work

Este inicio integrado es la opción más cómoda para el trabajo diario. La ejecución manual explicada en las subsecciones anteriores sigue siendo útil cuando se necesita iniciar, detener o diagnosticar cada proceso por separado.

31.10 Diagnóstico rápido

Prueba	Qué confirma	Si falla, revisar primero
Abrir <code>http://127.0.0.1:8000</code>	Laravel responde y la aplicación puede atender HTTP.	<code>php artisan serve</code> , <code>.env</code> y conexión MySQL.
La interfaz carga estilos y JavaScript	Vite está sirviendo los recursos de desarrollo.	<code>npm run dev</code> y consola de Vite.
<code>php artisan inertia:check-ssr</code>	El servidor SSR responde.	Bundle <code>bootstrap/ssr/ssr.js</code> e <code>inertia:start-ssr</code> .
Enviar un comentario	El Job entra a moderation y puede ser procesado.	<code>queue:work</code> , <code>TRAMA_NODE_BINARY</code> y modelos.
Programar una noticia cercana	El Scheduler puede ejecutar la publicación programada.	<code>schedule:work</code> y <code>routes/console.php</code> .

32. Cómo publicar TRAMA en un servidor

Publicar TRAMA significa pasar del entorno local de desarrollo a un servidor accesible desde Internet sin exponer archivos privados y manteniendo activos los procesos que la aplicación necesita. Esta documentación no presupone un proveedor concreto: describe los requisitos que cualquier despliegue debe resolver.

32.1 Qué debe ofrecer el servidor

Requisito	Por qué es necesario
PHP compatible	El <code>composer.lock</code> actual requiere al menos PHP 8.4.1; el entorno documentado fue verificado con PHP 8.5.8.

Requisito	Por qué es necesario
Composer	Instala las dependencias PHP de producción.
Node.js + NPM	Generan el build y siguen siendo necesarios en ejecución para SSR, Sharp, ONNX Runtime y moderación.
MySQL	Almacena los datos permanentes.
Servidor HTTP con HTTPS	Debe publicar únicamente public/ como raíz web.
Procesos persistentes	Queue worker y SSR deben mantenerse activos y reiniciarse si se detienen.
Programador de tareas	Debe invocar php artisan schedule:run cada minuto.
SMTP y escritura en storage/ y public/images/	Necesarios para correo, logs y archivos persistentes generados o reemplazados por el CMS.

32.2 Preparar .env de producción

La instancia de producción necesita su propio .env. APP_ENV debe ser production, APP_DEBUG debe permanecer false y APP_KEY debe contener una clave válida. Las credenciales reales y la clave de la aplicación no se publican en el repositorio.

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://<dominio-real>
APP_KEY=

DB_CONNECTION=mysql
DB_HOST=<host-mysql>
DB_PORT=<puerto-mysql>
DB_DATABASE=<base-produccion>
DB_USERNAME=<usuario>
DB_PASSWORD=<contraseña>
```

En una instalación nueva, después de crear .env, debe ejecutarse php artisan key:generate una sola vez para completar APP_KEY. No debe regenerarse una clave existente en una producción ya operativa salvo que se comprenda el impacto sobre datos cifrados, cookies y sesiones. También deben configurarse SMTP, zona horaria, TRAMA_NODE_BINARY y las variables INERTIA_SSR_* de acuerdo con el servidor real.

32.3 Instalar dependencias de producción

```
composer install --no-dev --optimize-autoloader
npm ci
```

Composer instala sólo las dependencias PHP necesarias para ejecutar la aplicación y optimiza el autoloader. npm ci usa package-lock.json para reconstruir de forma reproducible el árbol JavaScript. En la estructura actual de TRAMA, Vue, Inertia, Axios y otras piezas que el bundle SSR importa están en devDependencies; por eso no debe usarse npm ci --omit=dev ni eliminarse devDependencies después del build mientras el SSR dependa de ellas.

32.4 Generar cliente y bundle SSR

```
npm run build
```

En TRAMA, el script build ejecuta el build del cliente y después el build SSR. El resultado incluye public/build/ para los recursos del navegador y bootstrap/ssr/ssr.js para el proceso SSR. A diferencia del desarrollo, Vite no necesita permanecer ejecutándose después de generar los recursos de producción.

32.5 Aplicar migraciones sin destruir datos

```
php artisan migrate --force
```

--force permite ejecutar las migraciones en un entorno marcado como producción. Este comando actualiza la estructura existente; no debe sustituirse por migrate:fresh cuando la base contiene información que debe conservarse. Los seeders de prueba tampoco deben ejecutarse automáticamente sobre una base real salvo que exista una decisión explícita de poblarla con datos de prueba.

32.6 Configurar public/ como raíz web

El proyecto completo debe existir en el servidor porque Laravel necesita app/, vendor/, config/, resources/, storage/ y las demás carpetas internas. Lo que debe quedar expuesto por HTTP es únicamente public/. En Apache, Nginx o el servidor elegido, el DocumentRoot o raíz web debe apuntar a la carpeta public/ de TRAMA.

```
/ruta/al/proyecto/TRAMA/
├─ app/
├─ config/
├─ resources/
├─ storage/
├─ vendor/
├─ .env
└─ public/ ← raíz web del dominio
```

No significa copiar sólo public/ al servidor

Laravel necesita el proyecto completo. La regla es que public/ sea la única carpeta directamente accesible desde la web; .env, app/, vendor/ y resources/ deben quedar fuera de la raíz pública.

32.7 Mantener activo el worker de colas

```
php artisan queue:work --queue=moderation,default --tries=3 --timeout=70
```

En producción el worker debe mantenerse como proceso persistente mediante el mecanismo de supervisión disponible en el servidor. Si se detiene, los Jobs pueden seguir acumulándose y los comentarios pueden quedar en processing hasta que vuelva a ejecutarse.

32.8 Mantener activo el servidor SSR

```
php artisan inertia:start-ssr
```

Si se quiere conservar el SSR selectivo de Public/Home, este proceso también debe permanecer activo. El entorno de producción debe supervisarlo y reiniciarlo si termina; no debe depender de una terminal abierta manualmente.

32.9 Ejecutar el Scheduler cada minuto

```
php artisan schedule:run
```

En producción no se mantiene schedule:work como en desarrollo. El sistema operativo o la plataforma debe invocar schedule:run cada minuto. routes/console.php registra articles:publish-scheduled con everyMinute() y withoutOverlapping().

32.10 Permisos, modelos, logs, copias y HTTPS

- storage/, bootstrap/cache/ y las carpetas administradas por el CMS dentro de public/images/ —articles/, categories/, team/ y banners/— deben ser escribibles por los procesos de Laravel/Node que crean o reemplazan esos archivos.
- resources/models debe conservar los modelos ONNX y archivos auxiliares usados por comentarios e imágenes.
- TRAMA_NODE_BINARY debe apuntar a un ejecutable Node compatible disponible en el servidor.
- Los logs deben poder escribirse y conviene definir una política de rotación y diagnóstico.
- La base MySQL y los archivos persistentes, incluidas las imágenes administradas dentro de public/images/, requieren una política de copias de seguridad.

- El sitio público debe utilizar HTTPS y APP_DEBUG debe permanecer desactivado.

32.11 Comprobaciones después de publicar

Comprobación	Resultado esperado
Abrir el dominio	La portada responde mediante HTTPS.
Autenticación	Registro/login y sesiones funcionan con la base de producción.
SSR	php artisan inertia:check-ssr confirma que el servidor SSR responde.
Comentarios	Un comentario sale de processing cuando el worker procesa ModerateComment.
Programación	Una noticia programada puede publicarse mediante Scheduler.
Imágenes	Sharp/ONNX funcionan con TRAMA_NODE_BINARY y resources/models.
Correo	Verificación, recuperación y contacto utilizan el SMTP de producción.
Logs	Los errores técnicos quedan registrados sin exponerse al visitante.

Paso a paso: Carlos publica una versión de TRAMA en producción

Un despliegue correcto no consiste únicamente en copiar archivos: también debe dejar funcionando los procesos permanentes de Laravel, Node y el sistema operativo.

Paso	Qué hace la persona y qué ocurre internamente
1	Carlos copia el proyecto a un servidor que cumpla las versiones y extensiones necesarias y mantiene fuera del acceso público los archivos privados.
2	Crea el .env de producción con APP_ENV=production, APP_DEBUG=false, una APP_KEY válida y credenciales reales de MySQL, correo, Node y SSR.
3	Ejecuta composer install con las opciones de producción y npm ci para reconstruir exactamente las dependencias JavaScript del lockfile.
4	Ejecuta npm run build. Vite genera public/build/ y el bundle SSR que utilizará Node en producción.
5	Aplica php artisan migrate --force para actualizar la estructura sin destruir los datos existentes. No usa migrate:fresh sobre una base real.
6	Configura el servidor web para que únicamente public/ sea la raíz expuesta por HTTP.
7	Deja un worker de colas supervisado para que Jobs como ModerateComment no queden acumulados.
8	Deja activo y supervisado el servidor SSR si quiere conservar el pre-renderizado de la portada.
9	Configura una tarea del sistema que ejecute php artisan schedule:run cada minuto para las tareas periódicas.

Paso	Qué hace la persona y qué ocurre internamente
10	Comprueba permisos, HTTPS, logs, copias de seguridad, modelos locales y una navegación completa del portal antes de considerar terminado el despliegue.

33. Datos de prueba y seeders

Una instalación recién migrada tiene la estructura necesaria para funcionar, pero sus tablas pueden estar vacías. Eso obliga a crear manualmente usuarios, noticias, comentarios, estados editoriales y publicidades antes de poder recorrer el proyecto. TRAMA evita ese trabajo mediante un conjunto de datos de prueba preparado para reconstruir un escenario local completo y repetible.

TRAMA utiliza varios seeders para reconstruir automáticamente ese escenario de pruebas. En esta sección se explica primero qué es un seeder y, a partir de esa idea, cómo DatabaseSeeder coordina las clases PHP y los archivos JSON que terminan cargando los registros en MySQL.

Los seeders no crean la estructura de la base de datos. Esa tarea corresponde a las migraciones, que crean o modifican tablas, columnas, índices y relaciones. Los seeders trabajan después sobre esa estructura y cargan los registros necesarios. Por eso ejecutar las migraciones no equivale a cargar los datos de prueba: son dos pasos distintos.

Qué es un seeder y para qué sirve

Un seeder es una clase de Laravel cuya función es insertar datos automáticamente en tablas que ya existen. Su utilidad principal es evitar crear datos de forma manual y cargar de forma rápida y reproducible los registros necesarios para desarrollar, probar o demostrar una aplicación, sin tener que crearlos manualmente uno por uno.

En TRAMA, los seeders permiten volver a crear de forma controlada el conjunto de datos necesarios para probar el sistema: usuarios, empleados, categorías, etiquetas, noticias, comentarios, moderación, consultas de Contacto y publicidades. Si la base se prepara nuevamente desde cero, los mismos seeders pueden cargar otra vez esos datos automáticamente.

La idea importante es que un seeder trabaja sobre la estructura creada por las migraciones. Primero deben existir las tablas y columnas; después el seeder inserta filas dentro de ellas. En la sección 34 aparecerán las factories, que cumplen otra función: generar sólo los registros que una prueba concreta necesita.

Concepto	Para qué sirve
Migración	Crea o modifica la estructura de la base: tablas, columnas, índices y relaciones.
Seeder	Carga un conjunto de datos preparado sobre una estructura que ya existe.
Factory	Genera registros de prueba bajo demanda, normalmente para un escenario automatizado concreto.

Idea principal de esta sección

Las migraciones preparan dónde se guardará la información. Los seeders preparan qué información de prueba se guardará. TRAMA separa además los datos grandes en archivos JSON y utiliza clases PHP para decidir cómo y en qué orden llegan a MySQL.

33.1 De una base vacía a un escenario de pruebas

El comando habitual para cargar los datos sobre una base cuya estructura ya existe es `php artisan db:seed`. Laravel comienza por DatabaseSeeder, que funciona como punto de entrada de la carga. Desde allí se llaman otros seeders hasta completar usuarios, taxonomías, noticias, participación, moderación, contacto y publicidad.

```

php artisan db:seed
↓
DatabaseSeeder
↓
TramaContentSeeder + AdvertisementSeeder
↓
seeders especializados
↓
JsonSeedLoader / operaciones DB
↓
tablas MySQL
↓
TramaIntegritySeeder

```

El último paso es importante: `TramaIntegritySeeder` no agrega contenido. Revisa el resultado para detectar datos de prueba incoherentes antes de considerar terminada la carga.

33.2 Dónde están los datos: archivos JSON

JSON es un formato de texto que representa información mediante nombres de campos y valores. TRAMA lo utiliza para mantener la mayor parte de los registros de prueba separados del código PHP que organiza la carga. Esos archivos se encuentran dentro de `database/seeders/data/`.

Por ejemplo, `categories.json` contiene las categorías que después aparecerán en la tabla `categories`. Una de sus filas está representada así:

```

{
  "id": 1,
  "name": "Política",
  "slug": "politica",
  "description": "Poder, Congreso, gobiernos y decisiones públicas.",
  "accent_color": "#D8323C",
  "cover_image": "/images/categories/politica.webp",
  "sort_order": 1,
  "is_active": true,
  "created_at": "2026-06-05 08:00:00",
  "updated_at": "2026-06-05 08:00:00"
}

```

El archivo no crea por sí solo una fila en MySQL. Es solamente la representación de los datos. Una clase PHP debe leerlo, comprobar que pueda decodificarse correctamente y ejecutar la inserción correspondiente; en TRAMA esa tarea común se concentra en `JsonSeedLoader`.

33.3 DatabaseSeeder: punto de entrada y transacción

`DatabaseSeeder` es la clase que Laravel ejecuta cuando comienza la carga general. Su método `run()` no contiene miles de INSERT: delega el trabajo en otros seeders y envuelve el proceso completo en una transacción.

```

public function run(): void
{
    DB::disableQueryLog();

    DB::transaction(function (): void {
        $this->call([
            TramaContentSeeder::class,
            AdvertisementSeeder::class,
            TramaIntegritySeeder::class,
        ]);
    }, 3);
}

```

`run()` es el método que Laravel ejecuta en ese seeder. `DB::disableQueryLog()` evita conservar en memoria un registro de todas las consultas mientras se cargan miles de filas. `$this->call()` indica qué seeders deben ejecutarse a continuación.

`DB::transaction()` agrupa las operaciones de base de datos dentro de una misma unidad. Si una parte de la carga provoca una excepción antes de completarse, la transacción permite revertir sus cambios en lugar de dejar un escenario cargado sólo a medias. El segundo argumento, 3, permite reintentar la transacción hasta tres veces ante determinados conflictos de concurrencia, como un deadlock; no significa que cualquier error se ejecute tres veces.

33.4 TramaContentSeeder: por qué importa el orden

`TramaContentSeeder` organiza la parte principal del escenario. El orden no es decorativo: algunos registros necesitan que otros ya existan para poder guardar correctamente sus relaciones.

```
public function run(): void
{
    $this->call([
        TramaStaffSeeder::class,
        TramaTaxonomySeeder::class,
        TramaRegisteredUsersSeeder::class,
        TramaArticlesSeeder::class,
        TramaEditorialHistorySeeder::class,
        TramaEngagementSeeder::class,
        TramaModerationSeeder::class,
        TramaContactSeeder::class,
    ]);
}
```

Primero se preparan empleados, categorías, etiquetas y usuarios; después pueden cargarse noticias y las relaciones que dependen de ellas. Más adelante se incorporan historial, participación y moderación. Por último se cargan los mensajes de contacto.

Un ejemplo concreto aparece en `TramaEngagementSeeder`. Los comentarios principales se insertan antes que las respuestas porque una respuesta puede guardar en `parent_id` el identificador de otro comentario. Ese comentario padre debe existir antes de insertar la respuesta.

```
$comments = JsonSeedLoader::rows('comments.json');
$main = array_values(array_filter(
    $comments,
    static fn (array $row): bool => $row['parent_id'] === null,
));
$replies = array_values(array_filter(
    $comments,
    static fn (array $row): bool => $row['parent_id'] !== null,
));

foreach (array_chunk($main, 400) as $chunk) {
    DB::table('comments')->insert($chunk);
}
foreach (array_chunk($replies, 400) as $chunk) {
    DB::table('comments')->insert($chunk);
}
```

33.5 JsonSeedLoader: cómo un JSON termina en MySQL

Muchos seeders necesitan realizar exactamente el mismo trabajo: localizar un JSON, leerlo, comprobar que pueda decodificarse correctamente y que contenga una lista, y guardar sus filas. `JsonSeedLoader` existe para resolver esa lógica común y evitar repetirla en cada clase.

El método `rows()` resuelve primero la lectura y la validación del archivo:

```

public static function rows(string $filename): array
{
    $path = base_path('database/seaders/data/'.$filename);

    if (! is_file($path)) {
        throw new RuntimeException("No se encontró el archivo de seed: {$path}");
    }

    try {
        $rows = json_decode(
            (string) file_get_contents($path),
            true,
            512,
            JSON_THROW_ON_ERROR,
        );
    } catch (JsonException $exception) {
        throw new RuntimeException(
            "JSON inválido en {$filename}: {$exception->getMessage()}",
            previous: $exception,
        );
    }

    if (! is_array($rows)) {
        throw new RuntimeException("El archivo {$filename} no contiene una lista válida.");
    }

    return $rows;
}

```

`base_path()` construye la ubicación dentro del proyecto. `is_file()` evita continuar si el archivo no existe. `file_get_contents()` lee su contenido y `json_decode()` lo convierte a arrays de PHP. `JSON_THROW_ON_ERROR` hace que un JSON mal formado produzca un error controlable; el `catch` lo transforma en una excepción con un mensaje que identifica el archivo.

El método `insert()` utiliza esas filas y las guarda en la tabla indicada:

```

public static function insert(
    string $table,
    string $filename,
    int $chunkSize = 500,
    ?callable $transform = null,
): void {
    $rows = self::rows($filename);

    if ($transform !== null) {
        $rows = array_map($transform, $rows);
    }

    foreach (array_chunk($rows, $chunkSize) as $chunk) {
        if ($chunk !== []) {
            DB::table($table)->insert($chunk);
        }
    }
}

```

`$table` indica la tabla de destino y `$filename` el JSON que se debe leer. `$chunkSize` establece cuántas filas se agrupan en cada inserción. `array_chunk()` divide un conjunto grande en lotes más pequeños y cada lote se inserta con `DB::table(...)->insert(...)`.

`$transform` es opcional. Cuando un seeder necesita modificar una fila antes de guardarla, puede entregar una función de transformación. Si no la necesita, `JsonSeedLoader` inserta los datos tal como fueron leídos.

33.6 Ejemplo simple: TramaArticlesSeeder

Una vez entendido JsonSeedLoader, un seeder sencillo queda reducido a indicar qué archivo alimenta a qué tabla y qué tamaño de lote utiliza. TramaArticlesSeeder contiene:

```
public function run(): void
{
    JsonSeedLoader::insert('articles', 'articles.json', 250);
    JsonSeedLoader::insert('article_tag', 'article_tag.json', 500);
    JsonSeedLoader::insert('media_assets', 'media_assets.json', 250);
}
```

La primera línea puede leerse literalmente como: leer articles.json e insertar sus filas en la tabla articles en grupos de hasta 250. Las otras dos líneas hacen lo mismo con la tabla intermedia article_tag y con media_assets.

```
articles.json → JsonSeedLoader → articles
article_tag.json → JsonSeedLoader → article_tag
media_assets.json → JsonSeedLoader → media_assets
```

No todos los seeders son tan directos. TramaRegisteredUsersSeeder, por ejemplo, aplica una transformación antes de insertar los usuarios: deja temporalmente en null varios campos del resumen de moderación cuyas relaciones con comentarios o usuarios todavía no pueden resolverse y reemplaza password por un hash común de prueba. Después de que esas relaciones ya existen, TramaModerationSeeder restaura el resumen de moderación en users. Este recorrido vuelve a mostrar por qué el orden de carga es parte de la lógica del proyecto.

Un hash de contraseña es una representación no legible de la contraseña original. Las cuentas de prueba pueden autenticarse con password, pero la columna password no guarda esa palabra en texto plano.

33.7 TramaIntegritySeeder: comprobar que los datos tengan sentido

Cargar todas las filas no garantiza por sí solo que el escenario sea correcto. TramaIntegritySeeder se ejecuta al final para comprobar cantidades y también relaciones entre tablas.

Una validación simple comprueba, por ejemplo, que existan exactamente 90 noticias:

```
$this->assertSame(
    90,
    DB::table('articles')->count(),
    'noticias',
);
```

Otras comprobaciones no se limitan a contar. El siguiente fragmento busca comentarios asociados a noticias que no están publicadas ni archivadas. El resultado esperado es cero:

```
$nonPublicComments = DB::table('comments')
    ->join('articles', 'articles.id', '=', 'comments.article_id')
    ->whereNotIn('articles.status', ['published', 'archived'])
    ->count();

$this->assertSame(
    0,
    $nonPublicComments,
    'comentarios en noticias no públicas',
);
```

assertSame() es un método privado del propio seeder. Recibe el valor esperado, el valor obtenido y una etiqueta que identifica la comprobación. Si ambos valores no coinciden, lanza una RuntimeException:

```
private function assertSame(int $expected, int $actual, string $label): void
{
    if ($expected !== $actual) {
        throw new RuntimeException(
            "Integridad TRAMA: {$label}. Esperado {$expected}; obtenido {$actual}.",
        );
    }
}
```

Una excepción interrumpe la ejecución normal. Como DatabaseSeeder realiza la carga dentro de una transacción, una inconsistencia detectada al final puede provocar que esa carga no quede aplicada parcialmente.

Entre las reglas que TramaIntegritySeeder comprueba en esta versión se encuentran:

- 2.500 usuarios registrados y 6 empleados;
- 90 noticias y su distribución exacta por estado editorial;
- 6 categorías, 12 etiquetas activas, 15 publicidades y 285 filas de métricas publicitarias;
- ninguna vista ni comentario público en noticias que todavía no son públicas;
- coincidencia entre articles.views y la cantidad real de filas de article_views;
- ausencia de comentarios de cuentas públicas sin verificar;
- y que cada cuenta pública inactiva tenga una revisión administrativa con resolución de bloqueo.

33.8 Resultado de una carga limpia y comprobación en MySQL

Fuente actual de los datos de prueba

database/seeder/data/seed_stats.json registra 64.212 vistas de noticias, 35.298 impresiones y 145 clics publicitarios. RESULTADO_GENERADO.md todavía conserva 19.340 vistas, 35.293 impresiones y 140 clics. En este documento se toma seed_stats.json como resultado actual del conjunto de datos.

Después de una carga limpia, algunos totales permiten comprobar rápidamente que el escenario principal quedó disponible. Estos números describen los datos de prueba preparados por el proyecto; no son límites de TRAMA.

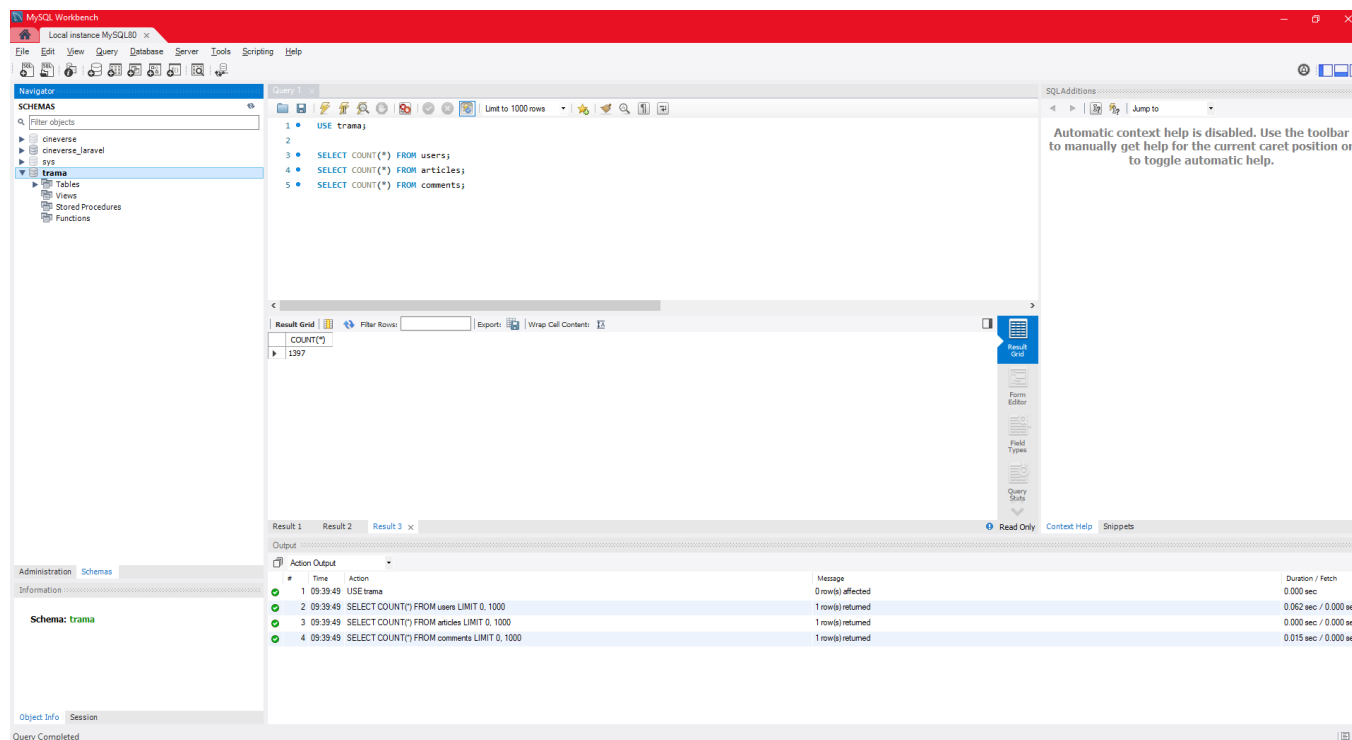
Elemento	Resultado esperado
Usuarios registrados	2.500
Empleados	6
Noticias	90
Vistas de noticias	64.212
Comentarios y respuestas	1.395
Publicidades	15
Registros diarios de métricas publicitarias	285

Las 90 noticias cubren distintos puntos del flujo editorial para que el panel pueda probarse sin crear cada caso manualmente:

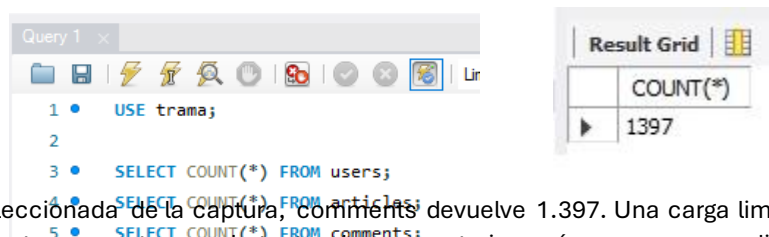
Estado	Cantidad
Publicadas	60
Archivadas	8
Programadas	5
En revisión	7
Devueltas para corrección	5

Estado	Cantidad
Borradores	5

Los mismos registros pueden comprobarse directamente en MySQL. La captura siguiente corresponde a una base local de TRAMA que ya fue utilizada y muestra consultas COUNT(*) sobre users, articles y comments.



Consultas de comprobación sobre una base local de TRAMA ya utilizada.



En la pestaña seleccionada de la captura, comments devuelve 1.397. Una carga limpia contiene 1.395 comentarios y respuestas. La captura muestra, por lo tanto, dos comentarios más que una carga limpia, algo esperable en una base local que ya fue utilizada. Por eso una base en uso puede diferir de los totales deterministas del seeder sin que exista una contradicción.

Reconstruir una base descartable desde cero

php artisan migrate:fresh --seed elimina las tablas de la base configurada, ejecuta nuevamente las migraciones y después carga los seeders. Debe utilizarse únicamente cuando esos datos puedan eliminarse. Si la estructura ya existe y sólo se quieren cargar los datos de prueba, el comando es php artisan db:seed.

33.9 Cuentas de prueba y relación con TramaClock

Las seis cuentas internas cargadas por los seeders permiten entrar al CMS con los roles disponibles sin registrar empleados manualmente. Todas utilizan la contraseña password sólo dentro del entorno local de pruebas.

Nombre	Email	Rol
Marina Ibarra	admin@trama.test	Administrador
Mateo Ledesma	mateo.ledesma@trama.test	Editor

Nombre	Email	Rol
Valentina Ríos	valentina.rios@trama.test	Periodista
Lucía Ferreyra	lucia.ferreyra@trama.test	Periodista
Bruno Salvatierra	bruno.salvatierra@trama.test	Periodista
Camila Torres	camila.torres@trama.test	Periodista

Las cuentas públicas de prueba utilizan la misma contraseña y direcciones terminadas en dominios .test, de modo que los datos tengan formato de correo sin apuntar a casillas reales. Estos valores existen para el entorno de pruebas y no representan una política de contraseñas para usuarios reales.

Las cinco noticias programadas de los datos de prueba incluyen fechas posteriores al 19/07/2026. Sirven para comprobar que el panel guarde, liste y filtre noticias con estado scheduled. Sin embargo, TramaClock tiene el límite temporal explicado en la sección 8 y no avanza automáticamente más allá del 19/07/2026. Por eso no todas pueden alcanzar su fecha de publicación simplemente dejando avanzar ese reloj.

Paso a paso: Carlos ejecuta db:seed y un archivo JSON termina convertido en filas de MySQL

El ejemplo conecta los archivos de datos con las clases PHP que organizan la carga.

Paso	Qué hace la persona y qué ocurre internamente
1	Carlos parte de una base ya migrada y ejecuta php artisan db:seed.
2	Laravel entra por DatabaseSeeder::run(), desactiva el query log para no acumular miles de consultas en memoria y abre la carga general dentro de una transacción.
3	DatabaseSeeder llama a TramaContentSeeder y a los demás seeders en un orden pensado para respetar dependencias entre usuarios, categorías, noticias, comentarios y otros datos.
4	Un seeder concreto solicita filas a JsonSeedLoader indicando un archivo de database/seeders/data/.
5	JsonSeedLoader localiza el JSON, lee el texto y lo convierte a arrays de PHP. Si el archivo no existe o el JSON está mal formado, detiene la carga con un error identificable.
6	El loader inserta las filas, normalmente por bloques, en la tabla correspondiente. Los seeders posteriores pueden usar esos identificadores para crear relaciones.
7	Al final, TramaIntegritySeeder no agrega contenido: revisa cantidades y reglas de coherencia para comprobar que el escenario quedó completo.
8	Si algo falla dentro de la transacción antes de terminar, la carga puede revertirse en lugar de dejar una base de datos poblada sólo a medias.

34. Pruebas automatizadas y comprobaciones técnicas

Los datos de prueba de la sección anterior y las pruebas automatizadas de esta sección cumplen funciones diferentes. Los seeders preparan una base poblada para recorrer el proyecto. Una prueba automatizada, en cambio, es código que prepara una situación concreta, ejecuta una acción y comprueba si el resultado coincide con lo esperado.

TRAMA utiliza PHPUnit para ejecutar las pruebas escritas en PHP. Al conjunto completo de pruebas se lo denomina suite. Estas pruebas sirven para detectar regresiones en comportamientos cubiertos por el código de prueba. Una regresión ocurre cuando una función que antes trabajaba correctamente deja de hacerlo después de un cambio. Las

pruebas automatizadas no sustituyen la comprobación de procesos externos como Vite, SSR, el worker, el Scheduler o los modelos locales.

Una aserción es una comprobación que expresa qué resultado espera una prueba. Por ejemplo, `assertRedirect()` afirma que la respuesta debe ser una redirección y `assertDatabaseHas()` afirma que la base de pruebas debe contener una fila con determinados valores. Si la realidad no coincide con esa expectativa, PHPUnit marca la prueba como fallida.

Qué es una factory y para qué sirve en las pruebas

Una factory es una clase que genera registros de prueba rápidamente. En lugar de depender de toda la base cargada por los seeders, una prueba puede pedir exactamente la cuenta, noticia o relación que necesita para construir su escenario.

Por ejemplo, `User::factory()->create()` crea una cuenta dentro de la base temporal de PHPUnit. Esa fila existe para la prueba que se está ejecutando y no representa una cuenta real del entorno local de TRAMA. `RefreshDatabase` ayuda a que el siguiente escenario no dependa de lo que dejó la prueba anterior.

No confundir: el seeder prepara un escenario amplio y repetible para recorrer el proyecto; la factory fabrica registros puntuales bajo demanda para una prueba. Ambos insertan datos, pero resuelven necesidades distintas.

34.1 Feature y Unit: dos niveles de prueba

Laravel organiza habitualmente las pruebas en `tests/Feature` y `tests/Unit`. Una prueba Feature puede recorrer varias piezas del sistema juntas: por ejemplo, autenticar una cuenta, llamar una ruta, atravesar middleware y controladores, utilizar Eloquent y comprobar qué se guardó en la base de pruebas.

Una prueba Unit busca aislar una pieza pequeña de lógica. En TRAMA la mayor parte de la cobertura útil está en Feature porque muchos comportamientos dependen de permisos, estados editoriales, rutas y relaciones entre tablas.

Los principales grupos presentes en el proyecto son:

Archivo o grupo	Qué parte del proyecto ejercita
<code>tests/Feature/Auth/*</code>	Registro, inicio y cierre de sesión, verificación de correo y recuperación de contraseña.
<code>ArticleIndexQueryServiceTest.php</code>	Visibilidad por rol, filtros, búsqueda, ordenamiento, etiquetas, conteos y paginación del listado de noticias.
<code>EditorialWorkflowTest.php</code>	Flujos públicos y editoriales: comentarios, creación de noticias, slugs, portadas, revisiones, programación y otras reglas.
<code>PublicCommentStageTwoTest.php</code>	Reportes, Me gusta, permisos y límites del sistema de comentarios y respuestas.
<code>TramaClockTest.php</code>	Inicio, avance, pausa, continuidad y límites de TramaClock.
<code>tests/Feature/ExampleTest.php</code>	Comprobación mínima de una respuesta de la aplicación.
<code>tests/Unit/ExampleTest.php</code>	Prueba unitaria mínima del entorno de PHPUnit.

La existencia de un archivo de prueba no demuestra por sí sola que toda la aplicación esté correcta. La suite debe ejecutarse después de los cambios para comprobar que sus expectativas siguen coincidiendo con el comportamiento actual del código.

34.2 El entorno que utiliza PHPUnit

Las pruebas no utilizan necesariamente la misma configuración con la que se abre TRAMA durante el trabajo diario. `phpunit.xml` define un entorno específico para PHPUnit. Entre sus variables se encuentran:

```
<env name="APP_ENV" value="testing"/>
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
<env name="MAIL_MAILER" value="array"/>
<env name="QUEUE_CONNECTION" value="sync"/>
<env name="SESSION_DRIVER" value="array"/>
```

APP_ENV=testing identifica la ejecución como un entorno de pruebas. DB_CONNECTION=sqlite y DB_DATABASE=:memory: hacen que PHPUnit trabaje con una base SQLite temporal en memoria, separada de la base MySQL usada normalmente por la aplicación. MAIL_MAILER=array evita enviar correos reales y SESSION_DRIVER=array mantiene las sesiones de prueba en memoria.

QUEUE_CONNECTION=sync indica que, salvo que una prueba sustituya la cola por una simulación (fake), los trabajos configurados para esa conexión se ejecutan de forma síncrona dentro de la propia ejecución de prueba.

34.3 RefreshDatabase y preparación de cada escenario

Muchas pruebas Feature utilizan RefreshDatabase. Es una característica de Laravel que prepara la base de pruebas para que un escenario no dependa de filas dejadas por otro. Así, cada prueba puede crear solamente los usuarios, noticias o relaciones que necesita.

```
use RefreshDatabase;
```

En ese contexto, métodos como assertDatabaseHas() consultan la base temporal de PHPUnit, no la base MySQL que se utiliza al navegar TRAMA normalmente.

34.4 Ejemplo real: guardar un comentario para moderación

EditorialWorkflowTest.php contiene un ejemplo que reúne autenticación, una petición HTTP, la base de datos y la cola de moderación. Antes de cada prueba del archivo se ejecuta setUp():

```
protected function setUp(): void
{
    parent::setUp();

    Queue::fake([
        ModerateComment::class,
    ]);
}
```

setUp() se ejecuta antes de cada método de prueba de esa clase. Queue::fake() reemplaza temporalmente la ejecución real de ModerateComment. De esta forma la prueba puede comprobar el estado que Laravel deja inmediatamente después de recibir el comentario sin iniciar Node ni los modelos de moderación.

Dentro del método, primero se prepara el usuario y la noticia y después se envía la petición:

```
$reader = User::factory()->create(['name' => 'Sofia Herrera', 'email' => 'sofia@example.com']);
$article = $this->publishedArticle();

$response = $this->actingAs($reader)->post(route('comments.store', $article), [
    'body' => 'Comentario con cuerpo suficiente.',
]);

$response->assertRedirect();
```

User::factory()->create() genera una cuenta sólo para ese escenario. publishedArticle() es un método auxiliar (helper) del propio archivo que prepara una noticia publicada. actingAs(\$reader) hace que la petición se comporte como si esa cuenta hubiera iniciado sesión. route('comments.store', \$article) genera la URL de la ruta y post() simula el envío HTTP hacia esa dirección. \$response conserva la respuesta devuelta por Laravel y assertRedirect() comprueba que sea una redirección.

A continuación se comprueba qué quedó guardado en la base temporal de PHPUnit:

```
$this->assertDatabaseHas('comments', [
    'article_id' => $article->id,
    'user_id' => $reader->id,
    'author_name' => 'Sofia Herrera',
    'author_email' => 'sofia@example.com',
    'status' => 'processing',
    'moderation_revision' => 1,
    'moderation_source' => 'automatic',
]);
```

`assertDatabaseHas()` verifica que exista un comentario con esos valores. En este caso confirma, entre otras cosas, que el estado inicial sea `processing`, que `moderation_revision` valga 1 y que `moderation_source` sea `automatic`.

Esta prueba no comprueba el resultado final de los modelos de moderación porque `ModerateComment` fue sustituido por `Queue::fake()`. La ejecución real del Job pertenece a una comprobación operativa con el worker y los recursos de moderación disponibles.

34.5 Cómo ejecutar las pruebas

La suite completa se inicia desde la raíz del proyecto con:

```
php artisan test
```

`composer.json` también define el atajo `composer test`, que limpia primero la caché de configuración y después ejecuta PHPUnit:

```
composer test
```

Cuando se está trabajando sobre una parte concreta, puede ejecutarse un archivo o un método específico:

```
php artisan test tests/Feature/EditorialWorkflowTest.php

php artisan test \
    --filter=test_authenticated_registered_user_comment_is_saved_as_processing
```

Ejecutar una prueba específica ayuda a aislar el trabajo durante el desarrollo; la suite completa es la que permite comprobar después si un cambio afectó otros comportamientos cubiertos.

34.6 Comprobaciones operativas del entorno

Una prueba automatizada puede comprobar código sin demostrar que todos los procesos necesarios estén activos en una instalación real. Por eso TRAMA también necesita comprobaciones operativas: acciones destinadas a confirmar que cada proceso externo está funcionando en el entorno donde se ejecuta la aplicación.

Comprobación	Resultado esperado	Qué confirma
<code>php artisan serve + abrir /</code>	La portada responde en 127.0.0.1:8000.	Laravel, rutas, configuración y acceso a la base usada por la aplicación.
<code>npm run dev</code>	Vite queda ready y la interfaz carga JavaScript y CSS de desarrollo.	Node.js, NPM y Vite.
<code>php artisan inertia:check-ssr</code>	El servidor SSR responde.	Bundle SSR y proceso <code>inertia:start-ssr</code> .
Enviar un comentario con el worker activo	El comentario sale de <code>processing</code> cuando <code>ModerateComment</code> termina.	Queue, worker, script de Node y recursos de moderación.
Programar una noticia con <code>schedule:work</code> activo	El Scheduler revisa la tarea y puede publicarla cuando corresponde.	Scheduler y comando de publicación programada.

Comprobación	Resultado esperado	Qué confirma
Subir una portada válida	La imagen se procesa y queda guardada en el formato esperado.	TRAMA_NODE_BINARY, Sharp y cadena de procesamiento de imágenes.

Las comprobaciones que requieren inferencia sólo pueden completarse si `resources/models` contiene los archivos ONNX y auxiliares que esperan los scripts. Esos recursos deben estar instalados antes de ejecutar una comprobación operativa que dependa de los modelos.

34.7 Build de producción y revisión final

El build comprueba otra parte distinta del proyecto: que Vite pueda generar los recursos destinados a producción. El comando definido por TRAMA construye el cliente y el bundle SSR:

```
npm run build
```

Cuando termina correctamente, el resultado incluye los archivos compilados para el navegador dentro de `public/build/` y el bundle utilizado por SSR. Esta comprobación no reemplaza las pruebas automatizadas ni una revisión funcional de la aplicación publicada.

Después de instalar o publicar TRAMA conviene revisar al menos:

- abrir la portada, una categoría y una noticia y comprobar la navegación pública;
- iniciar sesión con cuentas internas de distintos roles y confirmar que cada panel respete sus permisos;
- crear o editar una noticia y comprobar que el cambio quede persistido en la base usada por la aplicación;
- enviar un comentario y comprobar su recorrido con el worker activo;
- programar una noticia y confirmar la intervención del Scheduler;
- probar una carga de imagen, el correo y la escritura de logs en el entorno correspondiente;
- y, en producción, confirmar HTTPS, `APP_DEBUG=false`, permisos de escritura necesarios y que `public/` sea la única raíz web expuesta.

34.8 Observaciones técnicas y consistencia del proyecto

Las comprobaciones estáticas permiten revisar la sintaxis de los archivos PHP y de los scripts JavaScript/MJS. Además, existen algunos desajustes entre documentación, pruebas y código ejecutable que conviene conocer antes de interpretar una corrida completa como validación total del proyecto.

Hallazgo	Estado técnico actual
Modelos ONNX	Las cuatro carpetas bajo <code>resources/models</code> existen, pero están vacías. Los scripts están implementados; la inferencia real requiere instalar los modelos y archivos auxiliares esperados.
RESULTADO_GENERADO.md	Conserva cifras antiguas: 19.340 vistas, 35.293 impresiones y 140 clics. <code>seed_stats.json</code> contiene 64.212 vistas, 35.298 impresiones y 145 clics.
docs/empleados-acceso.md	Enumera sólo cinco empleados y datos antiguos. <code>staff.json</code> contiene seis cuentas: Marina Ibarra, Mateo Ledesma, Valentina Ríos, Lucía Ferreyra, Bruno Salvatierra y Camila Torres.
PublicCommentStageTwoTest.php	La prueba <code>test_like_limit_is_thirty_actions_per_minute</code> todavía espera 30 acciones por minuto. <code>CommentController</code> aplica actualmente 10 acciones de Me gusta cada 5 minutos. La documentación funcional sigue el controlador actual.
Carga de imágenes dentro del cuerpo	La ruta <code>admin/media</code> permanece comentada y <code>RichTextEditor</code> no ofrece carga de imágenes internas. Las portadas y demás imágenes administradas por el proyecto siguen sus flujos específicos.

Hallazgo	Estado técnico actual
Dependencias instaladas	vendor ynode_modules se generan durante la instalación mediante composer install y npm install, respectivamente, de acuerdo con el flujo documentado.

Criterio de consistencia documental

Cuando una nota auxiliar del repositorio y el código ejecutable no coinciden, esta documentación describe el comportamiento implementado por las rutas, componentes, Requests, controladores, Services, modelos y datasets actuales; la discrepancia se deja señalada en lugar de ocultarla.

Paso a paso: Carlos ejecuta una prueba y entiende qué está comprobando

Una prueba automatizada prepara su propio escenario, ejecuta una acción concreta y compara el resultado con una expectativa.

Paso	Qué hace la persona y qué ocurre internamente
1	Carlos ejecuta php artisan test o un archivo concreto. PHPUnit carga phpunit.xml y cambia la aplicación al entorno testing.
2	La configuración usa SQLite temporal, mailer en memoria y otras opciones aisladas para no modificar la base MySQL cotidiana ni enviar correos reales.
3	RefreshDatabase prepara la base de cada escenario y las factories crean únicamente las filas que esa prueba necesita.
4	Una prueba Feature puede usar actingAs() para simular una sesión y post(), patch() o get() para atravesar rutas, middleware y controladores.
5	assertDatabaseHas() comprueba el resultado en la base temporal y otras aserciones revisan redirecciones, respuestas, permisos o trabajos despachados.
6	Cuando Queue::fake() está activo, la prueba puede verificar que un Job fue enviado sin ejecutar realmente Node ni los modelos de moderación.
7	Si una expectativa falla, PHPUnit indica qué prueba y qué aserción no coincidieron con el comportamiento actual.
8	Carlos corrige el código o la prueba según corresponda y vuelve a ejecutar primero el caso afectado y después la suite completa para detectar regresiones.